COMP0078 Supervised Learning Lecture Notes

Melodie Li

UCL Department of Computer Science

Mar 2025

Contents

Chapter 1

Introduction to Supervised Learning

1.1 The Supervised Learning Pipeline

- **Collect Data:** Gather a dataset representative of the problem domain.
- **Pre-process Data:** Ensure data is structured and normalized for the learning algorithm.
- **Train a Model:** Optimize the parameters of the model to fit the data.
- **Test and Apply:** Evaluate performance on unseen data to measure generalization.

1.2 Least Squares Regression: Theory and Solution

We consider the problem of minimizing the squared loss function in a linear regression setting. Given a design matrix $X \in \mathbb{R}^{n \times d}$ and response vector $y \in \mathbb{R}^n$, the objective function is:

$$f(w) = \frac{1}{n} \|Xw - y\|^2.$$

1.2.1 Properties of the Squared Loss Function

Lemma 1.2.1 (Gradient of the Squared Loss)**.** *The gradient of the squared loss function is given by:*

$$\nabla f(w) = \frac{2}{n} X^\top (Xw - y).$$

Theorem 1.2.2 (Convexity of the Squared Loss)**.** *The squared loss function $f(w)$ is convex and differentiable. This follows from the Hessian matrix:*

$$H = \frac{2}{n} X^\top X.$$

Since $X^\top X$ is symmetric and positive semidefinite, the function is convex, meaning any stationary point is a global minimum.

1.2.2 Deriving the Optimal Solution

Theorem 1.2.3 (Optimality Condition)**.** *The optimal weight vector $\hat{w}$ satisfies the first-order optimality condition:*

$$\nabla f(w) = 0 \quad \Longrightarrow \quad X^\top (Xw - y) = 0.$$

*This leads to the **normal equation**:*

$$X^\top X \hat{w} = X^\top y.$$

Theorem 1.2.4 (Closed-form Solution for Least Squares Minimization)**.** *If $X^\top X$ is invertible (i.e., X has full column rank, rank$(X) = d$), the **unique minimizer** of the squared loss function is:*

$$\hat{w} = (X^\top X)^{-1} X^\top y.$$

Here,

- *X is the design matrix, where each row corresponds to a feature vector $x_i^\top$,*
- *y is the response vector containing all y_i,*
- *$(X^\top X)^{-1}$ is the inverse of the Gram matrix $X^\top X$, ensuring uniqueness when it exists.*

1.2.3 Example: Simple Linear Regression

Example 1.2.5 (Least Squares for Simple Linear Regression)**.** For $d = 1$ (simple linear regression), the model is:

$$y = w_1 x + b.$$

Let $X = [x_1, \ldots, x_n]^\top \in \mathbb{R}^{n \times 1}$ and $w = [w_1, b]^\top$. The closed-form solution simplifies to:

$$w_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad b = \bar{y} - w_1 \bar{x}.$$

This is the standard formula for simple least squares regression.

1.3 Regularization

Definition 1.3.1 (Ridge Regression)**.** To prevent overfitting or deal with ill-posed problems, a regularization term (ridge regression) is added:

$$\hat{w}_\lambda = \arg\min_w \frac{1}{n}\sum_{i=1}^{n}(w^\top x_i - y_i)^2 + \lambda\|w\|^2.$$

The closed-form solution is:

$$\hat{w}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y,$$

where $\lambda > 0$ controls the trade-off between fit and regularization.

1.4 Test Error Metrics

Evaluating a model's performance on unseen data is a fundamental problem in supervised learning. We introduce key concepts related to expected risk, excess risk, and generalization properties, leading to the bias-variance tradeoff.

1.4.1 Expected Risk and the Optimal Predictor

Definition 1.4.1 (Expected Risk)**.** Given a distribution $\rho(x, y)$, the expected risk of a function f is defined as:

$$E(f) = \mathbb{E}_{(x,y)\sim\rho}[\ell(f(x), y)],$$

where:

- (x, y) are samples from the unknown, underlying data distribution $\rho(x, y)$,
- $x \in \mathcal{X}$ is the input space, and $y \in \mathcal{Y}$ is the output space,
- $\ell(f(x), y)$ is the loss function that quantifies the error of predicting $f(x)$ for the true label y,
- $\rho(x, y)$ is the joint probability distribution over (x, y), which is **typically unknown**.

The expected risk measures the model's average performance on unseen data and is the **central quantity to minimize** in supervised learning.

Definition 1.4.2 (Bayes Estimator)**.** The optimal function f^* that minimizes the **expected risk** is called the **Bayes estimator**:

$$f^*(x) = \arg\min_{z\in\mathcal{Y}} \mathbb{E}_{y\sim\rho(y|x)}[\ell(z, y)],$$

where $\rho(y|x)$ is the conditional distribution of y given x. For squared loss, the Bayes estimator simplifies to:

$$f^*(x) = \mathbb{E}[y \mid x].$$

This represents the best possible prediction at each x.

Remark. *The Bayes estimator achieves the minimal possible expected risk, also called the **Bayes risk**:*

$$E(f^*) = \inf_f E(f).$$

This is an unattainable lower bound for practical models unless the true data distribution $\rho(x, y)$ is known.

Example 1.4.3 (Squared Loss and Bayes Estimator)**.** For the squared loss function $\ell(f(x), y) = (f(x) - y)^2$, the Bayes estimator $f^*(x)$ is the conditional mean:

$$f^*(x) = \mathbb{E}[y \mid x].$$

This minimizes the variance of predictions around $f^*(x)$, making it the optimal predictor under this loss.

1.4.2 Excess Risk and Generalization

Definition 1.4.4 (Excess Risk)**.** In practice, we train a model f_S on a finite dataset S drawn from $\rho(x, y)$. The **excess risk** quantifies the gap between the expected risk of f_S and the optimal risk:

$$\text{Excess Risk} = E(f_S) - E(f^*).$$

where:

- $E(f)$ is the expected risk defined earlier,
- f_S is the **learned model** trained on dataset $S = \{(x_i, y_i)\}_{i=1}^n$,
- f^* is the **Bayes estimator** that minimizes the expected risk.

Remark. *The excess risk captures the combined impact of:*

- *Limited training data (statistical error),*
- *Model capacity (approximation error),*
- *Optimization error (suboptimal training).*

A smaller excess risk indicates a model closer to optimality.

1.4.3 Bias-Variance Tradeoff

Theorem 1.4.5 (Decomposition of Expected Excess Risk)**.** *The expected excess risk for the squared loss can be decomposed as:*

$$\mathbb{E}[(f_S(x) - f^*(x))^2] = (\mathbb{E}[f_S(x)] - f^*(x))^2 + \mathit{Var}[f_S(x)],$$

where:

- $(\mathbb{E}[f_S(x)] - f^*(x))^2$ *is the* ***bias****, which is the expected/average distance from the ground-truth, quantifying systematic errors due to model simplifications (e.g., underfitting),*

- $\mathit{Var}[f_S(x)]$ *is the* ***variance****, which is the actual variance output of models as trained on different datasets, quantifying the model's sensitivity to variations in the training set* S*.*

Remark. *The bias-variance tradeoff illustrates a fundamental challenge in supervised learning:*

- ***High-bias models*** *(e.g., linear regression with few features) systematically deviate from the true function, leading to underfitting.*

- ***High-variance models*** *(e.g., deep neural networks) can fit noise in the training data, leading to overfitting.*

- *The goal is to balance bias and variance to minimize the total error.*

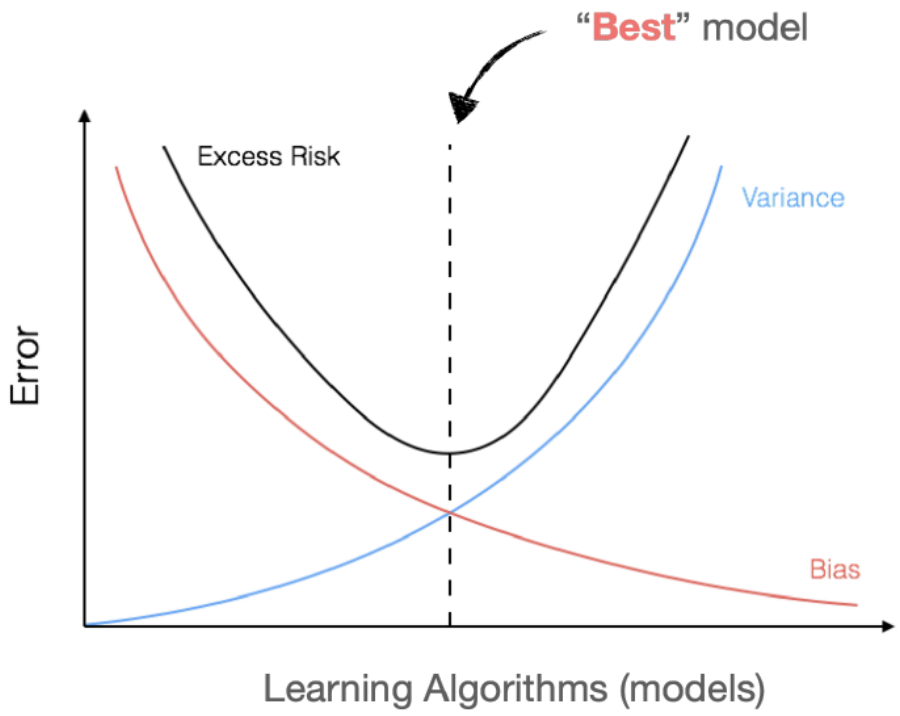

Figure 1.1: Bias-Variance Tradeoff.

Example 1.4.6 (Bias-Variance Tradeoff in Polynomial Regression)**.** Consider fitting a polynomial model to a dataset:

- A **low-degree polynomial (e.g., linear regression)** may underfit, resulting in high bias but low variance.
- A **high-degree polynomial** may overfit, resulting in low bias but high variance.
- The optimal polynomial degree balances these two factors, leading to the best generalization.

1.5 No Free Lunch Theorem

Theorem 1.5.1 (No Free Lunch for Binary Classification)**.** *For any learning algorithm A with a training dataset S of size n, there exists a distribution $\rho(x, y)$ such that:*

- *A Bayes estimator f^* exists with zero error.*

- *The model f_S trained by A satisfies:*

$$P(f_S(x) \neq y) \geq \frac{1}{8}, \quad \textit{with probability at least } \frac{1}{7}.$$

1.5.1 Proof of the Theorem

Proof. 1. **Setup**: Define the input space $\mathcal{X} = \{x_1, \ldots, x_{2n}\}$, with labels equally split across $y = 0$ and $y = 1$. Let $\rho(x, y)$ assign equal probability to all pairs (x, y).

2. **Bayes Estimator** f^*: The Bayes optimal function f^* minimizes expected risk:

$$f^*(x) = \begin{cases} 0, & \text{if } \rho(y = 0 \mid x) > \rho(y = 1 \mid x), \\ 1, & \text{otherwise.} \end{cases}$$

By construction, f^* achieves zero error.

3. **Learning Algorithm** A: Let A train a model f_S on a subset S of size n. f_S has no information about the remaining n unseen points, leading to random predictions for them.

4. **Error Bound**: For the n unseen points, the expected error is at least $1/2$ per point. Concentration inequalities (e.g., Hoeffding's inequality) show that with high probability:

$$P(f_S(x) \neq y) \geq \frac{1}{8}.$$

□

Chapter 2

Feature Maps, Kernels, and Regularization

2.1 Beyond Linear Models: Feature Maps

2.1.1 Recap of Linear Models

Linear models assume a direct relationship between input features and outputs:

- A linear model is defined as:
$$f(x) = w^\top x, \quad x \in \mathbb{R}^d.$$
- Given training data $\{(x_i, y_i)\}_{i=1}^n$, we minimize the squared loss:
$$\hat{w} = \arg\min_w \frac{1}{n} \sum_{i=1}^n (w^\top x_i - y_i)^2.$$
- Using matrix notation, the loss function can be expressed as:
$$L(w) = \frac{1}{n} \|Xw - y\|^2.$$
- The optimal solution satisfies the normal equation:
$$X^\top X \hat{w} = X^\top y.$$
- If $X^\top X$ is invertible, the closed-form solution is:
$$\hat{w} = (X^\top X)^{-1} X^\top y.$$

2.1.2 Challenges of Linear Models

Problem: Linear models work well when x and y have a linear relationship. However, many real-world problems involve complex, non-linear dependencies that cannot be captured by a simple weighted sum of input features.

2.1.3 Polynomial Feature Maps

A natural approach to extending linear models is to introduce polynomial (non-linear) transformations of the input features.

Definition 2.1.1 (Polynomial Feature Map)**.** Given an input $x \in \mathbb{R}$, a polynomial of degree p is defined as:

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_p x^p.$$

This can be rewritten in vector form using a feature mapping function:

$$f(x) = w^\top \phi(x),$$

where the polynomial feature map is:

$$\phi(x) = (1, x, x^2, \ldots, x^p)^\top \in \mathbb{R}^{p+1}.$$

which is like an "embedded space".

Remark. *Despite the non-linearity in x, the optimization problem remains **linear** in terms of the parameters w. That is, learning the coefficients $a_0, \ldots, a_p$ is still a linear regression problem.*

2.1.4 Polynomial Least Squares Estimation

When learning a polynomial function to fit a dataset, we solve the optimization problem:

$$\min_{a_0,\ldots,a_p} \frac{1}{n} \sum_{i=1}^{n} (a_0 + a_1 x_i + a_2 x_i^2 + \cdots + a_p x_i^p - y_i)^2.$$

Key observation: The transformed terms x_i^q for $q = 0, \ldots, p$ are just numerical values. The problem is still a linear regression problem in the parameters a_q, making it solvable using the normal equation.

2.1.5 Feature Maps and Generalization

Instead of working with polynomial transformations explicitly, we can generalize this approach to **feature maps**.

Definition 2.1.2 (Feature Map)**.** A feature map is a function:

$$\phi : \mathcal{X} \to \mathbb{R}^D,$$

which transforms input data into a higher-dimensional space, allowing linear models to fit non-linear relationships.

For example, defining $\phi(x)$ as the polynomial feature map:

$$\phi(x) = (1, x, x^2, \ldots, x^p)^\top \in \mathbb{R}^{p+1}$$

allows us to rewrite the polynomial regression function as:

$$f(x) = w^\top \phi(x).$$

Remark. *This formulation shows that polynomial regression is **a linear model in the transformed feature space** $\phi(\mathcal{X})$, even though it is non-linear in the original space $\mathcal{X}$.*

2.1.6 Optimization in Feature Space

Given training data $\{(x_i, y_i)\}_{i=1}^n$, where x_i is mapped to a higher-dimensional space via $\phi(x_i)$, the least squares problem becomes:

$$\min_w \frac{1}{n} \sum_{i=1}^n (w^\top \phi(x_i) - y_i)^2.$$

Defining $\Phi \in \mathbb{R}^{n \times p}$ as the feature matrix where each row is the embedded point $\phi(x_i)^\top$, we express the problem as:

$$L(w) = \frac{1}{n} \|\Phi w - y\|^2.$$

The optimal parameters satisfy the normal equation:

$$\Phi^\top \Phi \hat{w} = \Phi^\top y.$$

Solution: If $\Phi^\top \Phi$ is invertible, we obtain:

$$\hat{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y.$$

Remark. *This is the same optimization procedure as in the linear case, except we have replaced x with its transformed feature representation $\phi(x)$.*

2.1.7 Generalized Feature Maps: Basis Functions

The concept of feature maps extends beyond polynomials to arbitrary transformations.

- **Polynomial feature maps**: Transform an input x into polynomial terms.
- **Basis functions**: More generally, we can define $\phi(x)$ using any set of functions that capture useful properties of the data.

Example 2.1.3 (Polynomial Feature Map for Multivariate Inputs)**.** For $x \in \mathbb{R}^d$, we can define a polynomial feature map of degree p as:

$$\phi(x) = (x_1^{j_1} x_2^{j_2} \dots x_d^{j_d})_{(j_1,\dots,j_d)\in J_p},$$

where $J_p = \{(j_1, \dots, j_d) \mid \sum_{i=1}^d j_i = p\}$ is the index set for polynomial terms up to degree p.

Remark. *More generally, we can define feature maps $\phi : \mathcal{X} \to \mathbb{R}^D$ that map inputs into some higher-dimensional **latent space**, allowing linear methods to capture non-linearity.*

2.1.8 Generalized Feature Maps

Feature maps can be generalized to arbitrary transformations $\phi : \mathbb{R}^d \to \mathbb{R}^D$. For example:

$$\phi(x) = \left(x_1^{j_1} x_2^{j_2} \cdots x_d^{j_d}\right)_{j_1+j_2+\cdots+j_d\le p}.$$

2.2 Computational Considerations

When solving the squared loss minimization problem, we must consider the efficiency of computing the optimal solution. The computational cost depends on factors such as the number of training samples n and the feature space dimension D.

2.2.1 Efficiency of Least Squares Minimization

Question: How efficient is the computation of $\hat{w}$?

In other words, how much time and memory would it take to obtain $\hat{w}$?

The answer depends on:

- The computational power of the machine,
- The training size n,
- The number of input dimensions D.

2.2.2 Empirical Timing Analysis

From experimental observations (e.g., matrix inversion on a specific machine), the cost of solving the normal equations increases significantly **as matrix size grows**. The empirical relationship between matrix size and inversion time suggests cubic growth in complexity.

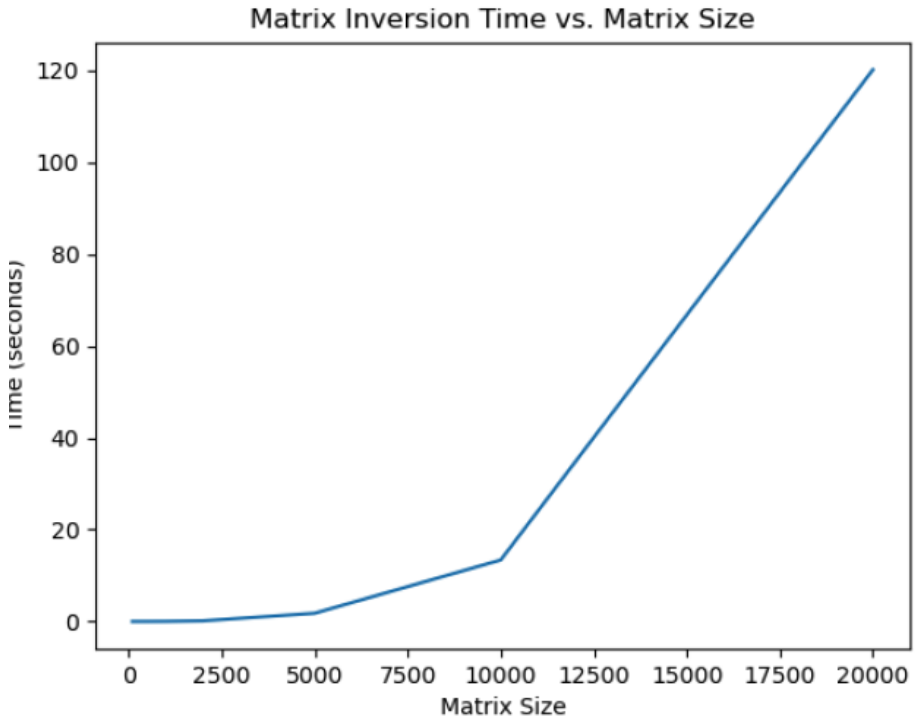

Figure 2.1: Empirical runtime of matrix inversion as a function of matrix size.

2.2.3 Big-O Complexity Analysis

A more rigorous approach to understanding computational cost involves Big-O notation.

Question: What is the computational complexity in Big-O notation?

Answer:

- **Time Complexity:** $O(D^3 + nD^2)$.
- **Memory Complexity:** $O(D^2 + nD)$.

These results are based on standard numerical methods implemented in BLAS/LAPACK for matrix operations.

2.2.4 Understanding Big-O Notation

Big-O notation describes the asymptotic behavior of computational costs.

Definition 2.2.1 (Big-O Notation)**.** A function $f(x)$ is $O(g(x))$ if there exist constants $C > 0$ and x_0 such that for any $x > x_0$:

$$|f(x)| \leq Cg(x).$$

Key Observations:

- Two functions with the same Big-O complexity might have vastly different runtime due to different constant factors C (one might have a much larger constant).
- The activating point x_0 at which the behavior becomes apparent may vary between methods.

Implication: While two methods may have the same asymptotic complexity, they can differ significantly in practice in terms of execution time and memory usage.

2.2.5 Feature Space Growth and Its Impact

The feature space dimension D can grow rapidly when applying polynomial feature maps.

Question: How large can we make D in practice?

Ideally, we would like D to be as large as possible. However, the computational cost grows **cubically** in D, which limits its practical size.

Definition 2.2.2 (Feature Dimension Growth)**.** For polynomial feature maps, the dimension D is given by:

$$D = \binom{d+p}{p},$$

where:

- d is the original input dimension,
- p is the polynomial degree.

Example Calculations: If $n = 1000$ is fixed, then:

- $d = 1, p = 3 \Rightarrow D = 4$,
- $d = 3, p = 5 \Rightarrow D = 56$,
- $d = 5, p = 10 \Rightarrow D = 3003$,
- $d = 10, p = 15 \Rightarrow D = 184756$.

Takeaway: Even for moderate d and p, the feature space dimension D can become extremely large, making computations expensive.

2.2.6 Breakdown of Computational Cost

The overall complexity $O(D^3 + nD^2)$ arises from two key matrix operations:

- **Matrix Inversion:** $O(D^3)$ for computing $(\Phi^\top \Phi)^{-1}$.
- **Matrix Multiplication:** $O(nD^2)$ for computing $\Phi^\top \Phi$.

This can be visualized as follows:

$$\hat{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y.$$

where:

- $\Phi^\top \Phi$ as multiplication is $(D * n) * (n * D)$ so complexity $O(nD^2)$.
- $(\Phi^\top \Phi)^{-1}$ as Inversion has complexity $O(D^3)$

2.2.7 Reducing Computational Costs

Given the high computational cost, we ask:

Is there a way to compute $\hat{w}$ faster?

Potential solutions include:

- Using **iterative methods** such as gradient descent instead of direct inversion.
- Exploiting **sparsity** in Φ to speed up matrix operations.
- Using **approximate matrix factorization techniques** (e.g., SVD, Cholesky decomposition).

Summary:

- The computational cost of least squares minimization is dominated by $O(D^3)$ inversion and $O(nD^2)$ multiplication.
- The feature space dimension D grows rapidly for polynomial expansions, limiting practical applications.
- We need alternative optimization techniques to help reduce computational burdens.

2.3 Dual Solution and Computational Benefits

2.3.1 Alternative Formulation for Squared Loss Minimization

We seek an alternative characterization of the least squares solution that is computationally efficient in certain regimes. Starting from the standard closed-form solution:

$$\hat{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y,$$

we explore reformulations that leverage linear algebra tools such as Singular Value Decomposition (SVD).

2.3.2 Singular Value Decomposition (SVD)

A key technique for analyzing the structure of Φ is the Singular Value Decomposition (SVD), given by:

$$\Phi = U \Sigma V^\top.$$

where:

- $r = \min(n, D)$ is the rank of Φ,
- $U \in \mathbb{R}^{n \times r}$ and $V \in \mathbb{R}^{D \times r}$ have **orthonormal** columns:

$$U^\top U = I, \quad V^\top V = I.$$

- $\Sigma \in \mathbb{R}^{r \times r}$ is a **diagonal** matrix with only non-negative entries, containing the singular values.
- The diagonal vector $\sigma = diag(\Sigma)$ is the spectrum of Σ.
- The number of non-zero entries in σ corresponds to the rank of Φ.

Using the SVD, we can express the least squares solution in an equivalent but computationally insightful form.

2.3.3 Rewriting the Least Squares Solution Using SVD

Substituting $\Phi = U \Sigma V^\top$ into the least squares formula:

$$\hat{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y.$$

- Since $\Phi^\top \Phi = V \Sigma^\top U^\top U \Sigma V^\top = V \Sigma^2 V^\top$, we obtain:

$$(\Phi^\top \Phi)^{-1} = (V \Sigma^2 V^\top)^{-1} = V \Sigma^{-2} V^\top.$$

- Substituting this back, we get:
$$\hat{w} = V\Sigma^{-2}V^\top \Phi^\top y.$$
- Recognizing that $U^\top y$ is a transformed version of the target labels, we simplify:
$$\hat{w} = V\Sigma^{-1}U^\top y.$$

Key Observation: The solution can be rewritten in terms of U, Σ, V, leading to an alternative characterization that enables further simplifications.

2.3.4 Dual Formulation

Rather than solving for w explicitly, we introduce the **dual variable** α, defined as:

$$\alpha = (\Phi\Phi^\top)^{-1}y.$$

Using this, we rewrite the solution as:

$$\hat{w} = \Phi^\top \alpha.$$

This formulation is known as the **dual solution**, where the key computation involves inverting $K = \Phi\Phi^\top$, the **Gram matrix**.

Theorem 2.3.1 (Dual Solution for Squared Loss)**.** *The optimal weights can be expressed as:*

$$\hat{w} = \Phi^\top(\Phi\Phi^\top)^{-1}y.$$

where $K = \Phi\Phi^\top$ is the Gram matrix.

2.3.5 Comparison of Computational Costs

The dual formulation alters the computational cost of solving the least squares problem. Comparing the two approaches:

- **Primal Formulation (from before):**
$$\hat{w} = (\Phi^\top\Phi)^{-1}\Phi^\top y, \quad O(D^3 + nD^2).$$
- **Dual Formulation:**
$$\hat{w} = \Phi^\top(\Phi\Phi^\top)^{-1}y, \quad O(n^3 + n^2D).$$

Key Insight: The dual formulation is computationally advantageous when $n < D$, since inverting an $n \times n$ matrix (instead of a $D \times D$ matrix) reduces complexity. It's critical when we consider features space that are infinite dimensional.

2.3.6 Interpreting the Dual Formulation

At first glance, the dual and primal formulations may appear similar. However, they involve different matrix operations:

$$\hat{w} = \Phi^\top (\Phi\Phi^\top)^{-1} y \quad \text{vs} \quad \hat{w} = (\Phi^\top\Phi)^{-1}\Phi^\top y.$$

- In the primal approach, we invert $\Phi^\top\Phi$, a $D \times D$ matrix.
- In the dual approach, we invert $\Phi\Phi^\top$, an $n \times n$ matrix.

Since $n < D$ in many cases, the dual formulation is preferred **when the number of samples n is small relative to the feature space dimension D**.

2.3.7 Computational Benefits of the Dual Formulation

- When $n < D$, the dual formulation is **faster** since the Gram matrix $K = \Phi\Phi^\top$ is only $n \times n$, leading to $O(n^3)$ inversion instead of $O(D^3)$.
- This is especially useful when working with **high-dimensional feature spaces, such as in kernel methods**, where D can be infinite-dimensional.

2.3.8 Connection to the Pseudoinverse

The dual formulation is closely related to the **Moore-Penrose pseudoinverse**. The SVD of the pseudoinverse is given by:

$$\Phi^\dagger = V\Sigma^\dagger U^\top.$$

where $\Sigma^\dagger$ is the diagonal matrix with entries:

$$(\Sigma^\dagger)_{ij} = \begin{cases} \frac{1}{\Sigma_{ij}}, & \text{if } \Sigma_{ij} \neq 0, \\ 0, & \text{otherwise.} \end{cases}$$

Using the pseudoinverse, we can rewrite the derivation of dual formation as:

$$\hat{w} = \Phi^\dagger y = V\Sigma^\dagger U^\top y.$$

This confirms that the dual formulation naturally arises from the properties of the pseudoinverse.

2.3.9 Summary

- The standard least squares solution involves inverting $\Phi^\top\Phi$, leading to a cost of $O(D^3+nD^2)$.
- The dual formulation reformulates the problem in terms of $\alpha = (\Phi\Phi^\top)^{-1}y$, requiring inversion of an $n \times n$ matrix, **reducing the cost** to $O(n^3 + n^2D)$.
- The dual approach is preferable when $n < D$, as it avoids **direct inversion of a high-dimensional matrix**.
- The SVD provides an insightful way to decompose the least squares solution, highlighting the role of singular values in solving for w.
- The pseudoinverse provides a rigorous way to generalize the solution, further reinforcing the connection between the primal and dual formulations.

2.4 Implicit Feature Maps and Kernels

2.4.1 Inner Products and Kernel Trick

One key observation in the dual formulation is that computations involving the feature map $\phi(x)$ only require **inner products** of the form:

$$k(x, x') = \phi(x)^\top\phi(x').$$

Theorem 2.4.1 (Kernel Trick)**.** *For any feature map ϕ, the inner product can be computed using a kernel function $k(x, x')$, **without explicitly constructing** $\phi(x)$:*

$$k(x, x') = \phi(x)^\top\phi(x').$$

This allows us to implicitly operate in high-dimensional feature spaces without explicitly **computing feature transformations**, making learning more efficient.

2.4.2 Decoupling Training and Evaluation

In the dual formulation, the solution to the least squares problem is:

$$\hat{w} = \Phi^\top\alpha, \quad \text{where} \quad \alpha = (\Phi\Phi^\top)^{-1}y.$$

This decouples:

- **Training:** Requires computing $\alpha = (\Phi\Phi^\top)^{-1}y$, which involves computation intensive inverting an $n \times n$ matrix.

- **Evaluation:** Given a new input x, we compute:

$$\hat{f}(x) = \phi(x)^\top \hat{w} = \phi(x)^\top \Phi^\top \alpha.$$

2.4.3 Kernel Representation of Inner Products

A key observation is that the Gram matrix:

$$K_{ij} = \phi(x_i)^\top \phi(x_j)$$

contains all required inner products. Moreover, for a new input x, the evaluation vector is defined as:

$$v(x) = \left[\phi(x)^\top \phi(x_1), \ldots, \phi(x)^\top \phi(x_n)\right]^\top.$$

Thus, we can evaluate the learned function using:

$$\hat{f}(x) = v(x)^\top \alpha.$$

Key Insight: All operations required for training and evaluation depend only on inner products $\phi(x)^\top \phi(x')$, meaning we never need to explicitly compute $\phi(x)$.

2.4.4 Example: Polynomial Kernel

Consider the polynomial feature map with $D = 3$:

$$\phi(x) = \left[x_1^2, \sqrt{2}x_1 x_2, x_2^2\right]^\top.$$

Instead of explicitly computing the transformed vectors, we rewrite as one sum and one product:

$$\phi(x)^\top \phi(x') = x_1^2 x_1'^2 + 2x_1 x_2 x_1' x_2' + x_2^2 x_2'^2 = (a+b)^2.$$

This can be rewritten as:

$$(x^\top x' + 1)^p,$$

which is known as the **polynomial kernel**.

Definition 2.4.2 (Polynomial Kernel)**.** For a polynomial feature map of **degree** p, the kernel function is:

$$k(x, x') = (x^\top x' + 1)^p.$$

2.4.5 Computational Advantage of Kernels

- The feature dimension D grows as $O(p^d)$, which is often prohibitively large.
- Using only inner products, we avoid explicit computation of $\phi(x)$, reducing complexity.
- Evaluating $(x^\top x'+1)^p$ requires only $O(d)$ operations, instead of constructing a high-dimensional feature vector.

This leads to a computational cost reduction:

- **Primal:** $O(p^{3d} + np^{2d})$
- **Dual:** $O(n^3 + p^d n^2)$
- **Inner Product + Dual:** $O(n^3 + dn^2)$

Thus, the combination of the **dual formulation and kernel methods** provides significant computational savings.

2.4.6 Generalizing to Kernels

Definition 2.4.3 (Kernel Function)**.** A kernel function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ implicitly defines a feature map $\phi(x)$ such that:

$$k(x, x') = \phi(x)^\top \phi(x').$$

where the calculation does not require explicit knowledge of $\phi(x)$

Remark. *If evaluating $k(x, x')$ can be done in $O(k)$ operations, it follows that learning $\hat{w}$ requires $O(n^3 + k * n^2)$ with the inner product + dual formulations.*

Example 2.4.4 (Gaussian (RBF) Kernel)**.** The Gaussian kernel, corresponding to an infinite-dimensional feature space, is given by:

$$k(x, x') = e^{-\frac{\|x-x'\|^2}{2\sigma^2}}.$$

2.4.7 Key Question: Existence of Feature Maps

Rather than designing $\phi(x)$ explicitly, we often ask:

Can we find a kernel function $k(x, x')$ whose associated feature map $\phi(x)$ we never need to compute?

A fundamental question in kernel methods is:

When does a function $k(x, x')$ correspond to the inner product of some feature map $\phi(x)$?

This leads to the study of **Mercer's theorem**, which characterizes valid kernel functions.

2.4.8 Intuition: Inner Product as Similarity Measures

The kernel function measures similarity between inputs. If $k(x, x') = \phi(x)^\top \phi(x')$ is:

- Positive and large, x and x' are considered similar according to ϕ.
- Close to zero. x and x' are unrelated (not very similar).
- Negative and Large, x and x' are oppositely-correlated.

Key Observation: Often, it is easier to define a similarity function $k(x, x')$ rather than explicitly constructing $\phi(x)$. However, how do we find a set of rules to construct valid kernels $k(x, x')$ that correspond to some implicit feature map $\phi(x)$?

2.5 Positive Definite Kernels

2.5.1 Positive Semi-definite Gram Matrices

We seek an **alternative characterization** of an inner product function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ that does **not require explicit knowledge** of the feature map ϕ.

Definition 2.5.1 (Positive Semidefinite Matrix)**.** A matrix $K \in \mathbb{R}^{n \times n}$ is **positive semidefinite (PSD)** if it is symmetric and satisfies:

$$v^\top K v \geq 0, \quad \forall v \in \mathbb{R}^n.$$

Equivalently, K is PSD if its singular value decomposition (SVD) takes the form:

$$K = U \Sigma U^\top,$$

where Σ is a diagonal matrix with non-negative entries.

Definition 2.5.2 (Recovering Inner Products)**.** Given a function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, consider a dataset $(x_i)_{i=1}^n$ and define any Gram matrix $K \in \mathbb{R}^{n \times n}$ with entries:

$$K_{ij} = k(x_i, x_j).$$

Then, if k is an inner product kernel, then K is positive semi-definite (PSD), i.e., $K \succeq 0$.

Key Question: Is the converse true? That is, if K is PSD, does it necessarily correspond to an inner product kernel?

2.5.2 The Need for Positive Definite Kernels

While $k(x, x')$ being an inner product implies K is PSD, the reverse is not necessarily true. However, defining an estimator using a PSD matrix is still possible:

$$\hat{f}(x) = v(x)^\top \alpha, \quad \text{where} \quad \alpha = K^\dagger y,$$

with $v(x) = (k(x, x_1), \ldots, k(x, x_n))^\top$, the evaluation vector.

However, this approach introduces:

- **Numerical** issues in practice,
- **Methodological** issues in algorithms,
- **Statistical** issues in theoretical guarantees.

Thus, requiring $k(x, x')$ to be PSD ensures stable computations.

Definition 2.5.3 (Positive Definite Kernel)**.** We say that $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ or $k(x, x')$ is a **positive definite kernel** if, for any dataset $(x_i)_{i=1}^n$, the corresponding Gram matrix K with entries $K_{ij} = k(x_i, x_j)$ is positive semi-definite (PSD).

But is this enough for k to be an inner product? (when k is PD + Gram matrix is PSD)?

2.5.3 Inner Product vs. Positive Definite Kernel

Theorem 2.5.4. *k is an inner product $\iff$ k is a positive definite kernel.*

In Depth. What can be shown is that there exist:

- A **Hilbert** space $\mathcal{H}$ with **inner product** $\langle \cdot, \cdot \rangle_{\mathcal{H}}$.
- A **feature map** $\phi : \mathcal{X} \to \mathcal{H}$.

Such that:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}, \quad \forall x, x' \in \mathcal{X}.$$

This result establishes the connection between kernels and inner products in Hilbert spaces: **A function $k(x, x')$ is a kernel if and only if it corresponds to an inner product in some Hilbert space.**

Sketch. If $k(x, x')$ is an inner product, then for any finite dataset, the Gram matrix K is PSD. Conversely, if $k(x, x')$ generates a PSD Gram matrix for any dataset, then **there exists a Hilbert space** $\mathcal{H}$ and a feature map $\phi : \mathcal{X} \to \mathcal{H}$ such that:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}.$$

□

2.5.4 Hilbert Spaces and Kernels

A Hilbert space $\mathcal{H}$ is a complete **inner product space** with properties:

- Equipped with an inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}}$,
- Norm defined as $\|x\|_{\mathcal{H}}^2 = \langle x, x \rangle_{\mathcal{H}}$,
- Complete with respect to $\| \cdot \|_{\mathcal{H}}$.

Theorem 2.5.5 (Hilbert Space Representation)**.** *If $k(x, x')$ is a positive definite kernel, then there exists a Hilbert space $\mathcal{H}$ and a feature map $\phi : \mathcal{X} \to \mathcal{H}$ such that:*

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}.$$

Recall. An **inner product** $\langle \cdot, \cdot \rangle$ over a (real) vector space $\mathcal{V}$ is a function

$$\langle \cdot, \cdot \rangle : \mathcal{V} \times \mathcal{V} \to \mathbb{R}$$

such that $\forall v, v', v'' \in \mathcal{V}$ and $\alpha, \beta \in \mathbb{R}$:

- **Symmetric** $\langle v, v' \rangle = \langle v', v \rangle$
- **(Bi)linear** $\langle \alpha v + \beta v', v'' \rangle = \alpha \langle v, v'' \rangle + \beta \langle v', v'' \rangle$
- **Positive definite** $\langle v, v \rangle > 0$

2.5.5 Infinite-Dimensional Feature Spaces

If the Hilbert space is finite-dimensional ($\mathcal{H} \cong \mathbb{R}^D$), then feature maps can be **explicitly computed**. However, for kernels like the Gaussian kernel:

$$k(x, z) = e^{-\|x-z\|^2/2\sigma^2},$$

the associated feature space is **infinite-dimensional**. Using primal or (naive) dual approaches would be computationally infeasible when D is extremely large, but using **Kernel + Dual** methods reduces the complexity to $O(n^3 + dn^2)$.

2.5.6 Gaussian Kernel Feature Map

For simplicity, when the Gaussian kernel with $\sigma = 1$:

$$k(x, z) = e^{-(x-z)^2/2} = e^{-(x^2/2 - xz + z^2/2)} = e^{-x^2/2} e^{xz} e^{-z^2/2}$$

we can expand the squared term and use the Taylor series expansion for the exponential term:

$$e^{xz} = \sum_{m=1}^{\infty} \frac{(xz)^m}{m!} = \sum_{m=1}^{\infty} \frac{x^m}{\sqrt{m!}} \frac{z^m}{\sqrt{m!}}.$$

Let ℓ^2 denote the space of all (real) square-summable sequences.
Recall. ℓ^2 is a Hilbert space with

$$\langle (a_m)_{m=1}^{+\infty}, (b_m)_{m=1}^{+\infty} \rangle = \sum_{m=1}^{+\infty} a_m b_m.$$

Define feature map $\phi : \mathbb{R} \to \ell^2$ the map such that $\forall x \in \mathbb{R}$

$$\phi(x) = \left(e^{-x^2/2} \frac{x^m}{\sqrt{m}} \right)_{m=1}^{+\infty}.$$

Then,

$$\langle \phi(x), \phi(z) \rangle_{\ell^2} = \sum_{m=1}^{+\infty} e^{-x^2/2} \frac{x^m}{\sqrt{m}} \frac{z^m}{\sqrt{m}} e^{-z^2/2} = e^{-(x-z)^2/2}.$$

where we've proved that Gaussian kernel is indeed a kernel by explicitly finding the Hilbert space and the feature map. However, as stated before, we don't need to know the feature map associated to a kernel!

2.5.7 Rules for Constructing Kernels

Theorem 2.5.6. *Let $k_1, k_2 : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be two positive definite kernels, then:*

- *αk_1 is a kernel for any $\alpha \geq 0$.*
- ***Sum:** $k_1 + k_2$ is a kernel.*
- ***Point-wise product:** $k(x, x') = k_1(x, x') k_2(x, x')$ is a kernel.*
- ***Composition:** $k_1(\varphi(z), \varphi(z'))$ is a kernel for any mapping $\varphi : \mathcal{Z} \to \mathcal{X}$.*

2.5.8 Examples of Kernels

Polynomial Kernel. The function

$$k(x, x') = (x^\top x' + \alpha)^p$$

with $\alpha \geq 0$ is a kernel because:

- $k_1(x, x') = x^\top x'$ is a kernel.
- A constant function $k_2(x, x') = \alpha$ is a kernel.
- $k_3 = k_1 + k_2$ is a kernel.
- $k = k_3^p$ is a power of a kernel.

Polynomials OF Kernels. The function

$$k(x, x') = \sum_{m=1}^{p} \alpha_m \widetilde{k}(x, x')^m$$

is a kernel as it is a sum of powers of kernels.

Series of Kernels. Taking the limit $m \to +\infty$ in the above construction, if the series converges, then it is still a kernel.

For example, let $\alpha_m = \frac{1}{m!}$ and $\widetilde{k}(x, x') = (x^\top x')^m$, the polynomial kernel, we obtain

$$k(x, x') = \sum_{m=1}^{+\infty} \frac{(x^\top x')^m}{m!} = e^{x^\top x'}.$$

This results in the exponential kernel.

Gaussian Kernels. The Gaussian kernel is given by

$$k(x, x') = e^{-\|x-x'\|^2}.$$

Expanding the square, we rewrite it as:

$$k(x, x') = e^{-\left(\|x\|^2/2 - x^\top x' + \|x'\|^2/2\right)}$$

which can be rewritten using the point-wise product rule as:

$$k(x, x') = e^{-\|x\|^2} e^{x^\top x'} e^{-\|x'\|^2}.$$

where:

- $k_1(x, x') = \phi_1(x)\phi(x')$ is a kernel with feature map $\phi(x) = e^{-\|x\|^2}$.
- k_2 is the exponential kernel.

Hence, $k(x, x')$ is a valid kernel.

2.5.9 Further with Kernels

We barely scratched the **surface** of what can be studied about kernels.

Given the scope of this module, we will limit ourselves to the **algorithmic perspective** on kernels and corresponding Hilbert spaces that we introduced in this class.

2.5.10 Conclusion

The key insight from this discussion is that kernel methods allow us to work in infinite-dimensional feature spaces without explicitly computing $\phi(x)$. Instead, all computations rely on inner products:

$$k(x, x') = \phi(x)^\top \phi(x').$$

This significantly reduces computational cost and enables efficient learning in high-dimensional spaces.

2.6 Regularization

2.6.1 The Problem with Large Feature Spaces

Kernels allow us to efficiently work with large (or even infinite-dimensional) feature spaces. However, there are potential problems:

- The number of training points, n, is **finite**. We cannot have an infinite dataset/observations.
- n is potentially **smaller** than the feature space dimension D (e.g., polynomial or Gaussian kernels).

2.6.2 Fitting Linear Models

A linear function $f : \mathbb{R}^D \to \mathbb{R}$ is uniquely determined by $n = D$ points. If $n < D$, multiple functions satisfy the same constraints. This corresponds to solving the linear system:

$$\hat{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y$$

where:

- corresponding to solving $\Phi \hat{w} = y$.
- There are n equations (one per point).
- There are D variables.

If $n < D$, the system is under-determined, leading to multiple solutions and potential overfitting.

2.6.3 Overfitting

Overfitting occurs when the model complexity is too high relative to the available data, leading to a poor generalization. Below is an illustration of overfitting using polynomial regression:

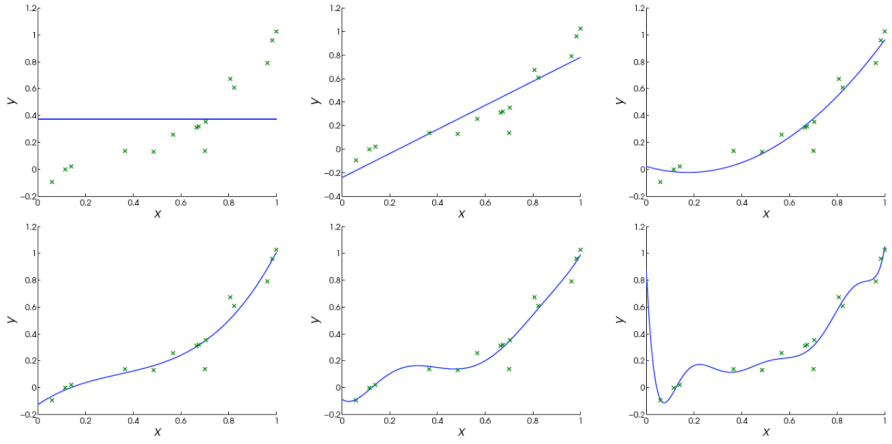

Figure 2.2: The polynomial degree p increases from 0 to 5, showing a transition from underfitting to overfitting.

2.6.4 How Does Overfitting Arise?

Mathematically, overfitting arises when $n < D$, making $\Phi^\top \Phi$ **non-invertible**. However, solutions exist:

- Use the **dual formulation** ($n \times n$):

$$\hat{w} = \Phi^\top (\Phi \Phi^\top)^{-1} y.$$

- Use the **pseudoinverse** instead of the inverse $\Phi\Phi^\top$.

However, the pseudoinverse is not always ideal due to numerical instabilities.

2.6.5 Instability of the Pseudoinverse

Using the singular value decomposition (SVD):

$$\Phi = U \Sigma V^\top,$$

we obtain the squared loss minimizer/ least squares solution:

$$\hat{w} = V\Sigma^{\dagger}U^{\top}y.$$

If $\sigma_n = 10^{-p}$ (smallest singular value), then small input perturbations along corresponding singular directions get amplified by 10^p, leading to instability.

2.6.6 Regularization: A Fix for Ill-posed Problems

Idea: Discard small singular values using a threshold $\lambda > 0$:

$$P_\lambda(\Sigma)_{ii} = \begin{cases} \frac{1}{\Sigma_{ii}}, & \text{if } \Sigma_{ii} > \lambda \\ 0, & \text{otherwise} \end{cases}.$$

Then, the regularized solution becomes:

$$\hat{w}_\lambda = VP_\lambda(\Sigma)^t U^{\top}y.$$

2.6.7 Tikhonov Regularization

Definition 2.6.1 (Tikhonov Regularization)**.** The regularized least squares solution is:

$$\hat{w}_\lambda = (\Phi^{\top}\Phi + \lambda I)^{-1}\Phi^{\top}y.$$

Alternatively, this solution arises from minimizing the following objective function:

$$\hat{w}_\lambda = \arg\min_{w\in\mathbb{R}^D} \frac{1}{n}\sum_{i=1}^{n}(w^{\top}\phi(x_i) - y_i)^2 + \lambda\|w\|^2.$$

2.6.8 Bias-Variance Tradeoff

Regularization has tradeoffs:

- **Bias:** Regularization increases bias by moving the solution away from training data:

$$\text{Bias}(\hat{w}_\lambda, f_*) = \mathbb{E}_x[(f_*(x) - \hat{w}_\lambda^{\top}\phi(x))^2].$$

- **Variance:** Regularization reduces variance, stabilizing predictions across different datasets:

$$\text{Var}(\hat{w}_\lambda) = \mathbb{E}_{x,S}[(\hat{w}_\lambda - \mathbb{E}_S[\hat{w}_\lambda])^{\top}\phi(x)]^2.$$

- Larger λ decreases variance but increases bias. Choosing λ correctly is crucial.

2.7 Model Selection

Approach: Choose λ to minimize generalization error using:

- Hold-out validation.
- K-fold cross-validation.

Chapter 3

Support Vector Machines

Support Vector Machines (SVMs) are a powerful supervised learning method for binary classification. They aim to find the optimal hyperplane that separates data points of different classes with the maximum margin.

3.1 Optimal Separating Hyperplane (OSH)

3.1.1 Separating Hyperplanes

A **hyperplane** in $\mathbb{R}^n$ is defined as:

$$H_{w,b} = \{x \in \mathbb{R}^n : w^\top x + b = 0\}$$

where $w \in \mathbb{R}^n$ is the normal vector to the hyperplane, and $b \in \mathbb{R}$ is the bias term.
Given a **training set**:

$$S = \{(x_i, y_i)\}_{i=1}^m \subseteq \mathbb{R}^n \times \{-1, 1\}$$

we assume that the data is **linearly separable**, meaning that there exist parameters $w \in \mathbb{R}^n$ and $b \in \mathbb{R}$ such that:

$$y_i(w^\top x_i + b) > 0, \quad \forall i \in [m].$$

A hyperplane satisfying this condition is called a **separating hyperplane**. Importantly, the strict inequality ensures that no data point lies exactly on the hyperplane.

3.1.2 Distance to a Hyperplane

The perpendicular **distance** of a point $x \in \mathbb{R}^n$ from a hyperplane $H_{w,b}$ is given by:

$$\rho_x(w,b) = \frac{|w^\top x + b|}{\|w\|}$$

which measures how far the point x is from the decision boundary.

If a hyperplane $H_{w,b}$ separates the dataset S, we define the **margin** as the minimum distance between the hyperplane and any training point:

$$\rho_S(w,b) = \min_{i\in[m]} \rho_{x_i}(w,b) = \min_{i\in[m]} \frac{|w^\top x_i + b|}{\|w\|}$$

where $\rho_S(w,b)$ quantifies how well-separated the two classes are by the hyperplane.

Additionally, we define the **signed margin** of a point x, which can be positive or negative, as:

$$\frac{w^\top x + b}{\|w\|}$$

indicating whether the point lies on the positive or negative side of the hyperplane.

3.1.3 Projection Onto the Hyperplane

The projection p of a point x onto the hyperplane $H_{w,b}$ is given by:

$$p = x - \frac{w(b + \langle w, x\rangle)}{\|w\|^2}$$

This projection satisfies:

1. p lies on the hyperplane.
2. The vector $x - p$ is perpendicular to the hyperplane (orthogonal to $p - x'$ where x' is some other point on the hyperplane $H_{w,b}$.

Verifying 1: We observe that,

$$\langle w, p\rangle + b = \langle w, x\rangle - \frac{\langle w, w\rangle(b + \langle w, x\rangle)}{\|w\|^2} + b = 0.$$

Verifying 2: We observe that

$$\langle p - x, p - x'\rangle = \left\langle -\frac{w(b + \langle w, x\rangle)}{\|w\|^2}, p - x' \right\rangle$$

$$= \frac{(b + \langle w, x\rangle)^2}{\|w\|^2} - \left\langle x - x', \frac{w(b + \langle w, x\rangle)}{\|w\|^2} \right\rangle$$

$$= 0 \quad \text{since } \langle w, x' \rangle + b = 0.$$

Computing the distance $\rho_x(w, b) = \|x - p\|$ of a point x from a hyperplane $H_{w,b}$, we get:

$$\sqrt{\langle p - x, p - x \rangle} = \sqrt{\left\langle \frac{w(b + \langle w, x \rangle)}{\|w\|^2}, \frac{w(b + \langle w, x \rangle)}{\|w\|^2} \right\rangle}$$
$$= \frac{|b + \langle w, x \rangle|}{\|w\|}$$

which matches our previous formula for distance to a hyperplane.

3.1.4 Optimal Separating Hyperplane (OSH)

Among all possible separating hyperplanes, the **Optimal Separating Hyperplane (OSH)** is the one that **maximizes** the margin:

$$\rho(S) = \max_{w,b} \min_{i \in [m]} \frac{y_i(w^\top x_i + b)}{\|w\|}$$

This is the objective of **Support Vector Machines (SVMs)** in the **hard-margin case**.

3.2 Formulation and Solution

3.2.1 Canonical Parameterization

A separating hyperplane is parameterized by (w, b), but this choice is not unique because rescaling with a positive constant gives the same separating hyperplane. Two possible ways to fix the parameterization are:

- **Normalized hyperplane**: Set $\|w\| = 1$, in which case

$$\rho_x(w, b) = |w^\top x + b|, \quad \rho_S(w, b) = \min_{i \in [m]} y_i(w^\top x_i + b).$$

- **Canonical hyperplane**: Choose $\|w\|$ such that

$$\rho_S(w, b) = \frac{1}{\|w\|}$$

  i.e., we require that

$$\min_{i \in [m]} y_i(w^\top x_i + b) = 1.$$

  This is a data-dependent parameterization.

We will mainly work with the second parameterization:

- If we work with normalized hyperplanes, we have:

$$\rho(S) = \max_{w,b} \min_i \left\{ y_i(w^\top x_i + b) \mid y_j(w^\top x_j + b) \geq 0, \|w\| = 1, j \in [m] \right\}$$

- If we work with canonical hyperplanes, instead, we have:

$$\rho(S) = \max_{w,b} \left\{ \frac{1}{\|w\|} \mid \min_{i \in [m]} y_i(w^\top x_i + b) = 1, y_j(w^\top x_j + b) \geq 0 \right\}$$

$$= \max_{w,b} \left\{ \frac{1}{\|w\|} \mid y_i(w^\top x_j + b) \geq 1, i \in [m] \right\}$$

$$= \frac{1}{\min_{w,b} \{ \|w\| \mid y_i(w^\top x_i + b) \geq 1, i \in [m] \}}$$

3.2.2 Primal Problem

The Optimal Separating Hyperplane (OSH) can be found by solving the following optimization problem of canonical hyperplanes:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \ldots, m. \end{aligned}$$

This formulation ensures that the margin $\frac{1}{\|w\|}$ is maximized.

3.2.3 Lagrangian and Dual Problem

To solve the primal problem, we introduce Lagrange multipliers $\alpha_i \geq 0$ and define the Lagrangian function:

$$L(w, b; \alpha) = \frac{1}{2} w^\top w - \sum_{i=1}^{m} \alpha_i \big(y_i(w^\top x_i + b) - 1 \big).$$

Taking the derivatives with respect to w and b and setting them to zero, we obtain:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{m} \alpha_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum_{i=1}^{m} \alpha_i y_i x_i.$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{m} \alpha_i y_i = 0.$$

By substituting these conditions into the Lagrangian, we obtain the **dual problem**:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{subject to} \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, \ldots, m. \end{aligned}$$

where the complexity of this problem depends on m, not the number of input components n.

3.2.4 Support Vectors

The **support vectors** are the data points x_i for which $\alpha_i > 0$, while α_i is the solution of the dual problem. The solution to the optimization problem can be written as:

$$w = \sum_{i=1}^{m} \alpha_i y_i x_i.$$

The bias term b is determined using the **Kuhn-Tucker complementary slackness** condition:

$$\alpha_i (y_i (w^\top x_i + b) - 1) = 0.$$

For any support vector x_j, we obtain:

$$b = y_j - w^\top x_j.$$

3.2.5 Final Decision Rule

Once w and b are determined, we classify a new point x using:

$$\hat{y} = \text{sgn}\left(\sum_{i=1}^{m} y_i \alpha_i x_i^\top x + b \right).$$

3.2.6 Connection Between Support Vectors and Generalization

A key property of the OSH is that it is determined **only by the support vectors (SVs)**, which are typically a small subset of the training data. The entire dataset could be replaced by just these points, and the **same hyperplane would be found**.

Additionally, let n_{SV} be the expected number of support vectors in an SVM trained on m examples sampled i.i.d. The **expected generalization error of an SVM** trained on $m-1$ examples is bounded above by:

$$\frac{n_{SV}}{m}.$$

3.2.7 Intuitions of SVM

Support Vector Sparsity and Generalization

1. We argue qualitatively that **sparsity** of support vectors implies good generalization performance.
2. Intuitively, a larger margin leads to better generalization performance. Note: *There are several theoretical arguments in the literature that provide qualitative bounds on the generalization error based on the margin, but for our purposes, these are beyond the scope of this class.*
3. A larger margin typically implies fewer support vectors, assuming all other conditions remain constant.

Geometric Interpretation of SVM

1. The SVM optimization problem can be visualized as finding the **shortest line segment** that connects the convex hull of the positive class to the convex hull of the negative class. The perpendicular bisector of this segment defines the separating hyperplane.
2. The number of support vectors associated with a **face** in the convex hull is equal to the number of vertices in that face.
3. Given a convex polytope, a point closer to it is generally nearer to a larger-dimensional face than a farther point.

3.3 Soft Margin SVM

3.3.1 Linearly Non-separable Case

For datasets that are not linearly separable, we introduce **slack variables** $\xi_i \geq 0$ to allow some points to violate the margin constraints:

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad i = 1, \ldots, m.$$

The goal is to balance maximizing the margin and minimizing classification errors by penalizing misclassified points.

3.3.2 Primal Problem with Slack Variables

The **soft-margin SVM** is formulated as:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}\xi_i \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \ldots, m. \end{aligned}$$

where C is a regularization parameter controlling the trade-off between margin maximization and classification errors.

3.3.3 Hinge Loss and Convex Optimization

Ideally, we would like to minimize the **misclassification loss**, which is given by:

$$\frac{1}{2}w^\top w + C\sum_{i=1}^{m} V_{\text{mc}}(y_i, w^\top x_i + b),$$

where

$$V_{\text{mc}}(y, \hat{y}) = \mathbb{I}[y = \text{sign}(\hat{y})].$$

However, this loss function leads to a **non-convex, NP-hard** optimization problem, making it computationally intractable.

To overcome this, we introduce the **hinge loss function**, which is a convex upper bound on the misclassification loss:

$$V_{\text{hinge}}(y, \hat{y}) = \max(0, 1 - y\hat{y}).$$

Thus, we reformulate the optimization problem as:

$$\frac{1}{2}w^\top w + C\sum_{i=1}^{m} V_{\text{hinge}}(y_i, w^\top x_i + b),$$

which leads to a **convex quadratic optimization problem** that is solvable using standard optimization techniques.

3.3.4 Dual Problem for Soft Margin SVM

The **Lagrangian** for the primal problem is:

$$L(w, \xi, b; \alpha, \beta) = \frac{1}{2}w^\top w + C\sum_{i=1}^{m}\xi_i - \sum_{i=1}^{m}\alpha_i\left[y_i(w^\top x_i + b) + \xi_i - 1\right] - \sum_{i=1}^{m}\beta_i\xi_i.$$

where $\alpha_i, \beta_i \geq 0$ are the Lagrange multipliers.

Solving for the saddle point, we obtain the **dual problem**:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{subject to} \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, \ldots, m. \end{aligned}$$

The key difference from the hard-margin SVM is the **box constraint** $0 \leq \alpha_i \leq C$, which limits the contribution of each support vector.

3.3.5 Kuhn-Tucker Conditions and Support Vectors

From the **complementary slackness** conditions, we derive:

$$\alpha_i(y_i(\bar{w}^\top x_i + \bar{b}) - 1 + \bar{\xi}_i) = 0,$$

$$(C - \alpha_i)\bar{\xi}_i = 0.$$

From this, we categorize points as:

- If $y_i(\bar{w}^\top x_i + \bar{b}) > 1$, then $\alpha_i = 0$ (not a support vector).
- If $y_i(\bar{w}^\top x_i + \bar{b}) < 1$, then $\alpha_i = C$ (a support vector with slack $\xi_i > 0$).
- If $y_i(\bar{w}^\top x_i + \bar{b}) = 1$, then $\alpha_i \in (0, C)$ (a support vector on the margin).

3.3.6 Effect of the Regularization Parameter C

The parameter C controls the trade-off between maximizing the margin and minimizing classification training errors:

- **Large** C: More emphasis on correct classification (smaller margin, fewer misclassified points).
- **Small** C: More emphasis on margin maximization (larger margin, more tolerance for misclassified points).

3.3.7 Computing Leave-One-Out (LOO) Error

- C is often selected by minimizing the **leave-one-out (LOO) cross-validation error**.
- The number of support vectors n_{SV} provides an upper bound on the LOO error:

$$\frac{n_{\text{SV}}}{m}.$$

- The SVM only needs to be retrained at most n_{SV} times to compute the LOO error, making it efficient.

3.4 Kernels and Non-linear SVMs

3.4.1 Kernel Trick

Definition 3.4.1 (Kernel)**.** A **kernel function** $K(x, x')$ computes the inner product in a high-dimensional feature space:

$$K(x, x') = \phi(x)^\top \phi(x').$$

Common kernel functions include:

- **Linear kernel**: $K(x, x') = x^\top x'$.
- **Polynomial kernel**: $K(x, x') = (x^\top x' + c)^d$.
- **Gaussian (RBF) kernel**: $K(x, x') = \exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$.

3.4.2 SVM with Kernels

The above SVM formulation holds even when we use a **feature map** $\phi : \mathcal{X} \to \mathcal{W}$, replacing x with $\phi(x)$ and the dot product $x^\top t$ with the kernel function $K(x, t)$.

Thus, the **decision function** for an SVM becomes:

$$f(x) = \sum_{i=1}^{m} \alpha_i y_i K(x_i, x) + b, \quad x \in \mathcal{X}.$$

where the parameters α_i solve the dual optimization problem and the bias term b is determined as discussed in previous sections.

A new data point $x \in \mathcal{X}$ is classified as:

$$\text{sgn}(f(x)).$$

3.5 Connection to Regularization

Theorem 3.5.1 (SVM and Regularization)**.** *The SVM objective function can be rewritten as a* ***regularized optimization problem****:*

$$E_\lambda(w, b) = \sum_{i=1}^{m} \max(0, 1 - y_i(w^\top \phi(x_i) + b)) + \lambda \|w\|^2,$$

where the regularization parameter is defined as $\lambda = \frac{1}{2C}$.

The full optimization problem formulation is:

$$\begin{aligned} \min_{w,b,\xi} \quad & C\sum_{i=1}^{m}\xi_i + \frac{1}{2}\|w\|^2 \\ \textit{subject to} \quad & y_i(w^\top\phi(x_i)+b) \geq 1-\xi_i, \quad \xi_i \geq 0, \quad i=1,\ldots,m. \end{aligned}$$

This problem can be equivalently rewritten in the loss function form:

$$\min_{w,b} \quad C\sum_{i=1}^{m}\max(0, 1-y_i(w^\top\phi(x_i)+b)) + \frac{1}{2}\|w\|^2.$$

3.6 SVM Regression

3.6.1 ϵ-Insensitive Loss

Definition 3.6.1 (Loss Function)**.** For regression, SVMs use the ϵ-insensitive loss function:

$$|y - f(x)|_\epsilon = \max(0, |y-f(x)| - \epsilon).$$

This function ignores errors smaller than ϵ, making the model robust to small noise.

3.6.2 Optimization Problem for SVM Regression

The optimization problem for support vector regression (SVR) is formulated as:

$$\begin{aligned} \min_{w,b,\xi,\xi^*} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}(\xi_i+\xi_i^*) \\ \text{subject to} \quad & y_i - w^\top x_i - b \leq \epsilon + \xi_i, \\ & w^\top x_i + b - y_i \leq \epsilon + \xi_i^*, \\ & \xi_i, \xi_i^* \geq 0, \quad i=1,\ldots,m. \end{aligned}$$

SVM regression is **scale-sensitive**: errors below ϵ are ignored, which results in **sparse solutions**.

3.7 Solution Methods

The above optimization problems belong to **Quadratic Programming (QP)**. There exist several methods to solve QP problems efficiently:

- **Interior point methods** from convex optimization.
- **Sequential Minimal Optimization (SMO)** for large-scale SVMs.

If a **non-linear kernel** is used, the **number of features** N is typically much larger than the number of training examples m. In this case, it is preferable to solve the **dual problem**.

However, if the number of training examples is much larger than the number of features ($m \gg N$), solving the **primal problem** is computationally more efficient.

3.8 Decomposition of the Dual Problem

For very **large datasets** (e.g., $m > 10^5$), solving the full **dual problem** using standard optimization techniques is impractical because the **matrix A is dense**.

A **decomposition technique** is used, where we iteratively optimize with respect to a small "active set" $\mathcal{A}$ of variables:

- Start with $\alpha = 0$, and choose a subset $\mathcal{A}$ of $q \leq m$ variables.
- **Optimize** $Q(\alpha)$** with respect to the variables in $\mathcal{A}$.
- Remove variables that satisfy the **KKT conditions** and add new variables that **violate the KKT conditions**.
- Repeat until convergence.

Each iteration **increases the objective function $Q(\alpha)$**.

This approach is used in **SMO (Sequential Minimal Optimization)** algorithms for solving SVMs efficiently.

Chapter 4

First Order Optimization and ERM

4.1 Binary Classification via Convex Surrogates

4.1.1 Loss Functions

Binary classification seeks to minimize a loss function $\ell(f(x), y)$ that measures the discrepancy between predictions $f(x)$ and labels y. Many classification strategies fall into the framework of Empirical Risk Minimization (ERM), where the objective is:

$$\min_w \frac{1}{m} \sum_{i=1}^{m} \ell(w^\top x_i, y) + \lambda \|w\|^2$$

Several commonly used loss functions include:

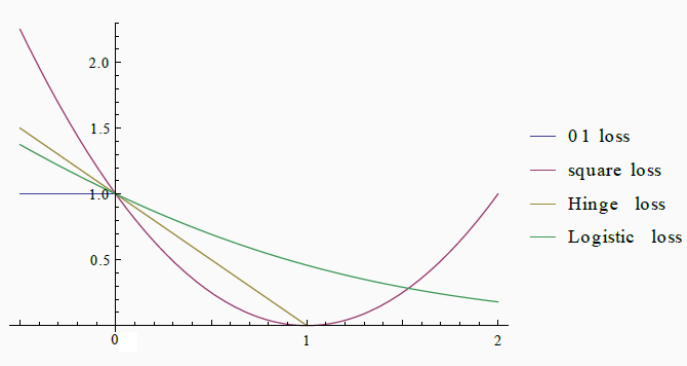

Figure 4.1: Comparsion of different loss functions.

- **0-1 Loss:** $\widetilde{\ell}(r) = \mathbf{1}\{r \leq 0\}$. This is the most natural loss for classification but is **non-convex**

and **computationally intractable**.

- **Square Loss:** $\widetilde{\ell}(r) = (1-r)^2$. While **convex and smooth**, it is more suitable for regression and does not align well with classification problems.
- **Hinge Loss (SVM):** $\widetilde{\ell}(r) = \max(0, 1-r)$. This corresponds to the support vector machine (SVM) formulation, encouraging a large margin between classes. It is **convex but non-smooth**.
- **Logistic Loss:** $\widetilde{\ell}(r) = \log(1+e^{-r})$. This is used in logistic regression and leads to probabilistic interpretations. It is **convex and smooth**, making it a practical choice for optimization.

4.1.2 Convex Surrogates

Directly minimizing the 0-1 loss is intractable because it is non-convex and discontinuous. Instead, **convex surrogates** serve as upper bounds, facilitating optimization while still approximating the classification objective. The square, hinge, and logistic losses are examples of convex surrogates.

These loss functions arise from different interpretations:

- **Square Loss:** The Bayes estimator under the square loss is given by:

$$f_*(x) = \mathbb{E}[y \mid x] = P(y = 1 \mid x) - P(y = -1 \mid x)$$

  Taking $\operatorname{sign}(f_*(x))$ recovers the Bayes optimal classifier under the 0-1 loss.
- **Hinge Loss:** The hinge loss leads to a geometric interpretation of classification by maximizing the margin between classes, as seen in SVMs.
- **Logistic Loss:** Logistic regression models class probabilities and can be interpreted as minimizing the negative log-likelihood under a probabilistic model.

Since these **surrogate losses are convex upper bounds** to the 0-1 loss, they enable efficient optimization while preserving meaningful classification behavior.

4.2 Logistic Regression

4.2.1 Model Formulation

Logistic regression originates from a probabilistic perspective, where we model the conditional probability of $y = 1$ given x as:

$$p_{\mu,s}(x) = P(y = 1 \mid x; \mu, s) = \frac{1}{1 + e^{-(x-\mu)/s}}$$

This corresponds to a sigmoid function, which maps real-valued inputs to probabilities in the range $(0, 1)$. By taking $w = 1/s$ and $b = -\mu/s$, we obtain an **alternative parameterization**:

$$p_{w,b}(x) = \frac{1}{1 + e^{-(wx+b)}}$$

This formulation can be further generalized to feature mappings $\phi(x)$, leading to the general logistic regression model:

$$p_w(x) = \frac{1}{1 + e^{-w^\top \phi(x)}}$$

For simplicity, we often assume $\phi(x) = x$, reducing it to:

$$P(y = 1 \mid x; w) = \frac{1}{1 + e^{-w^\top x}}$$

and $P(y = -1 \mid x; w) = 1 - P(y = 1 \mid x; w)$.

4.2.2 Optimization Problem

Given a dataset $S = \{(x_i, y_i)\}_{i=1}^m$, our objective is to find the best model $p_w(x)$ that maximizes the likelihood of the observed data. This leads to the likelihood function:

$$\max_w P(y_1, \ldots, y_m \mid x_1, \ldots, x_m; w)$$

Under the assumption that the **data points are sampled i.i.d.**, the joint probability decomposes as:

$$P(y_1, \ldots, y_m \mid x_1, \ldots, x_m; w) = \prod_{i=1}^m P(y_i \mid x_i; w)$$

Since the logarithm is a monotonic function, we equivalently maximize the log-likelihood:

$$\max_w \sum_{i=1}^m \log P(y_i \mid x_i; w)$$

Using the logistic model, we express the probability of each class:

$$P(y_i \mid x_i; w) = \frac{1}{1 + e^{-y_i w^\top x_i}}$$

Thus, the optimization problem becomes:

$$\max_w \sum_{i=1}^m \log \frac{1}{1 + e^{-y_i w^\top x_i}}$$

which simplifies to **minimizing the logistic loss**:

$$\min_w \sum_{i=1}^{m} \log(1 + e^{-y_i w^\top x_i}).$$

4.2.3 Logistic Regression and Empirical Risk Minimization

The logistic regression model can be viewed as a special case of empirical risk minimization (ERM) with the logistic loss function:

$$\min_w \sum_{i=1}^{m} \log(1 + e^{-y_i w^\top x_i}).$$

Similar to SVM and square loss regression, classification is performed by taking the sign of $f(x) = w^\top x$, since:

$$P(1 \mid x; w) > P(-1 \mid x; w) \iff \frac{1}{1 + e^{-w^\top x}} > \frac{1}{1 + e^{w^\top x}}$$

which reduces to:

$$-w^\top x < w^\top x$$

Thus, we assign class $y = 1$ if $w^\top x$ is positive and $y = -1$ otherwise.

4.2.4 Solving the Optimization Problem

The final ERM formulation involves minimizing the regularized loss:

$$\min_w \frac{1}{m} \sum_{i=1}^{m} \ell(w^\top x_i, y_i) + \lambda \|w\|^2$$

where $\ell(w^\top x_i, y_i) = \log(1 + e^{-y_i w^\top x_i})$ is the logistic loss.
This concludes the derivation of logistic regression under the ERM framework.

4.3 Gradient Descent

4.3.1 Smoothness and Convexity

In optimization, smoothness plays a key role in deriving convergence guarantees for gradient descent. The following definition formalizes smoothness:

Definition 4.3.1 (M-smooth Function)**.** A function $F : \mathbb{R}^N \to \mathbb{R}$ is M-smooth if its gradient is Lipschitz continuous with Lipschitz constant M, i.e.,

$$\|\nabla F(w) - \nabla F(z)\| \leq M\|w - z\|, \quad \forall w, z \in \mathbb{R}^N.$$

This condition ensures that F does not change too rapidly, making optimization more stable. Many common loss functions, such as the logistic loss and the square loss, satisfy this property.

4.3.2 Quadratic Upper Bound

A crucial result in convex optimization is the quadratic upper bound, which provides an upper estimate on the function's growth.

Theorem 4.3.2 (Quadratic Upper Bound)**.** *If F is M-smooth, then for any $w, z \in \mathbb{R}^N$,*

$$F(z) \leq F(w) + \nabla F(w)^\top (z - w) + \frac{M}{2}\|z - w\|^2.$$

This bound helps in designing optimization algorithms and analyzing their convergence properties. It can be derived using the fundamental theorem of calculus and the Lipschitz continuity of ∇F.

4.3.3 Towards Gradient Descent

The quadratic upper bound suggests a natural way to update w to ensure function decrease. Choosing:

$$z = w + \eta \nabla F(w)$$

for a step size η, we obtain:

$$F(z) \leq F(w) + \eta\|\nabla F(w)\|^2 + \frac{\eta^2 M}{2}\|\nabla F(w)\|^2.$$

To ensure $F(z) < F(w)$, we require $\eta(1 + \frac{\eta M}{2}) < 0$, which holds when $\eta \in (-\frac{2}{M}, 0)$. A common choice is $\eta = -\frac{1}{M}$, which leads to:

$$F(z) \leq F(w) - \frac{1}{2M}\|\nabla F(w)\|^2.$$

This confirms that gradient descent decreases the function value at each step.

4.3.4 Gradient Descent Algorithm

Using the insights from the quadratic bound, we define gradient descent as follows:

Definition 4.3.3 (Gradient Descent)**.** Gradient Descent (GD) is an iterative optimization method that updates w according to the rule:

$$w_{k+1} = w_k - \eta \nabla F(w_k),$$

where $\eta > 0$ is the step size (or learning rate in machine learning).

4.3.5 Convergence of Gradient Descent

If F is convex and M-smooth, we can derive convergence guarantees for gradient descent.

Theorem 4.3.4 (Convergence of Gradient Descent)**.** *If F is convex and M-smooth, and we choose $\eta = \frac{1}{M}$, then:*

$$F(w_k) - F(w^*) \leq \frac{M}{2k} \|w_0 - w^*\|^2,$$

where w^* is the minimizer of F.

This result shows that gradient descent has a convergence rate of $O(1/k)$, which ensures function value decrease over iterations.

4.3.6 Proof of the Quadratic Upper Bound

The quadratic upper bound is derived as follows:

1. Define $h(\theta) = F(w+\theta(z-w))$, which interpolates between w and z. 2. Using the fundamental theorem of calculus:

$$\int_0^1 h'(\theta) d\theta = h(1) - h(0).$$

3. Differentiating $h(\theta)$:

$$h'(\theta) = \nabla F(w + \theta(z - w))^\top (z - w).$$

4. Using the Cauchy-Schwarz inequality and smoothness:

$$(\nabla F(w + \theta(z - w)) - \nabla F(w))^\top (z - w) \leq M\theta \|z - w\|^2.$$

5. Integrating both sides and simplifying yields:

$$F(z) \leq F(w) + \nabla F(w)^\top (z - w) + M\|z - w\|^2 \int_0^1 \theta d\theta = F(w) + \nabla F(w)^\top (z - w) + \frac{M}{2} \|z - w\|^2.$$

This completes the proof, providing a fundamental result for analyzing gradient-based optimization.

4.3.7 When Does Gradient Descent Work?

While correctly choosing η ensures a decreasing sequence, an open question remains: does gradient descent converge to a minimizer? This depends on conditions such as strong convexity and further refinements of the step-size schedule.

This concludes our discussion on gradient descent, providing a rigorous foundation for its use in optimization problems.

4.4 Gradient Descent and Convexity

4.4.1 Convexity

A function $F(w)$ is convex if, for any $w, z \in \mathbb{R}^N$ and any $\theta \in [0, 1]$,

$$F(\theta w + (1-\theta)z) \leq \theta F(w) + (1-\theta)F(z).$$

Convex functions are particularly desirable in optimization because:

- They do not have local minima other than the global minimum.
- Several mathematical tools exist to approximate and find solutions efficiently.

4.4.2 First-Order Characterization of Convexity

For differentiable functions, convexity can be characterized using the first-order condition:

Theorem 4.4.1 (First-Order Condition)**.** *A differentiable function $F : \mathbb{R}^N \to \mathbb{R}$ is convex if and only if for any $w, z \in \mathbb{R}^N$,*

$$F(z) \geq F(w) + \nabla F(w)^\top (z - w).$$

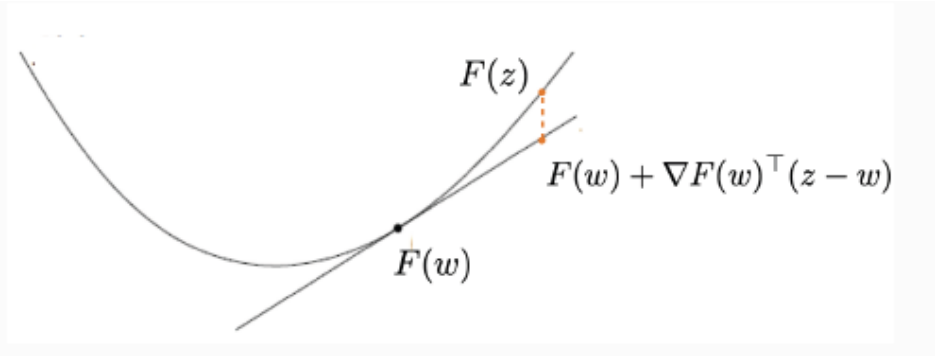

Figure 4.2: First-order Condition.

This condition implies that the function always lies above its first-order Taylor approximation, as illustrated in Figure **??**. This ensures that gradient-based methods make progress toward minimizing F.

Remark. *If $\nabla F(w_*) = 0$, then for any $z \in \mathbb{R}^N$,*

$$F(z) \geq F(w_*).$$

Thus, the minimizers of F are precisely the points where $\nabla F(w_*) = 0$.

Remark. *While smoothness provides an **upper bound**, convexity provides a **lower bound**, effectively sandwiching our objective function F between two constraints.*

4.4.3 Proof of the First-Order Condition

(Forward direction $\Rightarrow$): Assume F is convex and let $\theta \in (0, 1]$. Using the definition of convexity, we obtain:

$$F(\theta z + (1-\theta)w) \leq \theta F(z) + (1-\theta)F(w).$$

Rearranging,

$$\frac{F(\theta z + (1-\theta)w) - F(w)}{\theta} \leq F(z) - F(w).$$

Taking the limit $\theta \to 0$, we obtain:

$$\lim_{\theta \to 0} \frac{F(w + \theta(z-w)) - F(w)}{\theta} = \nabla F(w)^\top (z - w).$$

Thus, we derive:

$$F(z) \geq F(w) + \nabla F(w)^\top (z - w).$$

(Reverse direction $\Leftarrow$): Assume the first-order condition holds and define $u = \theta w + (1-\theta)z$. By assumption, we have:

$$F(w) \geq F(u) + \nabla F(u)^\top (w - u),$$

$$F(z) \geq F(u) + \nabla F(u)^\top (z - u).$$

Multiplying the first equation by θ and the second by $(1-\theta)$, then summing, we obtain:

$$\theta F(w) + (1-\theta)F(z) \geq F(u) = F(\theta w + (1-\theta)z).$$

Thus, F is convex.

4.4.4 Convergence of Gradient Descent

Given that convexity ensures a single global minimum, we analyze how gradient descent converges to it.

Theorem 4.4.2 (Gradient Descent Convergence)**.** *Let F be convex and M-smooth, and assume F admits a minimum at w_*. If $(w_k)_{k=1}^K$ is a sequence generated by gradient descent with step size $\eta = \frac{1}{M}$, then:*

$$F(w_K) - F(w_*) \leq \frac{M}{2K}\|w_0 - w_*\|^2.$$

Proof: By applying the first-order condition of convexity to w_k and w_*, we obtain:

$$F(w_*) \geq F(w_k) + \nabla F(w_k)^\top (w_* - w_k).$$

Rearranging, we get:

$$F(w_k) - F(w_*) \leq \nabla F(w_k)^\top (w_k - w_*).$$

Using the gradient descent update rule $w_{k+1} = w_k - \frac{1}{M}\nabla F(w_k)$, we obtain:

$$F(w_{k+1}) \leq F(w_k) - \frac{1}{2M}\|\nabla F(w_k)\|^2.$$

Bounding the Error: Using the step-size relation:

$$F(w_{k+1}) - F(w_*) \leq \frac{M}{2}\|w_k - w_*\|^2 - \frac{M}{2}\|w_{k+1} - w_*\|^2.$$

Summing over all iterations $k = 1, \ldots, K$, we obtain:

$$K(F(w_K) - F(w_*)) \leq \frac{M}{2}\|w_0 - w_*\|^2.$$

Dividing by K gives:

$$F(w_K) - F(w_*) \leq \frac{M}{2K}\|w_0 - w_*\|^2.$$

This confirms that gradient descent achieves an $O(1/K)$ convergence rate.

4.4.5 Conclusion

Gradient descent is guaranteed to converge for **convex, smooth functions**, with a rate depending on the smoothness constant M. This reinforces the importance of convexity in optimization, as it ensures that gradient-based methods effectively minimize the function without **getting trapped in local minima**.

4.4.6 The Importance of Being Smooth

The smoothness constant M of a function plays a crucial role in optimization:

- It determines the appropriate choice of step size η in gradient descent.
- It directly affects convergence rates.

This motivates the need to estimate the Lipschitz constant of a function.

4.4.7 Estimating the Lipschitz Constant

The Lipschitz constant M characterizes the smoothness of F. A general strategy for estimating M is:

Theorem 4.4.3. *Let $F : \mathbb{R}^N \to \mathbb{R}$ be differentiable and M-Lipschitz. Then,*

$$M = \sup_w \|\nabla F(w)\|.$$

This result applies to scalar-valued functions and can be generalized to vector-valued functions, such as the gradient itself.

4.4.8 Proof of the Lipschitz Constant Estimate

Proof. For any $\theta > 0$ and $v \in \mathbb{R}^N$, the Lipschitz condition gives:

$$\frac{|F(w + \theta v) - F(w)|}{\theta} \leq M.$$

Taking the limit as $\theta \to 0$, we obtain:

$$\nabla F(w)^\top v \leq M \quad \forall v \in \mathbb{R}^N,$$

which implies $\|\nabla F(w)\| \leq M$. Taking the supremum over w, we conclude:

$$M = \sup_w \|\nabla F(w)\|.$$

For any two points $w, z \in \mathbb{R}^N$, we use the fundamental theorem of calculus:

$$F(w) - F(z) = \int_0^1 \nabla F(w + \theta(z - w))^\top (z - w) d\theta.$$

Applying the Cauchy-Schwarz inequality:

$$\left| \int_0^1 \nabla F(w + \theta(z - w))^\top (z - w) d\theta \right| \leq \int_0^1 \|\nabla F(w + \theta(z - w))\| d\theta \cdot \|z - w\|.$$

Taking the supremum over all v, we get:

$$\sup_v \|\nabla F(v)\| \geq M.$$

Thus, we conclude $M = \sup_w \|\nabla F(w)\|$. □

4.4.9 Example: Square Loss

Consider the squared loss function:

$$\ell(w) = (w^\top x - y)^2.$$

The gradient is:

$$\nabla \ell(w) = 2(w^\top x - y)x.$$

For any w, z, we compute:

$$\nabla \ell(w) - \nabla \ell(z) = 2(xx^\top)(w - z).$$

Taking the norm:

$$\|\nabla \ell(w) - \nabla \ell(z)\| \leq 2\|x\|^2 \|w - z\|,$$

showing that the square loss is $2\|x\|^2$-Lipschitz.

4.4.10 Example: Regularized Square Loss

For the empirical risk function:

$$\mathcal{E}_S(w) = \frac{1}{m}\sum_{i=1}^m (w^\top x_i - y_i)^2 + \lambda \|w\|^2.$$

Its gradient is:

$$\nabla \mathcal{E}_S(w) = 2\left(\frac{1}{m}\sum_{i=1}^m x_i x_i^\top + \lambda I\right) w - \frac{2}{m}\sum_{i=1}^m y_i x_i.$$

Defining the uncentered covariance matrix:

$$C_S = \frac{1}{m}\sum_{i=1}^m x_i x_i^\top,$$

we obtain:

$$\|\nabla \mathcal{E}_S(w) - \nabla \mathcal{E}_S(z)\| \leq 2\|C_S + \lambda I\| \|w - z\|.$$

Thus, $\mathcal{E}_S$ is $2\|C_S + \lambda I\|$-smooth. Notably, this value can be computed from data.

4.4.11 Example: Least Squares Revisited

For the function:

$$F(w) = \mathcal{E}_S(w),$$

its gradient corresponds to the Hessian:

$$\nabla^2 F(w) = 2(C_S + \lambda I).$$

Taking its operator norm over all w, we obtain:

$$\sup_w \|\nabla F(w)\| = 2\|C_S + \lambda I\|,$$

which confirms our earlier result.

4.4.12 Conclusion

The smoothness constant M plays a crucial role in optimization, guiding step-size selection and convergence analysis. Practical estimation strategies rely on computing the supremum of the gradient norm or using properties of the Hessian, which can often be computed directly from data.

4.5 Sub-gradient Method

4.5.1 Gradient Descent and Non-Differentiable Losses

Gradient-based optimization methods, such as gradient descent (GD), require differentiability to compute gradients. However, some convex functions, like the **Hinge loss**, are not differentiable at certain points. This presents challenges for optimization since the gradient is undefined at these points.

Example 4.5.1 (Hinge Loss Non-Differentiability)**.** The Hinge loss function is given by:

$$F(w) = \max(0, 1 - yw^\top x)$$

This function is non-differentiable at points where $yw^\top x = 1$, making standard gradient descent inapplicable.

4.5.2 Sub-gradients

Definition 4.5.2 (Sub-gradient)**.** For a convex function $F : \mathbb{R}^n \to \mathbb{R}$, a vector $g \in \mathbb{R}^n$ is a sub-gradient at w if:

$$F(z) \geq F(w) + g^\top (z - w), \quad \forall z \in \mathbb{R}^n.$$

Unlike gradients, which are unique for differentiable functions, sub-gradients may form a set of valid values, called the **sub-differential**, denoted $\partial F(w)$.

Remark. *For a convex but non-differentiable function, multiple sub-gradients can exist at non-differentiable points. This means that at these points, there exist many linear lower bounds to the function rather than a single tangent.*

4.5.3 Sub-gradient Computation

Definition 4.5.3 (Sub-differential for a Single Variable)**.** For a convex function $F : \mathbb{R} \to \mathbb{R}$, the sub-differential at w is given by:

$$\partial F(w) = [a, b]$$

where

$$a = \lim_{\theta \to 0^-} \frac{F(w+\theta) - F(w)}{\theta}, \quad b = \lim_{\theta \to 0^+} \frac{F(w+\theta) - F(w)}{\theta}.$$

Example 4.5.4 (Sub-gradient of the Hinge Loss)**.** For the hinge loss function:

$$F(w) = \max(0, 1 - yw^\top x),$$

the sub-differential is:

$$\partial F(w) = \begin{cases} -yx, & \text{if } w^\top x < 1/y \\ -rx, \quad r \in [0, y], & \text{if } w^\top x = 1/y \\ 0, & \text{if } w^\top x > 1/y. \end{cases}$$

4.5.4 Sub-gradient Descent

Definition 4.5.5 (Sub-gradient Descent)**.** The sub-gradient descent update rule is:

$$w_{k+1} = w_k - \eta_k g_k,$$

where $g_k \in \partial F(w_k)$ is a sub-gradient and $\eta_k > 0$ is the step size.

Remark. *Unlike gradient descent, which follows a unique gradient direction, sub-gradient descent allows for any valid sub-gradient. This means different optimization paths are possible at non-differentiable points.*

4.5.5 Convergence of Sub-gradient Descent

Theorem 4.5.6 (Convergence of Sub-gradient Method)**.** *Let $F : \mathbb{R}^N \to \mathbb{R}$ be L-Lipschitz and let $(w_k)_{k=1}^K$ be a sequence obtained using the sub-gradient method with step sizes η_k. Then:*

$$F(w_{best}) - F(w_*) \leq \frac{\|w_0 - w_*\|^2 + L^2 \sum_{k=1}^K \eta_k^2}{2 \sum_{k=1}^K \eta_k}$$

where

$$w_{best} = \arg \min_{w \in \{w_1, \ldots, w_K\}} F(w).$$

Remark. *For constant step size $\eta_k = \eta$, we get:*

$$F(w_K^{best}) - F(w_*) \leq \frac{\|w_0 - w_*\|^2}{2\eta K} + \frac{L^2 \eta}{2}.$$

Choosing $\eta = \frac{1}{\sqrt{K}}$ leads to:

$$F(w_K^{best}) - F(w_*) \leq \frac{1}{2\sqrt{K}}(\|w_0 - w_*\| + L^2).$$

This results in a convergence rate of $O(1/\sqrt{K})$, which is slower than gradient descent ($O(1/K)$).

4.5.6 Sub-gradients for Non-smooth Losses

Example 4.5.7 (Sub-gradient for General Non-smooth Loss)**.** For a non-smooth loss function $F(w) = \ell(w^\top x, y)$, where ℓ is convex but non-differentiable, the sub-differential is given by:

$$\partial F(w) = \partial \ell(w^\top x, y) x = \{gx \mid g \in \partial \ell(w^\top x, y)\}.$$

4.5.7 Summary

- Several strategies exist for empirical risk minimization (ERM) in binary classification.
- Gradient descent works well for convex and smooth loss functions.
- For non-smooth but convex losses (e.g., hinge loss in SVMs), we use sub-gradient methods.
- The sub-gradient descent method converges, but at a slower rate $\mathcal{O}(1/\sqrt{K})$ compared to standard gradient descent $\mathcal{O}(1/K)$.
- Understanding sub-gradients is crucial for optimizing non-differentiable loss functions.

Chapter 5

Tree-Based Methods and Ensemble Learning

5.1 Tree-Based Learning Algorithms

5.1.1 Tree-Based Models

Definition 5.1.1. Tree-based methods partition the input space into a set of disjoint rectangles and fit a simple model in each one. The prediction function is:

$$f(x) = \sum_{p=1}^{P} c_p \mathbb{I}[x \in R_p],$$

where:

- $R_1, R_2, \ldots, R_P$ are hyper-rectangular regions such that $\cup_{p=1}^{P} R_p = \mathbb{R}^n$ and $R_a \cap R_b = \varnothing$ for $a \neq b$.
- $c_p = \text{mean}(y_i \mid x_i \in R_p)$ minimizes the square error in region R_p. It's some real parameters.

5.1.2 Recursive Binary Partitioning

Theorem 5.1.2. *Recursive binary partitioning divides the input space into two regions at each step. Each split is determined by:*

$$R_1(j, s) = \{x \mid x_j \leq s\}, \quad R_2(j, s) = \{x \mid x_j > s\}.$$

Optimal splitting minimizes:

$$\min_{j,s} \left(\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right).$$

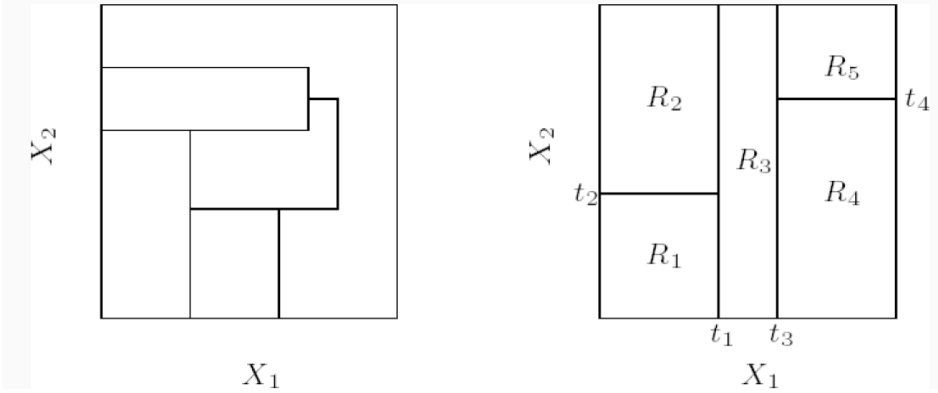

Figure 5.1: Recursive Binary Partitions. **Left Plot:** Each partition line has a simple description, yet partitions may be complicated to describe. **Right Plot:** Recursive binary partition (first split the space into two regions then iterate in each region.)

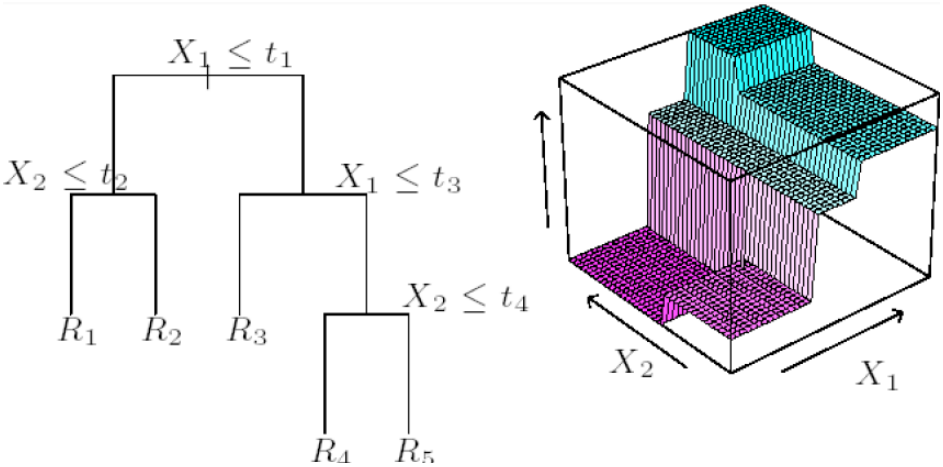

Figure 5.2: Tree Representation. **Left Plot:** Tree representation for the recursive binary partition. **Right Plot:** Prediction function $f(x) = \sum_{p=1}^{5} c_p \mathbb{I}[x \in R_p]$. Observe that each rectangle is definied by a conjunction: $R_4 := \{x : (x_1 > t_1) \wedge (x_1 > t_3) \wedge (x_2 \leq t_4)\}$.

Remark.

- *A nice feature of a recursive binary tree is its **interpretability** (e.g. in medical science the tree stratifies the population into strata of high and low outcome on the basis of patient characteristics)*

- *Warning: **risk for overfitting**! (with enough splits the tree can fit arbitrarily well the data)*

- ***Key Question: How to compute the tree (hence the regions R_n) and how to control overfitting?***

5.2 Classification and Regression Trees (CART)

5.2.1 Regression Trees

Regression trees aim to partition the feature space into disjoint regions and fit a simple model (typically a constant) in each region.

Objective Function

The goal is to determine the splitting variables and split points automatically. Given a partition of the feature space into regions $R_1, \ldots, R_P$, we predict the response y within each region using the mean:

$$c_p = \text{ave}(y_i | x_i \in R_p).$$

This corresponds to minimizing the squared error in region p. The ideal optimization problem is formulated as:

$$\min_{R_1,\ldots,R_P} \sum_{i=1}^{m} \left(y_i - \sum_{p=1}^{P} \text{ave}(y_j | x_j \in R_p) \mathbb{I}[x_i \in R_p] \right)^2 .$$

Since solving this optimization problem exactly is computationally intractable, we use an alternate heuristic approach.

Finding the Best Split

To find the first split, we define a pair of axis-aligned half-planes:

$$R_1(j, s) = \{x | x_j \leq s\}, \quad R_2(j, s) = \{x | x_j > s\}.$$

We seek to find optimal values j^* and s^* that minimize:

$$\min_{j,s} \left\{ \min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right\}.$$

The inner minimization is solved by:

$$\widetilde{c}_1 = \text{ave}(y_i | x_i \in R_1(j, s)), \quad \widetilde{c}_2 = \text{ave}(y_i | x_i \in R_2(j, s)).$$

For each splitting variable j, the best split point s can be found in $O(m)$ time. The overall complexity of the splitting process is $O(nm)$.

Tree Construction and Stopping Criteria

Regression trees are constructed recursively by partitioning the space at each step. However, stopping criteria must be defined to avoid overfitting:

- Stopping when the empirical error decreases beyond a threshold is insufficient since better splits may exist at deeper levels.
- Setting a maximum depth is also problematic as it may lead to underfitting or overfitting.
- A good stopping condition should balance tree depth and generalization performance.

Tree Pruning

To prevent overfitting, tree growth is controlled via:

- First grow a large tree T (stopping when a maximum number of data is assigned at each node)
- Next prune the tree using **cost-complexity pruning:** find the subtree T_λ minimizing:

$$C_\lambda(T) = \sum_{p \in \text{leaves of } T} m_p Q_p(T) + \lambda |T|,$$

  where $Q_p(T)$ is the impurity of node p and m_p is the number of points in p.
- Use cross-validation to choose λ^*, then prune T to obtain T_{λ^*}, an unique subtree that minimizes C_λ.
- C_λ is a kind of regularized empirical error: $T_0 = T$, the original tree. Large values of λ result in smaller trees. A good value of λ can be obtained by cross validation.

Weakest Link Pruning:

- Successively remove internal nodes with the smallest per-node increase in error.
- Continue pruning until only a single root node remains.
- Choose the optimal subtree T_{λ^*} using cross-validation.

Final Steps:

1. Use cross-validation to select the optimal λ^*.
2. Retrain the model on all data and prune to obtain T_{λ^*}.

5.2.2 Classification Trees

When the response variable is categorical $\mathcal{Y} = \{1, \ldots, K\}$, we modify regression trees as follows:

- Compute empirical class probabilities within each region R_P:

$$p_{pk} = \frac{1}{m_p} \sum_{x_i \in R_p} \mathbb{I}[y_i = k].$$

- Classify inputs based on the most probable class:

$$f(x) = \arg \max_{k=1,\ldots,K} \sum_{p=1}^{N} p_{pk} \mathbb{I}[x \in R_p].$$

- Use node impurity measures $Q_p(T)$ to assess splits.

Node Impurity Measures

- **Misclassification error:**

$$1 - p_{pk}(p), \quad \text{where } k(p) = \arg \max_{k=1,\ldots,K} p_{pk}.$$

- **Gini Index:**

$$\sum_k p_{pk}(1 - p_{pk}).$$

- **Cross-Entropy:**

$$\sum_k p_{pk} \log \frac{1}{p_{pk}}.$$

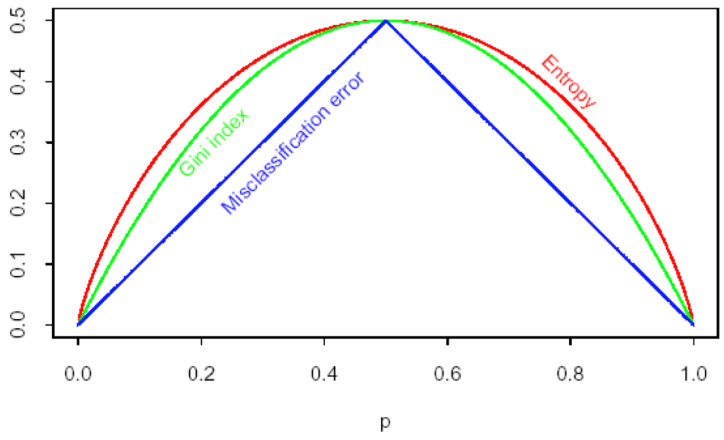

Figure 5.3: Classification Tree: Node Impurity Measures.

Usage:

- Gini index or cross-entropy is used to grow the tree, as they are more sensitive to changes in node class probabilities.
- Misclassification error is often used for pruning the tree.

Choosing Splitting Rules

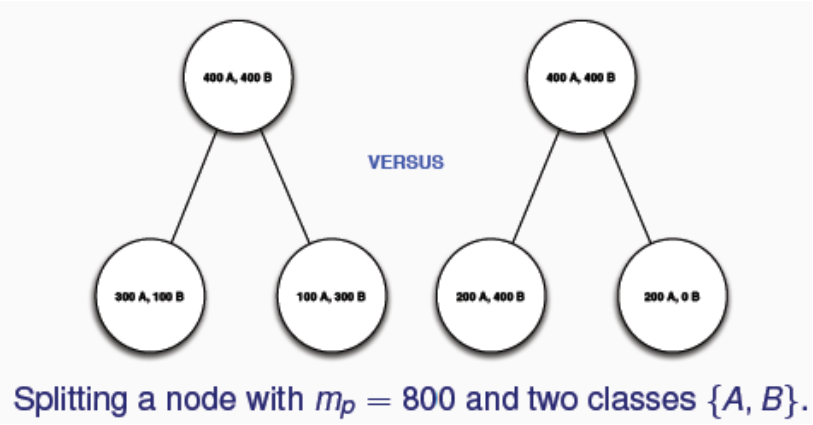

Figure 5.4: Splitting Rules in Classification Trees.

Why choose Gini or entropy as the splitting criterion? Consider splitting a node with $m_p = 800$ and two classes $\{A, B\}$. The reduction in impurity for different splitting strategies is computed as:

- **Misclassification Error Reduction:**

$$\text{Left: } 200 - \left(400 \times \frac{1}{4} + (400 \times \frac{1}{4})\right), \quad \text{Right: } 200 - (600 \times \frac{1}{3} + 200 \times 0).$$

- **Entropy Reduction:**

$$\text{Left: } 649.02 - 400 \times \left(\frac{1}{4}\log\frac{1}{4} + \frac{3}{4}\log\frac{3}{4}\right),$$

$$\text{Right: } 550.98 - 600 \times \left(\frac{1}{3}\log\frac{1}{3} + \frac{2}{3}\log\frac{2}{3}\right) + 200 \times 0.$$

Observing the reductions, the right split is preferable.

Remark. *The choice of impurity measure affects the split selection, influencing the final tree structure and its generalization ability.*

5.3 Bagging and Boosting

5.3.1 The Wisdom of the Crowd

A single individual might often be wrong, but the majority vote of a crowd may often be correct. This is based on the assumption that weak learners have individual correctness probability slightly better than random guessing.

Definition 5.3.1 (Idealized Scenario)**.** Suppose each individual classifier in a crowd $h_1, h_2, \ldots, h_{2T+1}$ independently predicts the correct outcome with probability $\frac{1}{2} + \gamma$.

The majority vote of the crowd is:

$$H_T := \text{sign}\left(\sum_{t=1}^{2T+1} h_t\right)$$

5.3.2 Computing the Probability of Error

The probability that H_T is incorrect is given by:

$$P(H_T \text{ is wrong}) = \sum_{i=0}^{T} \binom{2T+1}{i} \left(\frac{1}{2} + \gamma\right)^i \left(\frac{1}{2} - \gamma\right)^{2T+1-i}$$

This is difficult to compute exactly, so we use an upper bound.

Theorem 5.3.2 (Chernoff Bound)**.** *Let $X_1, X_2, \ldots, X_n$ be independent random variables where $0 \leq X_i \leq 1$. Define $X = \sum_{i=1}^{n} X_i$ and let $\mu = E[X] = \sum_{i=1}^{n} E[X_i]$. Then for all $0 \leq k \leq \mu$:*

$$P(X \leq k) \leq \exp\left(\frac{-(\mu - k)^2}{2\mu}\right).$$

Remark. *Using this bound for our scenario, we derive:*

$$P(H_T \textit{ is wrong}) \leq \exp\left(\frac{-4\gamma^2}{5}T\right).$$

This shows that the probability of error exponentially decays to zero.

5.3.3 Bagging (Bootstrap Aggregating)

Definition 5.3.3 (Bagging Algorithm)**.** A technique that reduces the variance of a classifier by training multiple classifiers and then voting to aggregate predictions.

Inputs:

1. Training data $\mathcal{S} = \{(x_1, y_1), \ldots, (x_m, y_m)\} \subset \mathbb{R}^d \times \{-1, 1\}$
2. Ensemble size T
3. Resample size M
4. Base classifier $h_{\mathcal{S}}(x)$

Algorithm:

1. For $t = 1$ to T:
   (a) Sample M examples **with replacement** from $\mathcal{S}$ to create $\mathcal{S}(t)$.
   (b) Train a classifier $h_{\mathcal{S}(t)}(x)$ on $\mathcal{S}(t)$.
2. Return the final classifier:
$$H(x) = \text{sign}\left(\sum_{t=1}^{T} h_{\mathcal{S}(t)}(x)\right).$$

Conventionally, we set $M = m$.

How Often Do Examples Appear in the Bag?

Consider the conventional advice to set $M = m$, then "roughly" how many unique examples from $\mathcal{S}$ are in the "bag $\mathcal{S}(t)$?

- The probability that a **particular** example does not appear in a bag is:

$$\left(1 - \frac{1}{m}\right)^m.$$

- Taking the limit:

$$\lim_{m\to\infty} \left(1 - \frac{1}{m}\right)^m = \frac{1}{e} \approx 0.368.$$

- Thus, roughly 63% of distinct examples appear in each $\mathcal{S}(t)$.

5.3.4 Random Forests

A refinement of bagging that introduces feature randomness to make trees more de-correlated:

- Decision trees are high variance models, making them ideal for bagging.
- To further reduce correlation, we select only a **subset** of features at each split.
- The subset size k is typically set to $\sqrt{n}$ or $\log n$.

5.3.5 Boosting

Spam Email Classification Example

Consider classifying emails as *spam* or *not spam*.

- Feature representation includes bag-of-words and additional metadata.
- Common classifiers: SVM, k-NN, CART.
- **Key observation:** Simple *rules of thumb* work, but they are not highly accurate.
- Example: "If 'business' appears in the email, predict spam."

Boosting refines these weak rules into a robust classification model.

Definition 5.3.4 (Weak Learner)**.** An algorithm that can consistently find classifiers (rules of thumb) at least slightly better than random guessing (e.g., above 55% accuracy).

Definition 5.3.5 (Boosting)**.** A general method of converting weak learners into a highly accurate classifier.

Boosting Algorithm:

1. Devise a computer program to derive rough rules of thumb.
2. Choose rules of thumb to fit a subset of examples.
3. Repeat T times.
4. Combine the classifiers via **weighted majority vote**.

Key ideas:

- How to choose the subset of examples at each round? Focus on the **hardest examples** (those most often misclassified by previous classifiers).
- Combine weak learners by **weighted majority**.

5.3.6 Adaboost

Definition 5.3.6 (Adaboost Algorithm)**.** A boosting algorithm that adaptively adjusts weights on training examples to focus on hard-to-classify cases.

Definitions:

- $D_t(i)$: Weight of example i at time t, where $\sum_{i=1}^{m} D_t(i) = 1$.
- α_t: Weight on weak learner t, where $\alpha_t \in \mathbb{R}$.
- $h_t(\cdot)$: Weak learner generated at time t, which is a function $h_t : \mathbb{R}^d \to \{-1, 1\}$.
- Aggregated classifier:
$$f(x) = \sum_{t=1}^{T} \alpha_t h_t(x).$$
- Final classifier:
$$H(x) = \text{sign}(f(x)).$$
- Weighted error of weak learner at time t:
$$\epsilon_t = \sum_{i=1}^{m} D_t(i) \mathbb{I}[h_t(x_i) \neq y_i].$$
- **Weak learner generator:** Given input
$$D_t(1), \ldots, D_t(m), (x_1, y_1), \ldots, (x_m, y_m)$$
outputs a weak learner $h_t(\cdot)$ such that $\epsilon_t < \frac{1}{2}$.

Adaboost Algorithm

Input: Training data $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$.

Initialization: Assign uniform weights to all data points:

$$D_1(i) = \frac{1}{m}, \quad \forall i.$$

For $t = 1, \ldots, T$:

1. Fit a weak classifier h_t using distribution D_t.
2. Compute the weighted error:
$$\epsilon_t = \sum_{i=1}^{m} D_t(i)\mathbb{I}[h_t(x_i) \neq y_i].$$
3. Compute classifier weight:
$$\alpha_t = \frac{1}{2}\log\left(\frac{1-\epsilon_t}{\epsilon_t}\right).$$
4. Update the distribution:
$$D_{t+1}(i) = \frac{D_t(i)e^{-\alpha_t y_i h_t(x_i)}}{Z_t},$$
where Z_t is a normalization factor to ensure that D_{t+1} is a valid probability distribution.

Return: Final classifier:

$$H(x) = \text{sign}\left(\sum_{t=1}^{T}\alpha_t h_t(x)\right).$$

Remark.
- *The basic idea in Adaboost is to **maintain a distribution** D on the training set and iteratively train a weak learner on it*
- *At each round larger weights are assigned to hard examples, hence the weak learner will focus mostly on those examples*

Weight by Majority

The final classifier is a weighted majority vote of the T weak classifiers:

$$f(x) = \sum_{t=1}^{T}\alpha_t h_t(x).$$

Since $\epsilon_t \leq 0.5$, it follows that $\alpha_t \geq 0$, ensuring that the weak learners contribute positively.

Thus, $f(x)$ is essentially a linear combination of the h_t classifiers, weighted by their training error.

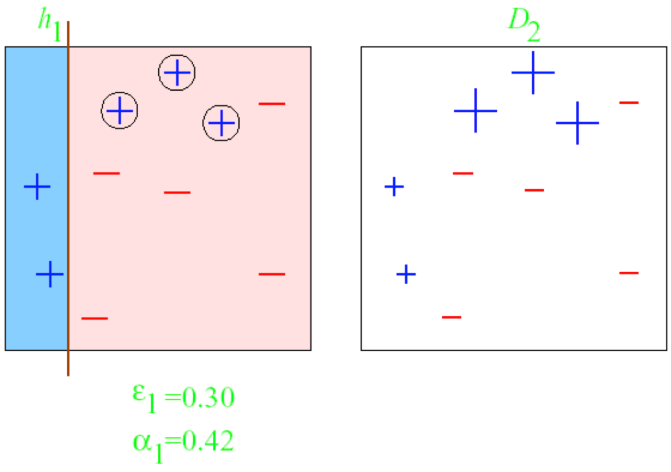

Figure 5.5: Round 1

Example (Round 1)

In the first round:

- The weak learner is a decision stump (vertical or horizontal splits).
- Some examples are misclassified, leading to a weighted error of $\epsilon_1 = 0.30$.
- The weight update gives $\alpha_1 = 0.42$.
- The next round will focus more on the misclassified examples.

Example (Round 2)

In the second round:

- The second weak learner adjusts to the misclassified examples.
- Weights are updated to emphasize harder cases.
- The weighted error improves to $\epsilon_2 = 0.21$, leading to a higher $\alpha_2 = 0.65$.

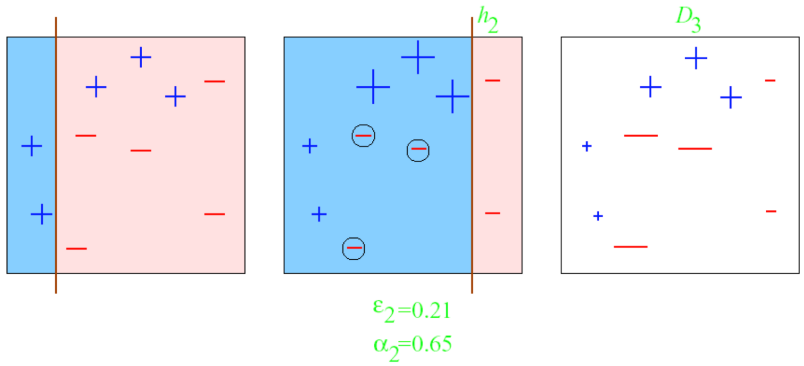

Figure 5.6: Round 2

Example (Round 3)

- The weak learner set can be finite.
- If using decision stumps, there are approximately $4m$ possible classifiers.
- The third weak learner achieves $\epsilon_3 = 0.14$, further increasing its weight $\alpha_3 = 0.92$.

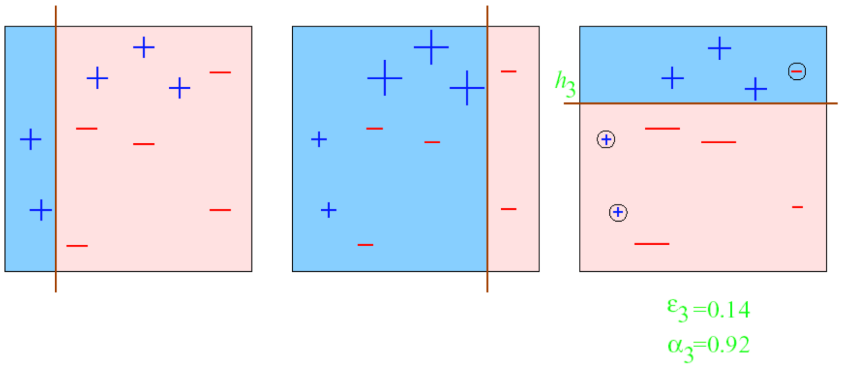

Figure 5.7: Round 3

Example (Final Round)

The final classifier has zero training error!

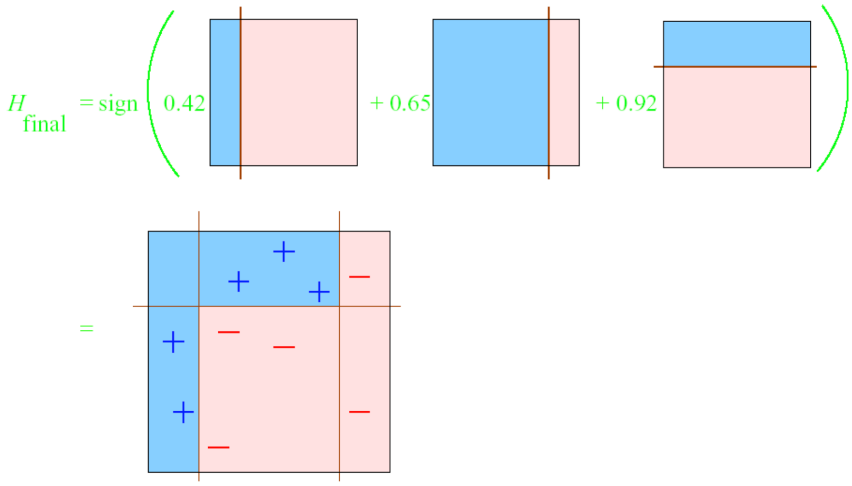

Figure 5.8: Final Round

Boosting Convergence

Theorem 5.3.7. *Given a training set* $\{(x_1, y_1), \ldots, (x_m, y_m)\}$ *and assuming that each iteration of Adaboost generates a weak learner with weighted error* $\epsilon_t \leq \frac{1}{2} - \gamma$*, the training error of the final hypothesis* H *satisfies the bound:*

$$\frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \exp(-2\gamma^2 T).$$

Analysis of the Training Error

The training error of Adaboost is bounded as follows:

$$\frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \frac{1}{m}\sum_{i=1}^{m} e^{-y_i f(x_i)} = \prod_{t=1}^{T} Z_t,$$

where $f(x) = \sum_t \alpha_t h_t(x)$ and $H(x) = \text{sign}(f(x))$.

Bounding the Training Error

1. The inequality follows from the fact that if $H(x_i) \neq y_i$, then $e^{-y_i f(x_i)} \geq 1$.

2. The equality follows from the recursive definition of the weight distribution D_t.

Expanding on the recursive definition:

$$D_{T+1}(i) = \frac{1}{m} \frac{\prod_{t=1}^{T} e^{-\alpha_t y_i h_t(x_i)}}{\prod_{t=1}^{T} Z_t}.$$

Minimizing the Training Error The goal is to minimize Z_t, which is defined as:

$$Z_t = \sum_i D_t(i) e^{-\alpha_t y_i h_t(x_i)}.$$

Since $h_t(x) \in \{-1, 1\}$, we can rewrite Z_t as:

$$Z_t = e^{\alpha_t} \sum_{i: y_i \neq h_t(x_i)} D_t(i) + e^{-\alpha_t} \sum_{i: y_i = h_t(x_i)} D_t(i).$$

Setting $\frac{dZ_t}{d\alpha_t} = 0$ leads to the optimal choice:

$$\alpha_t = \frac{1}{2} \log \frac{1 - \epsilon_t}{\epsilon_t}.$$

Final Training Error Bound Using the optimal α_t, the normalization factor Z_t simplifies to:

$$Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)} = \sqrt{1 - 4\gamma_t^2}.$$

Thus, the training error satisfies:

$$\frac{1}{m} \sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \prod_t Z_t \leq e^{-2\sum_t \gamma_t^2}.$$

Since $\gamma_t \geq \gamma > 0$, the training error decays exponentially:

$$\text{training error} \leq e^{-2T\gamma^2}.$$

Generalization Error

Dependence on T The training and test error exhibit different behaviors as T increases:

- When T is too large, over-fitting can occur.
- Typically, the test error continues to decrease even when the training error is zero.

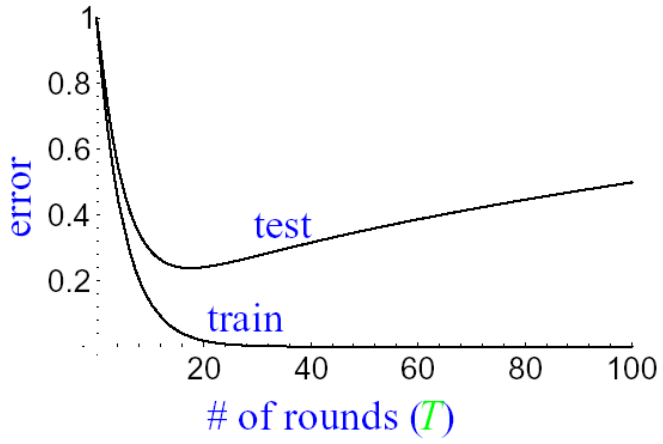

Figure 5.9: Generalization Error against number of rounds T.

Margin Distribution The reason test error improves is due to the concentration of the **margin distribution**, which explains why boosting remains effective even after training error is minimized.

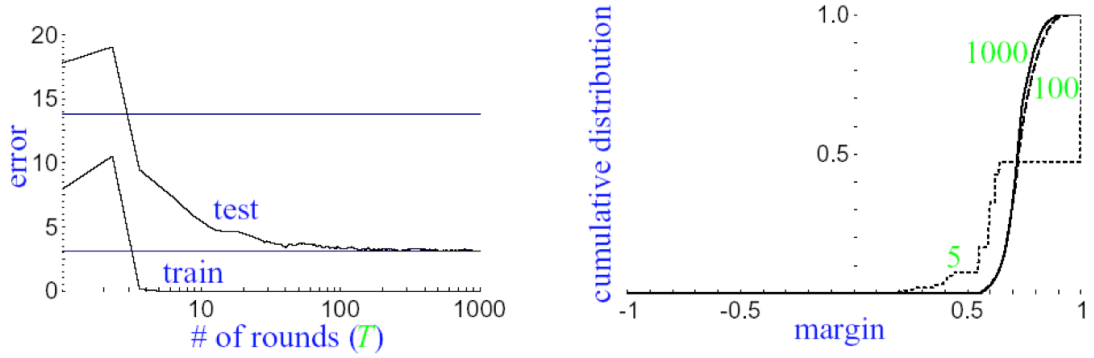

Figure 5.10: Marginal Distribution

5.3.7 A Motivation for Adaboost

Boosting can be seen as a greedy approach to solving the problem of finding a strong classifier by iteratively improving weak classifiers. The motivation behind boosting, and specifically Adaboost, lies in addressing two major problems in classification:

- The **misclassification loss** is not smooth, making it difficult to optimize.

- To make the function class $\mathcal{F}$ both *rich* and *smooth*, we typically map functions to $\mathbb{R}$ rather than $\{-1, 1\}$, and then predict using $\text{sign}(f)$.

Additive Models and Boosting

Boosting is formulated as a minimization problem over an additive model:

$$\min_{\alpha_1,\ldots,\alpha_T,h_1,\ldots,h_T\in\mathcal{H}} \sum_{i=1}^{m} V\left(y_i, \sum_{t=1}^{T} \alpha_t h_t(x_i)\right)$$

where $\mathcal{H}$ is the hypothesis class containing weak learners, and the loss function is typically exponential:

$$V(y, \hat{y}) = \exp(-y\hat{y}).$$

At each iteration, a new basis function is added:

$$f^{(t-1)} = \sum_{s=1}^{t-1} \alpha_s h_s$$

with updates given by:

$$(\alpha_t, h_t) = \arg\min_{\alpha_t,h_t} \sum_{i=1}^{m} V(y_i, f^{(t-1)}(x_i) + \alpha_t h_t(x_i)).$$

This is known as a **forward stagewise additive model**.

Why the Exponential Loss for Classification?

To better understand Adaboost, consider the following common loss functions:

$$V_{\text{MC}}(y, \hat{y}) = \mathbb{I}[y = \text{sign}(\hat{y})]$$

$$V_{\text{hinge}}(y, \hat{y}) = \max(0, 1 - y\hat{y})$$

$$V_{\text{exp}}(y, \hat{y}) = \exp(-y\hat{y})$$

$$V_{\text{sq}}(y, \hat{y}) = (y - \hat{y})^2$$

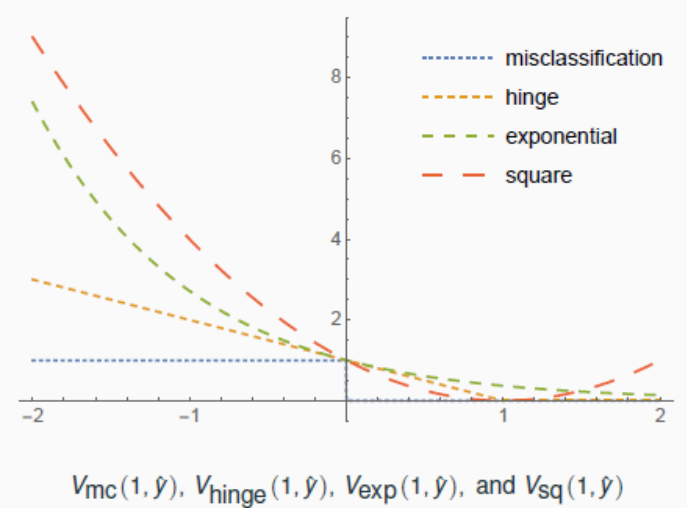

Figure 5.11: Loss Functions.

Some key observations:

- Misclassification loss is not continuous.
- Square loss penalizes large positive margins unnecessarily.
- Hinge loss only penalizes negative margins, but it is not differentiable everywhere.
- Exponential loss penalizes negative margins and encourages large positive margins.

Thus, exponential loss is chosen as the loss function in boosting due to its smoothness and ability to encourage large positive classification margins.

Boosting Convergence

Theorem 5.3.8. *Given a training set* $\{(x_1, y_1), \ldots, (x_m, y_m)\}$*, assume each iteration of Adaboost returns a weak learner with weighted error* $\epsilon_t \leq \frac{1}{2} - \gamma$*. Then, the training error of the final hypothesis satisfies:*

$$\frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \exp(-2\gamma^2 T).$$

This result shows that the training error decreases exponentially fast with the number of iterations T.

Boosting Summary

- Given a weak learner oracle, boosting combines weak hypotheses into a strong classifier.
- Boosting finds a linear combination of weak learners that minimizes training error.
- Theoretical guarantees ensure that, with certain assumptions, test error is also controlled.
- Boosting is best understood as a **forward stagewise additive model** with exponential loss.

Comparison with Decision Trees

Boosting and bagging are commonly applied to decision trees:

- Decision trees are useful in practice due to their interpretability.
- Ensemble methods (bagging/boosting) improve decision trees.
- **Bagging:** Uses full trees and reduces variance.
- **Boosting:** Uses decision stumps (one-level trees) and reduces bias.
- Boosting struggles with very noisy data.

Chapter 6

Statistical Learning Theory (Part I)

6.1 Key Questions in Learning Theory

Learning theory investigates why, when, and how supervised learning algorithms work. The main objectives will be covered include:

- Understanding **generalization error** and **excess risk**.
- **Studying consistency:** Does excess risk $E(f_n) - E(f^*) \to 0$ as $n \to \infty$?
- **Learning rates:** How quickly does this convergence occur for excess risk?
- Deriving non-asymptotic bounds for finite samples: complexity, tail bounds, error bounds etc.

6.2 Expected Risk and Empirical Risk Minimization

6.2.1 Expected Risk

Definition 6.2.1 (Expected Risk)**.** The goal of supervised learning is to find a "good" estimator $f^* : \mathcal{X} \to \mathcal{Y}$ that minimizes the expected risk. The *expected risk* of a function $f : \mathcal{X} \to \mathcal{Y}$ is defined as:

$$E(f) = \int_{\mathcal{X}\times\mathcal{Y}} \ell(f(x), y)\, d\rho(x, y),$$

where:

- $\mathcal{X}$ and $\mathcal{Y}$ denote the input and output spaces.
- $\rho(x, y)$ is the true joint distribution of input-output pairs (probably unknown).
- $\ell(f(x), y)$ is the loss incurred by predicting $f(x)$ when the true label is y (e.g. square loss).

Remark. *The expected risk $E(f)$ is the true measure of a model's performance but is generally inaccessible since the* ***data distribution*** *$\rho(x, y)$* ***is unknown****.*

6.2.2 Empirical Risk

Definition 6.2.2 (Empirical Risk)**.** Since the true distribution ρ is unknown, we approximate it by a proxy model: empirical risk. Given a finite training set $S = \{(x_i, y_i)\}_{i=1}^n$, the *empirical risk* of a function f is defined as:

$$E_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i),$$

i.e. the mean training loss. where:

- n is the number of training examples.
- (x_i, y_i) are drawn i.i.d. from $\rho(x, y)$.

The key idea is that the empirical risk approximates the expected risk when n is large due to **the Law of Large Numbers** (LLN).

6.2.3 Empirical Risk Vs Expected Risk

Theorem 6.2.3 (Unbiased Estimator)**.** *The empirical risk $E_n(f)$ is an* ***unbiased estimator*** *of the expected risk $E(f)$ because the expectation of empirical risk is just the expected risk:*

$$\mathbb{E}_{S\sim\rho^n}[E_n(f)] = E(f).$$

Proof. Expanding the expectation of empirical risk:

$$\mathbb{E}_{S\sim\rho^n}[E_n(f)] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{(x_i,y_i)\sim\rho}[\ell(f(x_i), y_i)] = \frac{1}{n}\sum_{i=1}^{n} E(f) = E(f).$$

□

Remark. *Although empirical risk is an unbiased estimator of the expected risk, its* ***variance*** *depends on* ***the training sample size*** *n* ***and the variability of the loss*** *$\ell(f(x), y)$.*

Theorem 6.2.4 (Convergence of Empirical Risk)**.** *As the training sample size n increases, the expected* ***squared distance*** *between $E_n(f)$ and $E(f)$ goes to 0. i.e. The empirical risk $E_n(f)$ converges to the expected risk $E(f)$:*

$$\mathbb{E}[(E_n(f) - E(f))^2] = \frac{Var(\ell(f(x), y))}{n}.$$

For any function f, the expected deviation (actual distance) between $E_n(f)$ and $E(f)$ is bounded by:

$$\mathbb{E}[|E_n(f) - E(f)|] \leq \sqrt{\frac{\text{Var}(\ell(f(x), y))}{n}}.$$

which is a concentration bound that controls the deviation probability. It follows from Jensen's inequality, which relates squared expectation to absolute expectation.

Remark. *The **convergence rate** $O(1/\sqrt{n})$ comes from **Central Limit Theorem**: the standard deviation of the sample mean decreases as $1/\sqrt{n}$. This rate depends on the variance of the loss function and highlights the need for **sufficient training data** to ensure accurate empirical risk estimation.*

6.2.4 Excess Risk Decomposition and Minimization

Theorem 6.2.5 (Excess Risk Decomposition)**.** *Excess risk measures how much worse the function f_n of your choice performs compared to the minimizer $f*$ of the expected risk. The **excess risk** for any function $f : \mathcal{X} \to \mathcal{Y}$ can be decomposed as:*

$$E(f_n) - E(f^*) = (E(f_n) - E_n(f_n)) + (E_n(f_n) - E_n(f^*)) + (E_n(f^*) - E(f^*)),$$

where:

- *f_n is any function of your choice and f^* is the optimal function that minimizes the expected risk $E(f)$.*
- *$E(f_n) - E_n(f_n)$: Generalization error (expected risk - empirical risk): measures how well the empirical risk estimates the true risk, how much the learned function generalizes to unseen data.*
  - *Depends on the sample size n: smaller datasets tend to have a larger generalization error.*
- *$E_n(f_n) - E_n(f^*)$: Empirical error difference between f_n and $f*$: measures how well the ERM procedure performs in minimizing empirical risk.*
  - *If ERM finds the best function of choice f_n, this term should $\to 0$.*
  - *Depends on the complexity of the function space: If optimization is hard, this term could be large.*
- *$E_n(f^*) - E(f^*)$: measure difference between empirical risk and expected risk of the true optimal function $f*$.*
  - *This error arises because **we only have a finite sample**: Even the best function might not have perfect empirical risk due to randomness in the data.*
  - *Diminishes as the sample size n grows.*

Definition 6.2.6 (ERM Principle)**.** The empirical risk minimization (ERM) principle chooses the function $f_n \in \mathcal{F}$ that **minimizes the empirical risk**:

$$f_n = \arg\min_{f \in \mathcal{F}} E_n(f).$$

so $E_n(f_n)$ - $E_n(f^*)$ must be non-positive for any training set. (Why? because f_n is chosen to minimize the empirical risk.)

Theorem 6.2.7 (ERM Generalization Error Bound)**.** *For the ERM solution f_n, the **generalization error** is bounded by:*

$$\mathbb{E}[E(f_n) - E(f^*)] \leq \mathbb{E}[E(f_n) - E_n(f_n)].$$

- $\mathbb{E}[E(f_n) - E(f^*)]$: Excess risk (how much worse f_n is than f^*).
- $\mathbb{E}[E(f_n) - E_n(f_n)]$: Generalization error (how much the expected risk deviates from the empirical risk of f_n).

Remark. *The ERM principle relies on the assumption that minimizing empirical risk $E_n(f)$ leads to a corresponding reduction in expected risk $E(f)$. Over-fitting occurs when this assumption fails: when the function class is too complex (too many parameters), then f_n may over-fit (very low empirical risk but very high expected risk) and generalization error is very large.*

6.2.5 Generalization Error in ERM

Definition 6.2.8 (Generalization Error in ERM)**.** The *generalization error* of a function f_n measures the discrepancy between the **expected risk** $E(f_n)$ **and the empirical risk** $E_n(f_n)$ as:

$$\text{Generalization Error} = E(f_n) - E_n(f_n).$$

The generalization error depends on the **complexity of the hypothesis space** $\mathcal{F}$ and **the number of training samples** n**.**

Theorem 6.2.9 (Controlling Generalization Error)**.** *In general:*

$$\mathbb{E}[E_n(f_n) - E(f_n)] \neq 0,$$

because both $E_n(f_n)$ and f_n depend on the training data. This invalidates the standard concentration bound:

$$\mathbb{E}[|E_n(f_n) - E(f_n)|] \leq \sqrt{\frac{Var(\ell(f_n(x), y))}{n}},$$

which holds for fixed function f but not for f_n chosen through empirical risk minimization. Therefore, even though we can bound generalization error for a fixed function, doing so for ERM-selected functions is much harder.

Remark. *The above discrepancy highlights a key challenge in statistical learning theory: understanding the interaction between the empirical risk minimizer f_n and the training data.*

6.2.6 Issues with Empirical Risk Minimization

Definition 6.2.10 (Overfitting)**.** An estimator f_n is said to *overfit* the training data if:

- The excess risk $\mathbb{E}[E(f_n) - E(f^*)] > C$ for some constant $C > 0$
  - f_n does not generalize well, even with increasing training data.
- The empirical risk difference $\mathbb{E}[E_n(f_n) - E_n(f^*)] \leq 0$
  - f_n fits the training data very well (possibly too well!).

Example 6.2.11 (Overfitting Example in ERM)**.** Let $\mathcal{X} = \mathcal{Y} = \mathbb{R}$, $\ell(y, y') = 0$ for $y = y'$, and let $\rho(x, y)$ be dense distribution (i.e. we see every possible value infinitely often). Define $f_n(x)$ as:

$$f_n(x) = \begin{cases} y_i, & \text{if } x = x_i \text{ for some } i \in \{1, \dots, n\}, \\ 0, & \text{otherwise.} \end{cases}$$

i.e. f_n memorizes the training data exactly. Then, For any number n of training points:

- Empirical Risk $\mathbb{E}[E_n(f_n)] = 0$,
- $\mathbb{E}[E(f_n)] = E(0)$, which is greater than $E(f^*)$ unless $f^* \equiv 0$. i.e. $E(f_n) > E(f^*)$

This shows that $E[E(f_n) - E_n(f_n)] = E(0) \not\to 0$ as $n \to \infty$, leading to over-fitting. Hence, when the model memorizes the data, leading to zero empirical risk but high expected risk, will have over-fitting issues.

6.3 ERM on Finite Hypothesis Spaces

6.3.1 Finite Hypothesis Spaces

Theorem 6.3.1 (Finite Hypothesis Spaces)**.** *When the hypothesis space $\mathcal{F}$ is finite, empirical risk minimization (ERM) has a **meaningful bound** on its generalization error. Specifically, for any $f_n \in \mathcal{F}$, the following inequality holds:*

$$\mathbb{E}|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \mathbb{E} \sup_{f \in \mathcal{F}} |\mathcal{E}_n(f) - \mathcal{E}(f)|$$

which holds as the generalization error cannot exceed the worst-case generalization error across the entire hypothesis space $\mathcal{F}$. Then, using Hoeffding 's Inequality, we get:

$$\leq \sum_{f \in \mathcal{F}} \mathbb{E}|\mathcal{E}_n(f) - \mathcal{E}(f)| \leq |\mathcal{F}| \sqrt{\frac{V_{\mathcal{F}}}{n}},$$

where:

- $V_{\mathcal{F}} = \sup_{f \in \mathcal{F}} V_f$ *is fixed and is the maximum variance of the loss function over* $\mathcal{F}$
- $|\mathcal{F}|$ *denotes the size of the hypothesis space* $\mathcal{F}$
- n *is the number of training samples*

The sum of expected errors for each f *in* $\mathcal{F}$ *is valid because* $\mathcal{F}$ *is* ***finite****, so the supremum can't exceed* ***sum of all errors****.* ***The third inequality uses the Hoeffding's Inequality.***

Here, we can observe that:

- *The bound shows that generalization error increases with* $|\mathcal{F}|$*: If* $\mathcal{F}$ *is large, the worst-case gap between empirical and expected risk increases.*
- *ERM works well if* $n \to \infty$ *because the generalization error will approach* 0*.*

Theorem 6.3.2 (Subspaces of Finite Hypotheses)**.** *Let* $\mathcal{H} \subset \mathcal{F}$ *be a* ***finite subspace*** *of hypotheses (even if* $\mathcal{F}$ *itself is not finite) and* $f^* \in \mathcal{H}$*. In this case, a similar generalization bound holds:*

$$E[|E(f_n) - E(f^*)|] \leq |\mathcal{H}| \sqrt{\frac{V_{\mathcal{H}}}{n}},$$

where $|\mathcal{H}|$ *represents the size of the subspace* $\mathcal{H}$ *and* $V_{\mathcal{H}}$ *is the maximum variance of the loss function over* $\mathcal{H}$*. Here, restricting the hypothesis space from* $\mathcal{F}$ *to* $\mathcal{H}$ *reduces generalization error because* $|\mathcal{H}|$ *is smaller.*

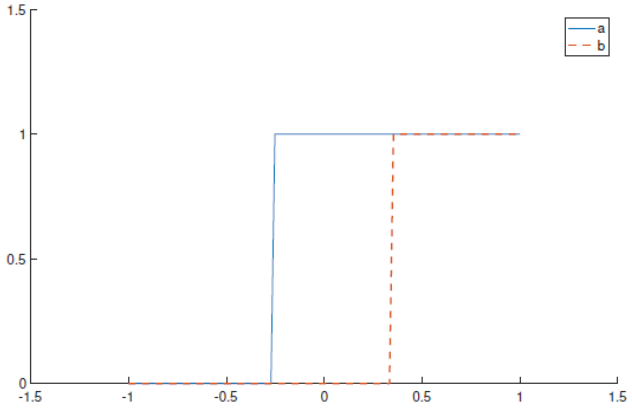

Figure 6.1: Threshold Functions.

Example 6.3.3 (Threshold Functions)**.** Consider a binary classification problem with $\mathcal{Y} = \{0, 1\}$. The true minimizer of the risk is a **"threshold function"** $f_a^*(x) = \mathbb{1}_{x \geq a^*}$, where $a^* \in [-1, 1]$ is the threshold.

In practice:

- On a computer, real numbers can only be represented **up to a given finite precision** p.
- To discretize the hypothesis space, we define a set of threshold values ($p > 0$):
  - $\mathcal{H}_p = \{a \in [-1, 1] \mid a = \lfloor a \cdot 10^p \rfloor / 10^p\}$
  - The number of possible thresholds is $|\mathcal{H}_p| = 2 \cdot 10^p$.

For such a discretized space, the generalization bound becomes:

$$E[|E(f_n) - E(f^*)|] \leq |\mathcal{H}_p| \sqrt{\frac{V_{\mathcal{H}_p}}{n}} \leq 10^p \sqrt{\frac{V_{\mathcal{H}_p}}{n}}.$$

This demonstrates that **increasing precision** p (i.e. making the space more expressive) or **reducing training sample size** n can lead to higher generalization error. This is a trade-off between model complexity and generalization.

Theorem 6.3.4 (Rates in Expectation vs Probability)**.** *In practice, even for small values of p, the bound:*

$$E[|E(f_n) - E(f^*)|] \leq 10^p \sqrt{\frac{1}{n}}$$

*requires **very large** n **to provide meaningful bounds** on the expected error. However, better constants (though not rates) and tighter confidence intervals can be achieved when working with **probability bounds** instead of expectation bounds.*

Remark. *This analysis highlights the trade-off between hypothesis space complexity (e.g., precision p) and the number of training samples n, emphasizing the importance of balancing these factors to achieve good generalization.*

6.3.2 Hoeffding's Inequality

Definition 6.3.5 (Hoeffding's Inequality Definition)**.** Let $X_1, X_2, \ldots, X_n$ be independent random variables such that $X_i \in [a_i, b_i]$ for all i. Define the sample mean as:

$$\overline{X} = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

Then, for any $\epsilon > 0$, the probability that the sample mean $\overline{X}$ deviates from the true expected mean $E[\overline{X}]$ is bounded by:

$$P\left(|\overline{X} - \mathbb{E}[\overline{X}]| \geq \epsilon\right) \leq 2 \exp\left(-\frac{2n^2\epsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right).$$

- *The probability of a large deviation from the mean* ***decreases exponentially*** *with n, meaning more samples lead to more precise estimates*
- *The bound* ***only depends on the range*** *$[a_i, b_i]$, making it* ***distribution-independent****.*
- *Hoeffding's inequality is particularly useful when the random variables have bounded support, as it only depends on the bounds $[a_i, b_i]$ and not on the specific distributions of the variables.*

Theorem 6.3.6 (Generalization Bound Using Hoeffding's Inequality)**.** *Consider a loss function $\ell(f(x), y)$ that is bounded by a constant $M > 0$ for all $f \in \mathcal{H}$, $x \in \mathcal{X}$, and $y \in \mathcal{Y}$, such that $|\ell(f(x), y)| \leq M$. Then, the generalization error for any hypothesis $f \in \mathcal{H}$ satisfies:*

$$P(|\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon) \leq 2 \exp\left(-\frac{n\epsilon^2}{2M^2}\right).$$

- *Large M (more variability in loss) leads to looser bounds*
- *More samples n improve generalization by reducing error probability*
- *Tighter bounds on the loss function improve generalization*

Proof. The empirical risk $\mathcal{E}_n(f)$ is the sample mean of the loss values on the training data:

$$\mathcal{E}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

By assumption, the loss values $\ell(f(x_i), y_i)$ are bounded within $[-M, M]$. Hoeffding's inequality applies directly to the sample mean, yielding the stated result. □

Theorem 6.3.7 (Union Bound for Multiple Hypotheses)**.** *To extend the generalization bound to a finite hypothesis space $\mathcal{H}$, we use the* ***union bound****. The probability that the generalization error exceeds ϵ for* ***at least one hypothesis*** *$f \in \mathcal{H}$ satisfies:*

$$P\left(\sup_{f \in \mathcal{H}} |\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon\right) \leq \sum_{f \in \mathcal{H}} P(|\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon).$$

Applying Hoeffding's inequality for each $f \in \mathcal{H}$, we have:

$$P\left(\sup_{f \in \mathcal{H}} |\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon\right) \leq 2|\mathcal{H}| \exp\left(-\frac{n\epsilon^2}{2M^2}\right).$$

Remark (Significance of the Union Bound)**.** *The union bound ensures that the generalization bound applies to the entire hypothesis space $\mathcal{H}$. However, it introduces a* ***dependence on the size of the hypothesis space*** *$|\mathcal{H}|$, which can lead to looser bounds if $|\mathcal{H}|$ is large. This highlights the importance of* ***controlling the complexity (e.g. regularization) of the hypothesis space*** *in machine learning.*

Remark (Generalization Bound in Probability)**.** *For a fixed confidence level $\delta \in (0, 1)$, the generalization error for the ERM hypothesis $f_n \in \mathcal{H}$ satisfies:*

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{2M^2 \log(2|\mathcal{H}|/\delta)}{n}}$$

with probability at least $1 - \delta$. This is a high-probability bound, meaning it holds for most cases except for a small fraction δ.

This bound depends on:

- *The complexity $\mathcal{H}$ via $log|\mathcal{H}|$ (using log to improve scalability)*
- *The number of training samples n (more data reduces the bound)*
- *The confidence level δ*

Example 6.3.8 (Threshold Functions in Probability)**.** Consider the hypothesis space $\mathcal{H}_p$ of threshold functions with precision p, defined as:

$$\mathcal{H}_p = \{f_a \mid a \cdot 10^p \in \mathbb{Z}, a \in [-1, 1]\}, \quad |\mathcal{H}_p| = 2 \cdot 10^p.$$

The loss function is bounded by $M = 1$. Applying the generalization bound, we obtain the probability bound:

$$P(|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \geq \epsilon) \leq 2 \cdot 10^p \exp\left(-\frac{n\epsilon^2}{2}\right).$$

Alternatively, for a confidence level $1 - \delta$, the bound becomes:

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{4 + 6p - 2\log(\delta)}{n}}.$$

For example, if $p = 3$ and $\delta = 0.001$, we have:

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{6p + 18}{n}}.$$

This holds with 99.9% confidence. More precision p **increases generalization error** unless n grows.

Remark (Bounds in Expectation vs. Probability)**.** *The generalization error can also be analyzed in **expectation**. Recall that:*

$$\mathbb{E}[|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)|] \leq 10^p/\sqrt{n}.$$

*However, the probabilistic bound provides a **tighter guarantee** for high-confidence scenarios:*

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{6p+18}{n}}$$

*with a confidence level of 99.9%. Both **bounds converge to** 0 at the rate $O(1/\sqrt{n})$, but the probabilistic bound is **more informative** for practical applications since they give high-confidence guarantees. Expectation bounds are **weaker** because they **do not account for worst-case scenarios**.*

Theorem 6.3.9 (Improving the Bound in Expectation)**.** *Using the probabilistic bound and the fact that $|X| < M$ for a random variable X, we can refine the expectation bound. For any $\epsilon > 0$ we have:*

$$\mathbb{E}|X| \leq \epsilon P(|X| \leq \epsilon) + M\mathbb{P}(|X| > \epsilon)$$

For any $\delta \in (0,1)$, the expectation of the generalization error satisfies:

$$\mathbb{E}[|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)|] \leq (1-\delta)\sqrt{\frac{2M^2 \log(2|\mathcal{H}_p|/\delta)}{n}} + \delta M.$$

*This result shows that the dependence on $|\mathcal{H}_p|$ becomes **logarithmic**, making the bound more practical for **large hypothesis spaces**. This bound also balances precision p, data size n, and generalization error.*

6.3.3 Infinite Hypothesis Spaces

Definition 6.3.10. If $f^* \notin \mathcal{H}_p$ for any $p > 0$,ERM on $\mathcal{H}_p$ will never minimize the expected risk, leading to a persistent gap:

$$E(f_n, p) - E(f^*).$$

For $p \to +\infty$, it is natural to expect this gap to **decrease**. However, if p increases too quickly with n, **the generalization error may become unbounded**:

$$|E_n(f_n) - E(f_n)| \leq \sqrt{\frac{6p+18}{n}} \to +\infty \quad \text{as } p \to +\infty.$$

Remark (Regularization)**.** *To address this, p (model complexity) must **increase gradually as a function $p(n)$ of the number of training samples n.** This controlled growth is known as regularization.*

Approximation Error for Threshold Functions

Theorem 6.3.11 (Decomposition of Excess Risk)**.** *The excess risk for threshold functions can be decomposed as:*

$$E(f_n) - E(f^*) = \underbrace{(E(f_n) - E_n(f_n)) + (E_n(f_n) - E_n(f_p))}_{\textit{Generalization Error (bounded)}} + \underbrace{(E_n(f_p) - E(f_p)) + (E(f_p) - E(f^*))}_{\textit{Approximation Error}},$$

where f_p is the best function in $\mathcal{H}_p$.

- $E(f_n) - E_n(f_n)$: *Generalization Error. Bounded by probabilistic guarantees.*
- $E_n(f_n) - E_n(f_p)$: *Empirical Error Difference. Different in empirical risks due to finite sample effects.*
- $E_n(f_p) - E(f_p)$: *Generalization Error of f_p. Still probabilistically bounded.*
- $E(f_p) - E(f^*)$: *Approximate Error. Depends on* ***the unknown data distribution*** *instead of the training data, and depend on* ***the expressiveness of*** $\mathcal{H}$ *. It can't be controlled without additional assumptions.* $\surd$

Remark. *The generalization error is bounded by probabilistic bounds derived earlier, but the approximation error $E(f_p) - E(f^*)$ depends on the unknown distribution ρ and characteristics of the problem.*

Definition 6.3.12 (Regularity Assumption)**.** To study $E(f_p) - E(f^*)$, assume the **marginal distribution** ρ_X has density $r_p(x)$ with support $\mathcal{X} = [-1, 1]$. For any integrable $g : \mathcal{X} \to \mathbb{R}$:

$$\int_{-1}^{1} g(x) d\rho_X(x) = \int_{-1}^{1} g(x) r_p(x) dx.$$

where we are asking the learning problem (through p) to satisfy some regularity assumption (i.e. it cannot be any arbitrarily "bad" learning problem).

Theorem 6.3.13 (Approximation Error for Threshold Functions)**.** *Under the regularity assumption:*

$$E(f_p) - E(f^*) \leq |a_p - a^*| \cdot \|r_p\|_\infty \cdot 10^{-p},$$

where $\|r_p\|_\infty = \sup_{x \in \mathcal{X}} |r_p(x)|$ is the sup norm of $r_p(x)$. The term 10^{-p} shows that approximation error ***decreases exponentially*** *with increasing p.*

Remark. *The approximation error decreases with increasing p,* ***but its rate depends on the regularity of*** *$r_p(x)$* ***and the precision*** *10^{-p}.*

Theorem 6.3.14 (Balancing Generalization and Approximation Errors - Optimal Precision)**.** *For any $\delta \in (0, 1)$, the total* ***excess risk*** *satisfies:*

$$E(f_n) - E(f^*) \leq 2\sqrt{\frac{4 + 6p - 2\log(\delta)}{n}} + \|r_p\|_\infty \cdot 10^{-p}.$$

The optimal precision $p(n, \delta)$ ***minimizes this bound****:*

$$p(n, \delta) = \arg\min_{p \geq 0} \phi(n, \delta, p),$$

where:

$$\phi(n, \delta, p) = 2\sqrt{\frac{4 + 6p - 2\log(\delta)}{n}} + \|r_p\|_\infty \cdot 10^{-p}.$$

Remark. *The trade-off between generalization and approximation errors determines the optimal choice of p. Larger n and smaller δ favor higher p, achieving better approximation without over-fitting. Choosing p too large leads to poor generalization (overfitting).*

6.3.4 Regularization and Complexity

Definition 6.3.15 (Regularization)**.** Regularization is the process of **controlling the complexity or "freedom" of the hypothesis space** as a function of the number of training examples. The goal is to mitigate over-fitting by **restricting the hypothesis space** in a way that adapts to the available data size.

*In many learning problems, the hypothesis space $\mathcal{H}$ is **excessively large**, leading to over-fitting when the training data size n is small. Regularization offers a principled way to address this by **gradually increasing the complexity of the hypothesis space as the number of training examples grows**.*

Theorem 6.3.16 (Regularization Parameterization)**.** *Let $\mathcal{H}$ be the union of a family of hypothesis spaces $\mathcal{H}_\gamma$, parameterized by a regularization parameter $\gamma > 0$:*

$$\mathcal{H} = \bigcup_{\gamma > 0} \mathcal{H}_\gamma.$$

*The parameter γ determines the **level of complexity** of $\mathcal{H}_\gamma$, with larger values of γ allowing for more complex hypotheses. To ensure consistency, assume that $\mathcal{H}_\gamma \subseteq \mathcal{H}_{\gamma'}$ for $\gamma \leq \gamma'$.*

Regularization can be interpreted as the process of choosing an appropriate $\mathcal{H}_\gamma$ for a given training dataset, with γ growing as $n \to \infty$ to allow more complex models when more data are available.

Remark (Regularization Algorithm)**.** *Given n training points, the regularization algorithm selects an estimator $f_{\gamma,n} \in \mathcal{H}_\gamma$ by solving an **optimization problem**, such as **empirical risk minimization (ERM)** over $\mathcal{H}_\gamma$. The regularization parameter $\gamma(n)$ is chosen to increase as the training size n grows, ensuring that **the hypothesis space adapts to the data size**.*

Theorem 6.3.17 (Decomposition of the Excess Risk under Regularization)**.** *For a given regularization parameter $\gamma > 0$, let $f_\gamma = \arg\min_{f \in \mathcal{H}_\gamma} \mathcal{E}(f)$ be the hypothesis that **minimizes the expected risk** over $\mathcal{H}_\gamma$. The excess risk can be decomposed into three terms:*

$$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_*) = \underbrace{\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_\gamma)}_{\textit{Sample Error}} + \underbrace{\mathcal{E}(f_\gamma) - \inf_{f \in \mathcal{H}} \mathcal{E}(f)}_{\textit{Approximation Error}} + \underbrace{\inf_{f \in \mathcal{H}} \mathcal{E}(f) - \mathcal{E}(f_*)}_{\textit{Irreducible Error}}.$$

Sample Error: *This term measures the deviation between the empirical minimizer $f_{\gamma,n}$ and the best hypothesis f_γ within $\mathcal{H}_\gamma$. It decreases as n increases, given enough training data.*

Approximation Error: *This term represents the sub-optimality of the hypothesis space $\mathcal{H}_\gamma$ compared to the full hypothesis space $\mathcal{H}$. It depends on the choice of γ; smaller γ restricts $\mathcal{H}_\gamma$, leading to a larger approximation error.*

Irreducible Error: *This is the error inherent to the problem, defined as the difference between the expected risk of the best hypothesis in $\mathcal{H}$ and the Bayes optimal hypothesis f_*. This term does not depend on the choice of γ or $\mathcal{H}_\gamma$.*

Approximation Error

Definition 6.3.18 (Approximation Error)**.** The approximation error quantifies the discrepancy between the expected risk of the best hypothesis in $\mathcal{H}_\gamma$ and the infimum of the expected risk over the entire hypothesis space $\mathcal{H}$:

$$\mathcal{E}(f_\gamma) - \inf_{f \in \mathcal{H}} \mathcal{E}(f).$$

Theorem 6.3.19 (Convergence of the Approximation Error)**.** *Under **mild assumptions**, the approximation error **converges to zero** as the **regularization parameter** γ increases:*

$$\lim_{\gamma \to \infty} \mathcal{E}(f_\gamma) - \inf_{f \in \mathcal{H}} \mathcal{E}(f) = 0.$$

Remark (Density Results)**.** *If $\mathcal{H}$ is a **universal** hypothesis space, the approximation error also converges to the **Bayes risk**:*

$$\lim_{\gamma \to \infty} \mathcal{E}(f_\gamma) - \mathcal{E}(f_*) = 0.$$

*This relies on the density property of $\mathcal{H}$, **ensuring that $\mathcal{H}_\gamma$ becomes increasingly expressive as γ grows**.*

Theorem 6.3.20 (Approximation Error Bounds)**.** *Without specific assumptions about the regularity of f_*, **no rates for the approximation error can be guaranteed** (related to the No Free Lunch Theorem). However, for s-regular functions, the approximation error satisfies:*

$$\mathcal{E}(f_\gamma) - \inf_{f \in \mathcal{H}} \mathcal{E}(f) \leq \mathcal{A}(\rho, \gamma),$$

where $\mathcal{A}(\rho, \gamma) = c_\gamma \gamma^{-s}$, and c_γ depends on the regularity of the target function and the structure of $\mathcal{H}_\gamma$.

Example 6.3.21 (Approximation Theory Tools)**.** Approximation error bounds are often studied using advanced tools like:

- Kolmogorov n-width,
- K-functionals,
- Interpolation spaces.

Sample Error

Definition 6.3.22 (Sample Error)**.** The sample error measures the difference between the empirical risk minimizer $f_{\gamma,n}$ and the expected risk minimizer f_γ within $\mathcal{H}_\gamma$:

$$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_\gamma).$$

Remark (Random Nature of Sample Error)**.** *The sample error is a* ***random quantity*** *because it depends on the specific training data. Two primary methods are used to study it:*

- *Capacity/complexity estimates of $\mathcal{H}_\gamma$,*
- *Stability analysis of the learning algorithm.*

Theorem 6.3.23 (Sample Error Decomposition)**.** *The sample error can be further decomposed as:*

$$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_\gamma) = (\mathcal{E}(f_{\gamma,n}) - \mathcal{E}_n(f_{\gamma,n})) + (\mathcal{E}_n(f_{\gamma,n}) - \mathcal{E}_n(f_\gamma)) + (\mathcal{E}_n(f_\gamma) - \mathcal{E}(f_\gamma)),$$

where:

- *$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}_n(f_{\gamma,n})$: Generalization error of $f_{\gamma,n}$,*
- *$\mathcal{E}_n(f_{\gamma,n}) - \mathcal{E}_n(f_\gamma)$: Excess empirical risk due to finite samples,*
- *$\mathcal{E}_n(f_\gamma) - \mathcal{E}(f_\gamma)$: Generalization error of f_γ.*

Remark (Controlling the Generalization Error)**.** *The terms $\mathcal{E}(f_{\gamma,n}) - \mathcal{E}_n(f_{\gamma,n})$ and $\mathcal{E}_n(f_\gamma) - \mathcal{E}(f_\gamma)$ can be controlled using* ***empirical process theory****. Specifically, for a finite hypothesis space $\mathcal{H}_\gamma$, we have:*

$$\mathbb{E}\left[\sup_{f\in\mathcal{H}_\gamma} |\mathcal{E}_n(f) - \mathcal{E}(f)|\right] \leq |\mathcal{H}_\gamma|\sqrt{\frac{V_{\mathcal{H}_\gamma}}{n}},$$

where $V_{\mathcal{H}_\gamma}$ is the variance of the loss over $\mathcal{H}_\gamma$.

Key Takeaways:

- *Generalization error decreases with n*
- *Hypothesis space complexity $|\mathcal{H}_\gamma|$ must be controlled to prevent over-fitting*
- *Regularization adapts hypothesis complexity to the training data*

Generalization Error

Definition 6.3.24 (Generalization Error)**.** The generalization error refers to the difference between the true risk $\mathcal{E}(f)$ and the empirical risk $\mathcal{E}_n(f)$:

$$\sup_{f\in\mathcal{H}_\gamma} |\mathcal{E}_n(f) - \mathcal{E}(f)|.$$

Remark (Empirical Process Perspective)**.** *The generalization error can be studied using tools from empirical process theory. For a finite hypothesis space $\mathcal{H}_\gamma$, the expectation of the supremum of the generalization error satisfies:*

$$\mathbb{E}\left[\sup_{f\in\mathcal{H}_\gamma} |\mathcal{E}_n(f) - \mathcal{E}(f)|\right] \leq |\mathcal{H}_\gamma|\sqrt{\frac{V_{\mathcal{H}_\gamma}}{n}},$$

where $V_{\mathcal{H}_\gamma}$ is the variance of the loss over $\mathcal{H}_\gamma$. A high variance hypothesis space leads to unstable learning.

Example 6.3.25 (Finite Hypothesis Spaces)**.** For $\mathcal{H}_\gamma$ finite, the generalization error bound ensures that as the number of training examples n increases, **the empirical risk becomes a more accurate estimate of the true risk**. This provides guarantees for algorithms like ERM.

Example 6.3.26 (Regularization in Threshold Functions)**.** Consider the hypothesis space of threshold functions:

$$\mathcal{H}_p = \{f_a \mid a \cdot 10^p \in \mathbb{Z}, a \in [-1, 1]\},$$

where the parameter p controls the precision of the thresholds. As p increases, the hypothesis space $\mathcal{H}_p$ becomes more expressive but also more prone to over-fitting. Regularization involves gradually increasing p with the number of training examples n, ensuring that the complexity of $\mathcal{H}_p$ matches the available data.

Remark (Balancing Regularization and Complexity)**.** *Regularization parameter highlights the trade-off between underfitting and overfitting:*

- ***If γ (or equivalently, the complexity of $\mathcal{H}_\gamma$) is too small**, the hypothesis space is overly restricted, leading to high approximation error (underfitting).*
- ***If γ is too large ($\mathcal{H}$ too flexible)**, the hypothesis space is too flexible, leading to high sample error (overfitting).*

By adapting γ to the number of training examples, regularization achieves a balance between these competing sources of error.

6.3.5 ERM on Convex Spaces

Remark. *Empirical risk minimization (ERM) on convex (thus dense) spaces is often much more amenable to computations. In principle, we have observed that on infinite hypothesis spaces it is difficult to control the generalization error but...*

*We might be able to leverage the **discretization argument** used for threshold functions to control the generalization error of ERM for a larger family of hypothesis spaces.*

6.3.6 Example: Risks for Continuous Functions

Let $\mathcal{X} \subset \mathbb{R}^d$ be a compact (i.e., closed and bounded) space and let $C(\mathcal{X})$ be the space of continuous functions. Define the norm:

$$\|f\|_\infty = \sup_{x \in \mathcal{X}} |f(x)|. \tag{6.1}$$

*If the loss function $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ is **uniformly Lipschitz** with constant $L > 0$, then for any $y \in \mathcal{Y}$:*

1. ***Expected risk bound:***
$$|\mathcal{E}(f_1) - \mathcal{E}(f_2)| \leq L\|f_1 - f_2\|_\infty. \tag{6.2}$$
2. ***Empirical risk bound:***
$$|\mathcal{E}_n(f_1) - \mathcal{E}_n(f_2)| \leq L\|f_1 - f_2\|_\infty. \tag{6.3}$$

Remark. *Therefore, "close" functions in $\|\cdot\|_\infty$ will have similar expected and empirical risks!*

6.3.7 Example: Covering Numbers

Definition 6.3.27 (Covering Number)**.** The *covering number* $\mathcal{N}(\mathcal{H}, \eta)$ of a hypothesis space $\mathcal{H}$ with radius $\eta > 0$ is the minimal number of **balls of radius** η needed to cover $\mathcal{H}$:

$$\mathcal{N}(\mathcal{H}, \eta) = \inf \left\{ m \mid \mathcal{H} \subset \bigcup_{i=1}^{m} B_\eta(h_i), \quad h_i \in \mathcal{H} \right\}. \tag{6.4}$$

Example 6.3.28. If $\mathcal{H} \cong B_R(0)$ is a ball of radius R in $\mathbb{R}^d$, then:

$$\mathcal{N}(B_R(0), \eta) = \left(\frac{4R}{\eta}\right)^d. \tag{6.5}$$

6.3.8 Example: Covering Numbers (Continued)

Remark. *The resolution η is related to precision in the sense of the distance between two hypotheses. Using a similar argument as with threshold functions:*

$$\mathbb{E}\left[\sup_{f \in \mathcal{H}} |\mathcal{E}(f_n) - \mathcal{E}(f)|\right] \leq 2L\eta + \mathcal{N}(\mathcal{H}, \eta)\sqrt{\frac{V_\mathcal{H}}{n}}. \tag{6.6}$$

As $\eta \to 0$, the covering number $\mathcal{N}(\mathcal{H}, \eta) \to +\infty$. However, for $n \to +\infty$, the bound tends to zero.

***Exercise**: Find a suitable $\eta(n)$ for which the bound tends to zero as $n \to +\infty$ when $\mathcal{H}$ is a ball of radius R.*

6.3.9 Complexity Measures

Remark. *In general, the error:*

$$\sup_{f \in \mathcal{H}_\gamma} |\mathcal{E}_n(f) - \mathcal{E}(f)| \tag{6.7}$$

can be controlled using capacity/complexity measures such as:

- *Covering numbers,*
- *Combinatorial dimensions (e.g., VC-dimension, fat-shattering dimension),*
- *Rademacher complexities,*
- *Gaussian complexities.*

6.3.10 Prototypical Results

Theorem 6.3.29. *A prototypical result under suitable assumptions (e.g., regularity of f_*):*

$$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f^*) \leq \underbrace{\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_\gamma)}_{\text{Variance} \sim \gamma^\beta n^{-\alpha}} + \underbrace{\mathcal{E}(f_\gamma) - \mathcal{E}(f^*)}_{\text{Bias} \sim \gamma^{-\tau}}. \tag{6.8}$$

The optimal choice of $\gamma(n)$ balances variance and bias.

6.3.11 Choosing $\gamma(n)$ in Practice

Remark. *Choosing the best $\gamma(n)$ depends on the unknown distribution ρ. This is a problem known as model selection. Possible approaches include:*

- *Cross-validation,*
- *Complexity regularization / structural risk minimization,*
- *Balancing principles.*

6.3.12 Abstract Regularization

Remark. *Regularization controls the expressiveness of the hypothesis space based on training examples, ensuring good prediction performance and consistency.*

Some common methods include:

- *Tikhonov (and Ivanov) regularization,*

- *Spectral filtering,*
- *Early stopping,*
- *Random sampling.*

Chapter 7

Statistical Learning Theory (Part II)

7.1 Excess Risk and Generalization Error: Recap

Theorem 7.1.1 (Excess Risk Decomposition)**.** *The excess risk for a hypothesis $f_{n,\gamma}$ in a regularized learning framework can be decomposed as:*

$$E(f_{n,\gamma}) - E(f^*) = \underbrace{E(f_{n,\gamma}) - E(f_\gamma)}_{\text{Sample Error(Variance)}} + \underbrace{E(f_\gamma) - \inf_{f\in\mathcal{H}} E(f)}_{\text{Approximation Error(Bias)}} + \underbrace{\inf_{f\in\mathcal{H}} E(f) - E(f^*)}_{\text{Irreducible Error}}.$$

The sample error depends on the expressiveness of $\mathcal{H}_\gamma$, controlled by γ, and the generalization error.

Remark (Generalization Error)**.** *The generalization error quantifies how well the empirical risk approximates the expected risk:*

$$\textit{Generalization Error: } E(f_{n,\gamma}) - E_n(f_{n,\gamma}).$$

This error is a key quantity to control when studying the sample error of a learning algorithm. It can be expressed as part of the decomposition of the sample error:

$$E(f_{n,\gamma}) - E(f_\gamma) = \underbrace{E(f_{n,\gamma}) - E_n(f_{n,\gamma})}_{\text{Generalization Error}} + \underbrace{E_n(f_{n,\gamma}) - E_n(f_\gamma)}_{\leq 0} + \underbrace{E_n(f_\gamma) - E(f_\gamma)}_{=0 \text{ in expectation, } O(1/\sqrt{n}) \text{ in probability}}.$$

For finite hypothesis spaces, the generalization error can be bounded as:

$$E(f_{n,\gamma}) - E_n(f_{n,\gamma}) \leq |\mathcal{H}_\gamma| \frac{\sqrt{V_\gamma}}{n},$$

where:

- *$|\mathcal{H}_\gamma|$ is the cardinality of the hypothesis space.*

- $V_\gamma = \sup_{f \in \mathcal{H}_\gamma} Var[\ell(f(x), y)]$ *measures the variability of the loss function.*
- *The bound implies that controlling* $|\mathcal{H}_\gamma|$ *and* V_γ *is essential for reducing the generalization error.*

Advantages and Challenges:

- ***Pros:*** *By plugging this bound into the excess risk decomposition, we can analyze the prediction performance of the learning algorithm* $f_{n,\gamma}$.
- ***Cons:*** *The cardinality* $|\mathcal{H}_\gamma|$ *is often prohibitively large. Even though statistical techniques like Hoeffding's inequality can mitigate this (e.g., by appearing as a logarithmic factor), solving empirical risk minimization (ERM) over* $|\mathcal{H}_\gamma|$ *becomes computationally expensive.*

7.2 Rademacher Complexity

Definition 7.2.1 (Empirical Rademacher Complexity)**.** To derive tighter bounds for the generalization error, we need a measure that directly captures the "worst-case" deviation between expected and empirical risk:

$$\mathbb{E}\left[\sup_{f \in \mathcal{H}}(E(f) - E_n(f))\right].$$

Let $S = (z_i)_{i=1}^n$ be a dataset. The empirical Rademacher complexity of a hypothesis space $\mathcal{H}$ is defined as:

$$\mathcal{R}_S(\mathcal{H}) = \mathbb{E}_\sigma\left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \sigma_i f(z_i)\right],$$

where

- $S = (z_i)_{i=1}^n$: The dataset sampled from the input space.
- $\sigma_i \in \{-1, 1\}$: Independent Rademacher random variables, each equally likely to be 1 or -1.
- $\sup_{f \in \mathcal{H}}$: The supremum computes the largest average correlation between the functions f in $\mathcal{H}$ and the noise σ_i.

Intuitively, the Rademacher complexity quantifies how well the hypothesis space $\mathcal{H}$ can "align" with random labels, providing insight into the expressiveness of $\mathcal{H}$ and its tendency to overfit.

Definition 7.2.2 (Rademacher Complexity)**.** Given a probability measure p over $\mathcal{Z}$, the Rademacher complexity is the expectation of the empirical Rademacher complexity over all datasets S drawn from p:

$$\mathcal{R}_n(\mathcal{H}) = \mathbb{E}_S\left[\mathcal{R}_S(\mathcal{H})\right].$$

7.3 Connecting Generalization Error to Rademacher Complexity

Definition 7.3.1 (Empirical Risk Property)**.** For clarity, let $S \sim \rho^n$ be a dataset sampled from a distribution ρ, and denote the empirical risk of a function f with respect to S as:

$$\mathcal{E}_S(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i).$$

The expected risk corresponds to:

$$\mathbb{E}_S \mathcal{E}_S(f) = \mathcal{E}(f).$$

7.3.1 Bounding the Supremum of the Generalization Error

Remark (Virtual Dataset)**.** *By introducing a "virtual" dataset $S' \sim \rho^n$, we can rewrite the supremum of the generalization error as:*

$$\mathbb{E}_S \left[\sup_{f\in\mathcal{H}} \mathcal{E}(f) - \mathcal{E}_S(f) \right] = \mathbb{E}_S \left[\sup_{f\in\mathcal{H}} \mathbb{E}_{S'} \left[\mathcal{E}_S(f) - \mathcal{E}_{S'}(f) \right] \right].$$

Using the convexity of the supremum, this can be further bounded as:

$$\mathbb{E}_S \left[\sup_{f\in\mathcal{H}} \mathbb{E}_{S'} \left[\mathcal{E}_S(f) - \mathcal{E}_{S'}(f) \right] \right] \leq \mathbb{E}_{S,S'} \left[\sup_{f\in\mathcal{H}} \mathcal{E}_S(f) - \mathcal{E}_{S'}(f) \right].$$

The final expression have two empirical risks, which we can further calculate.

7.3.2 Rademacher Variables

Remark (Symmetrization Using Rademacher Variables)**.** *For datasets $S = \{(x_i, y_i)\}_{i=1}^n$ and $S' = \{(x_i', y_i')\}_{i=1}^n$, we introduce Rademacher variables $\sigma_i \in \{-1, 1\}$ uniformly sampled. The generalization error can then be expressed as:*

$$\mathbb{E}_{S,S'} \left[\sup_{f\in\mathcal{H}} \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i'), y_i') - \ell(f(x_i), y_i) \right] = \mathbb{E}_{S,S',\sigma} \left[\sup_{f\in\mathcal{H}} \frac{1}{n}\sum_{i=1}^{n} \sigma_i \left(\ell(f(x_i'), y_i') - \ell(f(x_i), y_i) \right) \right].$$

Sampling $\sigma_i = -1$ can be interpreted as "swapping" the sample (x_i, y_i) from S with the (x_i', y_i') in S'. However, the expectation is considering all possible combinations of S and S'. This process only changes the order of elements in the expectation but not the result.

7.3.3 Connecting back to Rademacher Complexity

Remark (Subadditivity)**.** *Using the subadditivity of the supremum,* $\sup_x f(x) + g(x) \leq \sup_x f(x) + \sup_x g(x)$*, we can bound:*

$$\mathbb{E}_{S,S',\sigma}\left[\sup_{f\in\mathcal{H}} \frac{1}{n}\sum_{i=1}^{n} \sigma_i \ell(f(x_i'), y_i') - \ell(f(x_i), y_i)\right] \leq 2\mathbb{E}_{S,\sigma}\left[\sup_{f\in\mathcal{H}} \frac{1}{n}\sum_{i=1}^{n} \sigma_i \ell(f(x_i), y_i)\right].$$

Definition 7.3.2 (Composed Hypothesis Class)**.** Define the composed hypothesis class as:

$$\mathcal{G} = \ell \circ \mathcal{H} = \{g(x, y) = \ell(f(x), y) : f \in \mathcal{H}\}.$$

The Rademacher complexity of $\mathcal{G}$ is:

$$\mathcal{R}_n(\mathcal{G}) = \mathbb{E}_{S,\sigma}\left[\sup_{g\in\mathcal{G}} \frac{1}{n}\sum_{i=1}^{n} \sigma_i g(z_i)\right],$$

where $z_i = (x_i, y_i)$.

Theorem 7.3.3 (Generalization Error Bound)**.** *Bringing everything together, the worst-case generalization error can be bounded as:*

$$\mathbb{E}\left[\sup_{f\in\mathcal{H}} \mathcal{E}(f) - \mathcal{E}_n(f)\right] \leq 2\mathcal{R}_n(\ell \circ \mathcal{H}).$$

7.4 Contraction Lemma

7.4.1 Contraction Lemma

Remark (Generalization Bound with Rademacher Complexity)**.** *The generalization error has been bounded in terms of the Rademacher complexity of the composed hypothesis class* $\mathcal{R}_n(\ell \circ \mathcal{H})$*. In practice, it is desirable to express this in terms of* $\mathcal{R}_n(\mathcal{H})$*, the Rademacher complexity of the original hypothesis space. This is possible by making assumptions on the loss function* ℓ*.*

Theorem 7.4.1 (Contraction Lemma)**.** *Let* ℓ *be an* L*-Lipschitz loss function. The Rademacher complexity of the composed hypothesis class* $\ell \circ \mathcal{H}$ *can be bounded as:*

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq L\mathcal{R}_S(\mathcal{H}),$$

where L *is the Lipschitz constant of* ℓ*.*

Sketch of Proof.

- Start by isolating the contribution of the term $\sigma_1 \ell(f(x_1), y_1)$ in the Rademacher complexity:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) = \mathbb{E}_{S,\sigma}\left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i \ell(f(x_i), y_i)\right].$$

- Use the definition of expectation to explicitly consider σ_1 and split the supremum:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) = \frac{1}{2}\mathbb{E}_{S,\sigma}\left[\sup_{f \in \mathcal{H}} \left(\ell(f(x_1), y_1) + \sum_{i=2}^{n} \sigma_i \ell(f(x_i), y_i)\right)\right].$$

- Apply the subadditivity of the supremum to bound:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq \frac{1}{2} \sup_{f, f' \in \mathcal{H}} L|f(x_1) - f'(x_1)| + \text{remaining terms}.$$

- Using the Lipschitz property of the loss, the term simplifies to:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq L\mathcal{R}_S(\mathcal{H}).$$

  where the left is expected Rademacher complexity of the composed hypothesis space, and the right is the Rademacher complexity of the original hypothesis space.

□

Finally, by assuming l to be L-lipschitz, we can control the worst generalization error as:

Theorem 7.4.2 (Generalization Bound with Lipschitz Loss)**.** *Assuming the loss ℓ is L-Lipschitz, the worst generalization error can be controlled as:*

$$\mathbb{E}\left[\mathcal{E}(f_n) - \mathcal{E}_n(f_n)\right] \leq 2L\mathcal{R}_n(\mathcal{H}).$$

Remark (Intuition)**.** *The contraction lemma bridges the gap between the complexity of the composed hypothesis space $\ell \circ \mathcal{H}$ and the original hypothesis space $\mathcal{H}$, making it feasible to bound generalization error using simpler Rademacher complexity results for $\mathcal{H}$.*

7.5 Generalization Error Bounds - Probability Perspective

7.5.1 McDiarmid's Inequality

Theorem 7.5.1 (McDiarmid's Inequality)**.** *Let $\mathcal{Z}$ be a set, and let $g : \mathcal{Z}^n \to \mathbb{R}$ be a function such that there exists $c > 0$ satisfying:*

$$|g(z_1, \ldots, z_i, \ldots, z_n) - g(z_1, \ldots, z_i', \ldots, z_n)| \leq c,$$

for any $i = 1, \ldots, n$ and $z_1, \ldots, z_i, z_i', \ldots, z_n \in \mathcal{Z}$. Let $Z_1, \ldots, Z_n$ be independent random variables taking values in $\mathcal{Z}$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, we have:

$$|g(Z_1, \ldots, Z_n) - \mathbb{E}[g(Z_1, \ldots, Z_n)]| \leq c\sqrt{\frac{n}{2} \log \frac{2}{\delta}}.$$

7.5.2 Error Bound with Rademacher Complexity

Remark (Applying McDiarmid's Inequality)**.** *Let $z_i = (x_i, y_i)$ and define:*

$$g(z_1, \ldots, z_n) = \sup_{f \in \mathcal{H}} \left(\mathcal{E}(f) - \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i) \right).$$

here is the worst possible generalization error. Assume $|\ell(y', y)| \leq c$. Recall that for any two functions $\alpha, \beta : \mathcal{X} \to \mathbb{R}$, we have:

$$\sup_x \alpha(x) - \sup_x \beta(x) \leq \sup_x |\alpha(x) - \beta(x)|.$$

Therefore, for the function $g(z_1, \ldots, z_n)$, the following holds:

$$|g(z_1, \ldots, z_n) - g(z_1, \ldots, z_{i-1}, z_i', z_{i+1}, \ldots, z_n)| \leq \frac{1}{n} |\ell(f(x_i), y_i) - \ell(f(x_i'), y_i')| \leq \frac{2c}{n}.$$

By McDiarmid's inequality, this implies:

$$|g(z_1, \ldots, z_n) - \mathbb{E}[g(z_1, \ldots, z_n)]| \leq c\sqrt{\frac{2 \log(2/\delta)}{n}}.$$

7.5.3 Generalization Error Bound

Theorem 7.5.2 (Generalization Bound with Rademacher Complexity)**.** *For any $\delta > 0$, with probability at least $1 - \delta$, the generalization error satisfies:*

$$\sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f)) \leq \mathbb{E}\left[\sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f)) \right] + c\sqrt{\frac{2 \log(2/\delta)}{n}}.$$

Using the bound in terms of Rademacher complexity, we also have:

$$\sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f)) \leq 2L\mathcal{R}_n(\mathcal{H}) + c\sqrt{\frac{2 \log(2/\delta)}{n}}.$$

7.5.4 Recap

Remark (Summary of Results)**.** *We have shown that the generalization error of any algorithm learning a function $f \in \mathcal{H}$ can be controlled in terms of the Rademacher complexity of the hypothesis space $\mathcal{H}$:*

$$\sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f)) \leq 2L\mathcal{R}_n(\mathcal{H}) + c\sqrt{\frac{2\log(2/\delta)}{n}}.$$

This result holds for any learning algorithm, not just Empirical Risk Minimization (ERM). It emphasizes the importance of ensuring that the Rademacher complexity of $\mathcal{H}$ is finite and not too large to achieve meaningful bounds on the generalization error.

7.6 Caveats and Examples in Using Rademacher Complexity

7.6.1 Caveats in Using Rademacher Complexity

Remark (Finite Rademacher Complexity)**.** *With Rademacher complexity, we now have a tool to study the theoretical properties of estimators, such as the Empirical Risk Minimization (ERM) estimator:*

$$f_S = \arg\min_{f \in \mathcal{H}} \mathcal{E}_S(f).$$

*However, to ensure meaningful generalization bounds, it is necessary for $\mathcal{R}(\mathcal{H})$ to be **finite**. This raises two important questions:*

- *For which hypothesis spaces can we effectively control $\mathcal{R}(\mathcal{H})$?*
- *How do we solve the resulting constrained optimization problem?*

7.6.2 Example: Linear Hypotheses

Definition 7.6.1 (Linear Hypothesis Space)**.** Consider $\mathcal{X} = \mathbb{R}^d$ and the hypothesis space of linear functions:

$$\mathcal{H} = \{f(x) = \langle x, w\rangle \mid \forall x \in \mathcal{X}, \exists w \in \mathbb{R}^d\}.$$

The Rademacher complexity of this space can be written as:

$$\mathcal{R}_n(\mathcal{H}) = \frac{1}{n}\mathbb{E}\left[\sup_{w \in \mathbb{R}^d} \sum_{i=1}^{n} \sigma_i \langle x_i, w\rangle\right].$$

By applying the Cauchy-Schwarz inequality, $\langle x, w\rangle \leq \|x\|\|w\|$, this simplifies to:

$$\mathcal{R}_n(\mathcal{H}) \leq \frac{1}{n}\sup_{\|w\|} \|w\| \sum_{i=1}^{n} \|\sigma_i x_i\|,$$

which **diverges** unless additional constraints are imposed.

7.6.3 Example: Bounded Linear Hypotheses

Definition 7.6.2 (Balls in Linear Hypothesis Space)**.** Restricting $\mathcal{H}$ to a ball of radius γ:

$$\mathcal{H}_\gamma = \{f(x) = \langle x, w\rangle \mid \forall x \in \mathcal{X}, \exists w \in \mathbb{R}^d, \|w\| \leq \gamma\}.$$

The Rademacher complexity becomes:

$$\mathcal{R}_n(\mathcal{H}_\gamma) \leq \frac{\gamma}{n}\mathbb{E}\left[\left\|\sum_{i=1}^n \sigma_i x_i\right\|\right].$$

Using Jensen's inequality and assuming bounded input norms ($\|x_i\| \leq B$), we obtain:

$$\mathcal{R}_n(\mathcal{H}_\gamma) \leq \frac{\gamma}{\sqrt{n}}B.$$

which gives us a bound on the generalization error.

7.6.4 Example: Reproducing Kernel Hilbert Spaces (RKHS)

Definition 7.6.3 (RKHS Rademacher Complexity)**.** Let $\mathcal{X} \times \mathcal{X} \to \mathbb{R}$ be a kernel $k(x, x') \leq \kappa^2$, and let $\mathcal{H}_\gamma$ be the RKHS associated with k such that $\|f\|_{\mathcal{H}} \leq \gamma$. The Rademacher complexity of $\mathcal{H}_\gamma$ satisfies:

$$\mathcal{R}_n(\mathcal{H}_\gamma) \leq \frac{\gamma\kappa}{\sqrt{n}},$$

which generalizes the result for linear hypotheses.

7.6.5 Key Observations

Remark (Generalization Implications)**.** *The Rademacher complexity of bounded hypothesis spaces provides meaningful generalization bounds:*

- *The generalization error decreases as n (sample size) increases.*
- *Larger γ (radius of the hypothesis space) increases Rademacher complexity, leading to less meaningful bounds.*

7.7 Constrained Optimization

Remark (Tikhonov Regularization)**.** *Tikhonov regularization minimizes a combination of the empirical risk and a penalty on the norm of the hypothesis:*

$$w_{S,\lambda} = \arg\min_w \mathcal{E}_S(w) + \lambda\|w\|^2.$$

The choice of $\lambda > 0$ controls the trade-off between bias and variance. Here, $w_{S,\lambda}$ is the minimizer of the regularized problem.

7.7.1 Bounding the Norm of $w_{S,\lambda}$

Theorem 7.7.1. *Since $w_{S,\lambda}$ is the solution to the Tikhonov regularization problem, we have:*

$$\mathcal{E}_S(w_{S,\lambda}) + \lambda\|w_{S,\lambda}\|^2 \leq \mathcal{E}_S(0) + \lambda\|0\|^2 = \mathcal{E}_S(0).$$

Assuming $|\ell(y', y)| \leq M^2$ for a constant $M > 0$, this implies:

$$\|w_{S,\lambda}\| \leq \sqrt{\frac{\mathcal{E}_S(0)}{\lambda}} \leq \frac{M}{\sqrt{\lambda}}.$$

Thus, we can restrict $w_{S,\lambda}$ to the constrained hypothesis space $\mathcal{H}_\gamma$ with $\gamma = \frac{M}{\sqrt{\lambda}}$.

7.7.2 Decomposition of Excess Risk

Remark (Excess Risk Decomposition)**.** *Assuming $w_* \in \mathcal{H}$, the excess risk can be decomposed as:*

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}(w_*)] = \underbrace{\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}_S(w_{S,\lambda})]}_{\text{Rademacher Complexity}} + \underbrace{\mathbb{E}[\mathcal{E}_S(w_{S,\lambda}) - \mathcal{E}_S(w_*)]}_{\leq \lambda\|w_*\|^2} + \underbrace{\mathbb{E}[\mathcal{E}_S(w_*) - \mathcal{E}(w_*)]}_{=0}.$$

7.7.3 Bounding the Remaining Terms

Theorem 7.7.2. *The term $\mathcal{E}_S(w_{S,\lambda}) - \mathcal{E}_S(w_*)$ can be bounded by adding and removing $\lambda\|w_{S,\lambda}\|^2$ and $\lambda\|w_*\|^2$:*

$$\mathcal{E}_S(w_{S,\lambda}) - \mathcal{E}_S(w_*) \leq (\mathcal{E}_S(w_{S,\lambda}) + \lambda\|w_{S,\lambda}\|^2) - (\mathcal{E}_S(w_*) + \lambda\|w_*\|^2) + \lambda\|w_*\|^2.$$

Using the definition of $w_{S,\lambda}$, this reduces to:

$$\mathcal{E}_S(w_{S,\lambda}) - \mathcal{E}_S(w_*) \leq \lambda\|w_*\|^2.$$

7.7.4 Overall Bound and Rate

Theorem 7.7.3 (Final Excess Risk Bound)**.** *Combining the results, we conclude:*

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}(w_*)] \leq \frac{2LMB}{\sqrt{n\lambda}} + \lambda\|w_*\|^2.$$

Choosing $\lambda(n) = \frac{(LMB)^{2/3}}{\|w_*\|^{4/3} n^{1/3}}$, the optimal rate becomes:

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda(n)}) - \mathcal{E}(w_*)] \leq \frac{3(LMB)^{2/3}\|w_*\|^{2/3}}{n^{1/3}}.$$

7.7.5 Ivanov Regularization

Remark (Ivanov Regularization)**.** *Ivanov regularization constrains the hypothesis space explicitly:*

$$w_{S,\gamma} = \arg \min_{\|w\| \leq \gamma} E_n(w).$$

This yields tighter generalization bounds with an excess risk of:

$$\mathbb{E}[E(w_{S,\gamma}) - E(w^*)] \leq O\left(\frac{1}{\sqrt{n}}\right).$$

7.7.6 Projected Gradient Descent (PGD)

Definition 7.7.4 (Projected Gradient Descent)**.** For a smooth convex function F and a convex constraint set C, PGD iterates as:

$$w_{k+1} = \Pi_C(w_k - \eta \nabla F(w_k)),$$

where $\Pi_C(w)$ is the projection of w onto C.

Theorem 7.7.5 (Convergence of PGD)**.** *Let F be M-smooth and convex, and let w^* be the minimum in C. Then, after K iterations, PGD achieves:*

$$F(w_K) - F(w^*) \leq \frac{M}{2K}\|w_0 - w^*\|^2.$$

Summary

- *Unsatisfied by only controlling the generalization error of a learning algorithm for finite spaces of hypotheses, we focused on more refined bounding techniques.*
- *We observed that examining the worst-case generalization error within a class of functions (rather than summing all errors, which can be excessively large) can be controlled using the Rademacher complexity of the hypothesis space.*
- *For spaces of linear hypotheses, or more generally, for balls in a Reproducing Kernel Hilbert Space (RKHS), the Rademacher complexity can be bounded by a finite quantity that depends on:*
  - *The **number of training points**.*
  - *The **radius of the ball**.*
- *We developed an efficient algorithm to solve the corresponding constrained Empirical Risk Minimization (ERM) problem.*

Chapter 8

Online Learning (Part I)

8.1 Batch vs. Online Learning

Definition 8.1.1 (Batch vs Online Learning)**. Batch Learning:**

- Assumes IID training data $\{(x_i, y_i)\}_{i=1}^n$.
- Objective: Minimize *generalization error* on unseen data.
- To build a classifier from the training data that predicts well on new data from same distribution.

  Online Learning:

  - Operates on a online sequential data stream $\{(x_t, y_t)\}_{t=1}^m$.
  - No distributional assumptions on the data.
  - Evaluation metric: Cumulative error $\sum_{t=1}^m |y_t - \hat{y}_t|$.
  - To sequentially predict and updates a classifier to predict well on the sequence. (i.e. there is no training/testing set distinction).
  - Advantages include **non-asymptotic**, **no statistical assumptions**, and **there exist techniques to convert cumulative error guarantees to generalization error guarantees**.

8.2 Online Learning with Expert Advice

Definition 8.2.1 (Expert Predictions)**.** Let $S = \{(x_t, y_t)\}_{t=1}^m$ represent an online sequence of data, t is the time, where:

- $x_t \in \{0, 1\}^n$ is the set of predictions from n experts at time t about an outcome y_t.

- $y_t \in \{0, 1\}$ is the true outcome at time t.
- $x_{t,i}$ denotes the prediction of expert i at time t, with $i \in \{1, \ldots, n\}$.

Protocol of the Master Algorithm

For $t = 1, \ldots, m$:

Get instance	$\mathbf{x}_t \in \{0, 1\}^n$
Predict	$\hat{y}_t \in \{0, 1\}$
Get label	$y_t \in \{0, 1\}$
Incur loss (mistakes)	$\lvert y_t - \hat{y}_t \rvert$

Figure 8.1: Protocol of the Master Algorithm A

The goal is to design a *Master Algorithm* that:

- Combines the predictions of the experts, x_t, to output a single prediction $\hat{y}_t$ at each time t.
- Minimizes the *cumulative loss*, defined as:

$$L_A(S) = \sum_{t=1}^{m} |\hat{y}_t - y_t|,$$

where $\hat{y}_t = A(S_{t-1})(x_t)$ is the prediction made by the Master Algorithm A at time t. A is trained (online) on S_{t-1} and evaluated on x_t.

	experts						
	E_1	E_2	E_3	E_n	prediction	true label	loss
day 1	1	1	0	0	0	1	1
day 2	1	0	1	0	1	0	1
day 3	0	1	1	1	1	1	0
day t	$x_{t,1}$	$x_{t,2}$	$x_{t,3}$	$x_{t,n}$	$\hat{y}_t$	y_t	$\lvert y_t - \hat{y}_t \rvert$

Figure 8.2: Human Experts or the predictions of n separate algorithms.

Remark (Interpretation of Experts)**.** *An expert can be:*

- *A human forecaster providing predictions.*
- *A separate algorithm predicting the outcome based on its internal model.*

The Master Algorithm combines these individual predictions into a single prediction $\hat{y}_t$, leveraging past performance to optimize accuracy.

8.3 Halving Algorithm

8.3.1 Definition and Figure

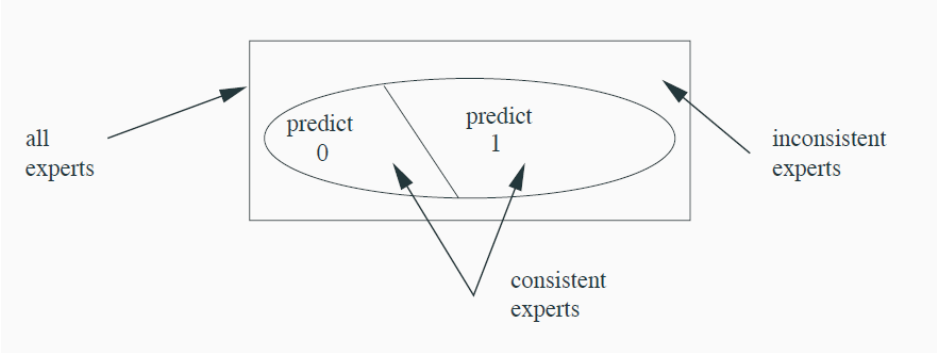

Figure 8.3: Halving Algorithm.

Definition 8.3.1 (Halving Algorithm)**.** If there exists at least one *perfect expert* (i.e., an expert that makes no mistakes), the **Halving Algorithm**:

- Keeps track of only the *consistent experts*, i.e., those experts that have not made any mistakes so far.
- Predicts by taking the *majority vote* among the predictions of the consistent experts.
- Eliminates inconsistent experts after each round.

Mistake Bound: For n experts, the Halving Algorithm makes at most $\log_2(n)$ mistakes.

8.3.2 Mistake Bound: Intuition

Remark (Mistake Bound: Intuition)**.**

- *At each round, the algorithm eliminates at least half of the experts as inconsistent.*
- *Starting with n experts, this process of elimination can happen at most $\log_2(n)$ times, since each elimination cuts down the pool of remaining experts by at least half.*

8.3.3 Example

Example 8.3.2 (Example of the Halving Algorithm)**.** Consider $n = 8$ experts. The following table illustrates a step-by-step run of the Halving Algorithm:

E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	$\hat{y}$	y	loss
1	1	0	0	1	1	0	0	1	0	1
x	x	0	1	x	x	1	1	1	1	0
x	x	x	1	x	x	0	0	0	1	1
x	x	x	↑	x	x	x	x			
			consistent							

Figure 8.4: Example of Halving Algorithm: find the consistent experts.

Here, x denotes an eliminated expert. At each step:

- **Day 1:** Majority vote among all experts predicts $\hat{y} = 1$, incurring a loss of 1.
- **Day 2:** Only consistent experts (those who predicted correctly on Day 1) remain, and the algorithm predicts $\hat{y} = 1$, incurring no loss.
- **Day 3:** A further majority vote is taken among the consistent experts, predicting $\hat{y} = 0$, and a loss of 1 is incurred.

8.3.4 Extension to Imperfect Experts

Theorem 8.3.3. *If no perfect expert exists, the algorithm's performance is measured against the best expert's cumulative loss:*

$$L_i(S) = \sum_{t=1}^{m} |y_t - x_{t,i}|,$$

where $L_i(S)$ is the total loss of expert i on sequence S. The following bound holds:

$$L_A(S) \leq a \cdot \min_i L_i(S) + b \cdot \log(n),$$

where a, b are "small" constants. These are known as Regret or Worst-case loss bounds.

8.4 Weighted Majority Algorithm (WMA)

8.4.1 Algorithm Description

Definition 8.4.1 (Weighted Majority Algorithm)**.** The Weighted Majority Algorithm (WMA) does not eliminate experts but instead assigns a weight $w_{t,i}$ to each expert i at trial t and updates it as follows:

$$w_{t+1,i} = w_{t,i} \cdot \beta^{\text{mistake}}, \quad \text{where } \beta \in [0, 1).$$

Key properties of the algorithm:

- **Reliability Tracking:** The weight $w_{t,i}$ measures the reliability of expert i, with higher weights given to more accurate experts.
- **Weighted Majority Vote:** The algorithm predicts the outcome $\hat{y}_t$ based on the larger weighted vote from all experts. Specifically:

$$\hat{y}_t = \begin{cases} 0, & \text{if } \sum_{i=1}^{n} w_{t,i} \cdot x_{t,i} < \frac{W_t}{2}, \\ 1, & \text{otherwise}, \end{cases}$$

  where $W_t = \sum_{i=1}^{n} w_{t,i}$ is the total weight of all experts at trial t.
- **Penalty for Mistakes:** Incorrect experts have their weights reduced by a factor of β, thus reducing their influence on future predictions.
- **No Expert Elimination:** Unlike the Halving Algorithm, the WMA adjusts weights adaptively without discarding any expert.

8.4.2 Number of Mistakes: Analysis

$$\begin{aligned} M &= \#\textit{ mistakes of the master algorithm at the end (over all trials)}, \\ M_{t,i} &= \#\textit{ mistakes of expert } E_i \textit{ at the start of trial } t, \\ M_i &= M_{m+1,i} \quad (\#\textit{ of total mistakes of expert } E_i), \\ w_{t,i} &= \beta^{M_{t,i}} \quad (\textit{weight of } E_i \textit{ at beginning of trial } t), \\ W_t &= \sum_{i=1}^{n} w_{t,i}, \quad (\textit{total sum of weights at the start of trial } t). \end{aligned}$$

Weight Updates per Trial:

- ***If the Master is right**, the weights of the 'minority" experts are multiplied by β. Hence:*

$$W_t = \textit{(Minority)} + \textit{(Majority)} \ \geq \ \beta \cdot \textit{(Minority)} + \textit{(Majority)} \ = \ W_{t+1}.$$

- ***If the Master makes a mistake**, then the majority gets multiplied by β. Thus:*

$$W_{t+1} \;\leq\; \frac{1}{2}W_t \;+\; \beta\,\frac{1}{2}W_t \quad \text{(why? major/minor split)}.$$

From this, it follows that

$$W_{t+1} \;\leq\; \frac{1+\beta}{2}\,W_t \quad\Longrightarrow\quad W_{m+1} \;\leq\; \left(\tfrac{1+\beta}{2}\right)^{M} W_1.$$

Since $W_{m+1} = \sum_{j=1}^{n} w_{m+1,j} = \sum_{j=1}^{n} \beta^{M_j} \geq \beta^{M_i}$, we obtain

$$\left(\tfrac{1+\beta}{2}\right)^{M} n \;\geq\; \beta^{M_i}.$$

Solving for M. *Taking logarithms:*

$$M \;\leq\; \frac{\ln\frac{1}{\beta}}{\ln\frac{2}{1+\beta}}\,M_i \;+\; \frac{1}{\ln\frac{2}{1+\beta}}\,\ln n.$$

For instance, choosing $\beta = 1/e$ gives

$$M \;\leq\; 2.63\,\min_i M_i \;+\; 2.63\,\ln n.$$

*Thus, for **all** sequences, the master algorithm's loss is **comparable** to that of the **best expert**.*

8.4.3 Refining and Generalising the Experts Model – 1

More generally, we want to obtain regret bounds for arbitrary loss functions

$$L : \mathcal{Y} \times \hat{\mathcal{Y}} \;\to\; [0, +\infty].$$

Our goal is to ensure

$$L_A(S) - \min_{i\in[n]} L_i(S) \;\leq\; o(m),$$

*where $o(m)$ is a **sublinear** function in m, possibly dependent on other parameters, and captures how much we regret not following the optimal predictor in hindsight.*

- *$L_A(S)$ is the cumulative sum of errors of algorithm A.*
- *$\frac{1}{m}L_A(S)$ is the average error.*

If

$$\frac{L_A(S) - \min_{i\in[n]} L_i(S)}{m} \;\leq\; \frac{o(m)}{m},$$

and $\frac{o(m)}{m} \to 0$ *as* $m \to \infty$*, then asymptotically the algorithm's average loss approaches that of the best expert.*

In the following, we show two examples of regret bounds generalising the Weighted Majority Algorithm analysis:

1. ***Entropic (Log) Loss****:* $L : \{0,1\} \times [0,1] \to [0,+\infty]$*, with*

$$L(y,\hat{y}) = y \ln \frac{y}{\hat{y}} + (1-y) \ln \frac{1-y}{1-\hat{y}}.$$

2. ***Arbitrary Bounded Loss****: For* $L : \mathcal{Y} \times \hat{\mathcal{Y}} \to [0,B]$*.*

For the first case, the regret bound will be a small constant $\log(n)$*. For the second, it is on the order of* $O(\sqrt{m \log n})$*.*

8.4.4 A Regret Bound for the Entropic (Log) Loss

Unlike in the basic Weighted Majority setting:

- *We now allow predictions in* $[0,1]$ *rather than restricting them to* $\{0,1\}$*.*
- *We predict with a* ***weighted average*** *rather than a strict 'majority" vote.*

Specifically, at trial t*, each expert* i *has weight*

$$w_{t,i} \;=\; \beta^{L_{t,i}} \;=\; e^{-\eta\, L_{t,i}},$$

where $L_{t,i}$ *is the cumulative loss of expert* E_i *at the* ***start*** *of trial* t*, and* $\eta > 0$ *is a learning rate.*

The ***Master*** *then predicts via the* ***weighted average****:*

$$\hat{y}_t \;=\; \sum_{i=1}^{n} \frac{w_{t,i}}{\sum_{j=1}^{n} w_{t,j}}\, x_{t,i} \;=\; \mathbf{v}_t \cdot \mathbf{x}_t,$$

where $x_{t,i}$ *is the prediction of expert* E_i *in trial* t*, and* $\mathbf{v}_t = \left(\frac{w_{t,1}}{W_t}, \ldots, \frac{w_{t,n}}{W_t}\right)$ *with* $W_t = \sum_{j=1}^{n} w_{t,j}$*.*

We set initial weights $\mathbf{w}_1 = (1,\ldots,1)$*, so that* $\mathbf{v}_1 = (\frac{1}{n}, \ldots, \frac{1}{n})$*.*

This setup leads to specific regret bounds for the entropic (log) loss, typically of order $\log(n)$*, reflecting that the master algorithm's performance remains close to the best single expert in hindsight.*

8.5 Weighted Average Algorithm (WAA)

8.5.1 Definition and Key Properties

Definition 8.5.1 (Weighted Average Algorithm)**.** The **Weighted Average Algorithm (WAA)** generalizes the Weighted Majority Algorithm (WMA) by allowing predictions in the **continuous range** $[0, 1]$ rather than being restricted to binary predictions $\{0, 1\}$.

At trial t, the prediction $\hat{y}_t$ is computed as:

$$\hat{y}_t = \sum_{i=1}^{n} v_{t,i} x_{t,i}, \quad \text{where } v_{t,i} = \frac{w_{t,i}}{\sum_{j=1}^{n} w_{t,j}}.$$

Here, $w_{t,i}$ represents the weight of expert i at trial t, and $v_{t,i}$ is its normalized weight.

Key Features:

- **Continuous Predictions:** Unlike WMA, WAA produces predictions as weighted averages, which can take any value in $[0, 1]$.

- **Multiplicative Weight Updates:** Weights are updated multiplicatively based on the incurred loss:

$$w_{t+1,i} = w_{t,i} \cdot e^{-\eta L(y_t, x_{t,i})},$$

  where $L(y_t, x_{t,i})$ is the loss function, and $\eta > 0$ is the learning rate.

8.5.2 Algorithm Description

Algorithm 1 Weighted Average Algorithm (WAA)

Initialize:

- Set initial weights $v_1 = \left(\frac{1}{n}, \ldots, \frac{1}{n}\right)$.
- Initialize cumulative loss counters $L_{WA} := 0$ and $L_i := 0$ for all experts.

Input:

- Learning rate $\eta \in (0, \infty)$.
- Loss function $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$.

1: **for** $t = 1$ to m **do**
2: Receive expert predictions $x_t \in [0, 1]^n$.
3: Compute prediction: $\hat{y}_t := v_t \cdot x_t$.
4: Observe true outcome $y_t \in [0, 1]$.
5: Compute loss:

$$L_{WA} := L_{WA} + L(y_t, \hat{y}_t).$$

6: Update experts' losses:

$$L_i := L_i + L(y_t, x_{t,i}) \quad \text{for } i \in [n].$$

7: Update weights:

$$v_{t+1,i} := \frac{v_{t,i} e^{-\eta L_i}}{\sum_{j=1}^{n} v_{t,j} e^{-\eta L_j}} \quad \text{for } i \in [n].$$

8: **end for**

8.5.3 Regret Analysis for WAA

Theorem 8.5.2 (Regret Bound for WAA)**.** *For any sequence of examples $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$, the regret of WAA satisfies:*

$$L_{WA}(S) - \min_i L_i(S) \leq \frac{\ln(n)}{\eta},$$

where:

- *$L_{WA}(S)$ is the cumulative loss of WAA over sequence S.*

- $L_i(S)$ *is the cumulative loss of expert* i *over sequence* S*.*
- $\eta > 0$ *is the learning rate.*

Concrete Examples:

- *Using the* ***entropic loss****, defined as:*

$$L(y_t, \hat{y}_t) = y_t \ln \frac{y_t}{\hat{y}_t} + (1 - y_t) \ln \frac{1 - y_t}{1 - \hat{y}_t},$$

  setting $\eta = 1$ *achieves this regret bound.*

- *For* ***squared loss****, defined as:*

$$L(y_t, \hat{y}_t) = (y_t - \hat{y}_t)^2,$$

  setting $\eta = 0.5$ *suffices.*

8.5.4 Generalized Regret Bounds for WAA

Remark (Connecting WAA to Generalized Regret Bounds)**.** *The WAA can be understood as a refinement and generalization of the expert-based learning model:*

- *By allowing continuous predictions, it provides a more nuanced prediction mechanism compared to the binary outputs of WMA.*
- *The regret bound:*

$$L_{WA}(S) - \min_i L_i(S) \leq o(m)$$

  implies that, asymptotically, the average loss of WAA matches the average loss of the best expert in the sequence.

- *This sublinear regret bound ensures that:*

$$\frac{L_{WA}(S) - \min_i L_i(S)}{m} \leq \frac{o(m)}{m}.$$

  Since $\frac{o(m)}{m} \to 0$ *as* $m \to \infty$*, this means that the algorithm's performance improves over time, eventually approaching that of the best expert.*

8.5.5 Comparison to Weighted Majority Algorithm (WMA)

Remark (Comparison to WMA)**.** *Both WAA and WMA leverage expert predictions but differ in key ways:*

- ***Support for Arbitrary Loss Functions:*** *WAA supports general loss functions $L(y, \hat{y}) \to [0, \infty)$, allowing broader applicability.*
- ***Continuous Predictions:*** *Unlike WMA, which makes discrete decisions based on majority votes, WAA computes a weighted average prediction. This enables:*
  - *Smoother transitions in learning,*
  - *More precise adaptations in non-binary settings.*
- ***Stronger Generalization Properties:*** *By allowing predictions to exist in the continuous domain, WAA adapts better to regression-type problems, where WMA would be inherently limited.*

8.6 Hedge Algorithm

Definition 8.6.1 (Hedge Algorithm)**.** The Hedge Algorithm generalizes the Weighted Majority Algorithm (WMA) to the ***allocation setting***, where instead of choosing a single action, the learner distributes its weight across multiple actions. The steps of the Hedge Algorithm are as follows:

- On each trial t, the learner selects a **probability distribution** $v_t \in \Delta^n$ over n actions, where Δ^n represents the **simplex of possible distributions**.
- The environment reveals a loss vector $\ell_t \in [0, 1]^n$, where each component $\ell_{t,i}$ represents the loss incurred by action i.
- The learner incurs a weighted loss $v_t \cdot \ell_t = \sum_{i=1}^n v_{t,i}\ell_{t,i}$.
- The weights are updated multiplicatively:

$$w_{t+1,i} = w_{t,i}e^{-\eta\ell_{t,i}}, \quad v_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^n w_{t+1,j}},$$

  where $\eta > 0$ is the learning rate parameter.

Allocation Setting

In the allocation setting:

- *The learner's goal is to allocate its weights optimally to minimize cumulative loss over m rounds.*

- *Two models for the learner's decision-making are considered:*
  1. ***HA-1**: The learner incurs the loss $L_A(t) = v_t \cdot \ell_t$.*
  2. ***HA-2**: The learner selects an action $\iota \in [n]$ randomly according to the distribution v_t, incurring the loss $\ell_{t,\iota}$. The expected loss is:*

$$\mathbb{E}[L_{HA}(t)] = v_t \cdot \ell_t.$$

Theorem 8.6.2 (Hedge Regret Bound)**.** *For any sequence of loss vectors $\ell_1, \ldots, \ell_m \in [0,1]^n$, the regret of the Hedge Algorithm (HA-2) satisfies:*

$$\mathbb{E}[L_{HA}(S)] - \min_i L_i(S) \leq \sqrt{2m \ln n},$$

where $S = \{\ell_1, \ldots, \ell_m\}$, and the learning rate is chosen as $\eta = \sqrt{\frac{\ln n}{2m}}$.

Proof: "Progress versus Regret" Inequality

*We first establish the following **progress versus regret inequality**:*

$$v_t \cdot \ell_t - u \cdot \ell_t \leq \frac{1}{\eta}\left(d(u, v_t) - d(u, v_{t+1})\right) + \frac{\eta}{2}\sum_{i=1}^{n} v_{t,i}\ell_{t,i}^2, \quad \forall u \in \Delta_n.$$

Step 1: Definition of Weight Update Rule *Define the normalizing constant:*

$$Z_t := \sum_{i=1}^{n} v_{t,i} \exp(-\eta \ell_{t,i}),$$

which ensures that the updated weights satisfy:

$$v_{t+1,i} = \frac{v_{t,i} \exp(-\eta \ell_{t,i})}{Z_t}.$$

Step 2: Change in Bregman Divergence *The change in Bregman divergence is given by:*

$$d(u, v_t) - d(u, v_{t+1}) = \sum_{i=1}^{n} u_i \ln \frac{v_{t+1,i}}{v_{t,i}}.$$

Expanding the logarithm:

$$= -\eta \sum_{i=1}^{n} u_i \ell_{t,i} - \sum_{i=1}^{n} u_i \ln Z_t.$$

Using Jensen's inequality on Z_t, we obtain:

$$-\sum_{i=1}^{n} u_i \ln Z_t \geq -\ln \sum_{i=1}^{n} v_{t,i} e^{-\eta \ell_{t,i}}.$$

Applying the bound $e^{-x} \leq 1 - x + \frac{x^2}{2}$ for $x \geq 0$, we approximate:

$$\sum_{i=1}^{n} v_{t,i} e^{-\eta \ell_{t,i}} \leq \sum_{i=1}^{n} v_{t,i}(1 - \eta \ell_{t,i} + \frac{1}{2}\eta^2 \ell_{t,i}^2).$$

Thus, using $\ln(1+x) \leq x$:

$$d(u, v_t) - d(u, v_{t+1}) \geq \eta(v_t \cdot \ell_t - u \cdot \ell_t) - \frac{1}{2}\eta^2 \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2.$$

Rearranging yields the desired inequality.

Step 3: Summing Over All Rounds *Summing over t from 1 to m, we obtain:*

$$\sum_{t=1}^{m} (v_t \cdot \ell_t - u \cdot \ell_t) \leq \frac{1}{\eta}(d(u, v_1) - d(u, v_{m+1})) + \frac{\eta}{2} \sum_{t=1}^{m} \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2.$$

Using the bounds:

$$d(u, v_1) \leq \ln n, \quad -d(u, v_{m+1}) \leq 0, \quad \sum_{t=1}^{m} \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2 \leq m,$$

we optimize η by setting:

$$\eta = \sqrt{\frac{2 \ln n}{m}},$$

which gives the final regret bound:

$$\mathbb{E}[L_{WA}(S)] - \min_i L_i(S) \leq \sqrt{2m \ln n}.$$

Extensions to Loss in Range $[0, B]$

If the loss is now in the range $[0, B]$, we scale the analysis accordingly.

Step 1: Scaling the Inequalities *Since the original proof assumes losses in* $[0, 1]$*, we rescale the loss:*

$$\ell'_{t,i} = \frac{\ell_{t,i}}{B} \in [0, 1].$$

Applying the same analysis with $\ell'_{t,i}$ *instead of* $\ell_{t,i}$*, the regret bound scales accordingly.*

Step 2: Adjusting the Learning Rate *Substituting* $\ell'_{t,i} = \ell_{t,i}/B$ *in the regret bound, we get:*

$$\sum_{t=1}^{m}(v_t \cdot \ell_t - u \cdot \ell_t) \leq \frac{\ln n}{\eta} + \frac{\eta B^2}{2}\sum_{t=1}^{m}\sum_{i=1}^{n} v_{t,i}\ell'^2_{t,i}.$$

Since $\ell'^2_{t,i} = (\ell_{t,i}/B)^2$*, we modify* η *to:*

$$\eta = \sqrt{\frac{2\ln n}{mB^2}}.$$

This results in the final regret bound:

$$\mathbb{E}[L_{WA}(S)] - \min_i L_i(S) \leq B\sqrt{2m\ln n}.$$

Conclusion *Thus, when the loss is scaled to* $[0, B]$*, the regret bound is modified accordingly while maintaining the same dependence on* m *and* n*, up to a multiplicative factor of* B*.*

So far we have given bounds which grow slowly in the number of experts. The only significant drawback is potentially computational if we wish to work with large classes of experts. With this is mind we may wish to work with structured sets of experts for either computational advantages or advantages in bound. We now consider ***linear combinations of experts that are linear classifiers****.*

Chapter 9

Online Learning (Part II)

9.1 Online Learning of Linear Classifiers

9.1.1 The Perceptron Algorithm

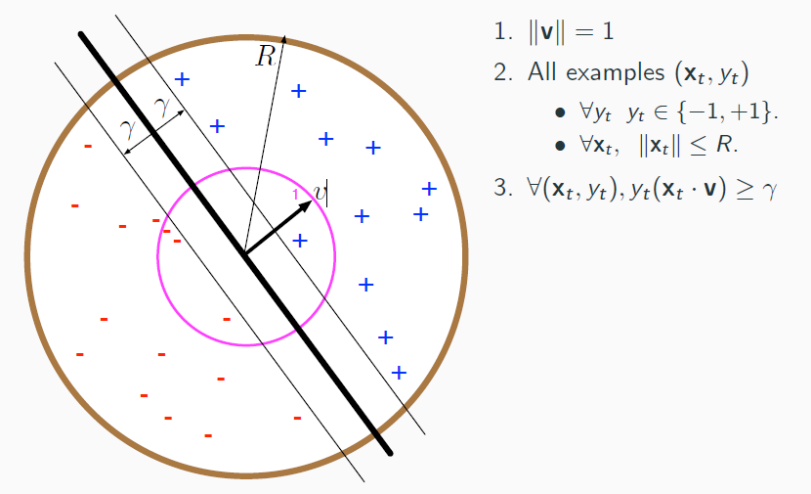

Figure 9.1: Perceptron Set-up: Assuming data is linearly separable by some margin γ. Hence there exists a hyperplane with normal vector v .

Definition 9.1.1 (Perceptron Algorithm)**.** The Perceptron algorithm is an online learning method for training linear classifiers. Given input-output pairs (x_t, y_t) for $t = 1, \ldots, T$, where:

- $x_t \in \mathbb{R}^n$ is the input,

- $y_t \in \{-1, +1\}$ is the corresponding label,

the algorithm iteratively updates the **weight vector** w_t based on prediction mistakes:

1. **Initialization**: Start with $w_1 = 0$.
2. **Prediction**: At time t, predict the label (linear decision rule):

   $$\hat{y}_t = \text{sign}(w_t \cdot x_t),$$

   where $\hat{y}_t \in \{-1, +1\}$.
3. **Update Rule**: If a mistake occurs ($y_t \hat{y}_t \leq 0$), update the weights:

   $$w_{t+1} = w_t + y_t x_t.$$

   Otherwise, $w_{t+1} = w_t$.

Theoretical Guarantees of the Perceptron Algorithm

Theorem 9.1.2 (Perceptron Mistake Bound)**.** *If the data is linearly separable, meaning there exists a weight vector u such that:*

$$y_t(u \cdot x_t) \geq \gamma, \quad \forall t,$$

where $\gamma > 0$ is the margin of separation and $\|u\| = 1$, and if $\|x_t\| \leq R$ for all t, then the total number of mistakes M made by the Perceptron algorithm is bounded by (converge to):

$$M \leq \left(\frac{R}{\gamma}\right)^2.$$

which means the Perceptron will stop updating once it finds a separating hyperplane.

Bound on Number of Mistakes. The proof relies on tracking the growth of the weight vector w_t and leveraging the separability assumption:

1. **Growth of the Inner Product with u**: At each mistake t, the update increases the alignment of w_t with u:

   $$w_{t+1} \cdot u = w_t \cdot u + y_t(x_t \cdot u).$$

   Using the separability assumption $y_t(u \cdot x_t) \geq \gamma$, we get:

   $$w_{t+1} \cdot u \geq w_t \cdot u + \gamma.$$

   After M mistakes:

   $$w_{M+1} \cdot u \geq M\gamma.$$

2. **Bounding the Norm of w_t**: Each mistake also increases the norm of w_t:

$$\|w_{t+1}\|^2 = \|w_t\|^2 + \|x_t\|^2 + 2y_t(w_t \cdot x_t).$$

Since $y_t(w_t \cdot x_t) \leq 0$ (due to the mistake), we have:

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \|x_t\|^2.$$

Using $\|x_t\| \leq R$, after M mistakes:

$$\|w_{M+1}\|^2 \leq MR^2.$$

3. **Combining the Bounds**: Using the Cauchy-Schwarz inequality:

$$w_{M+1} \cdot u \leq \|w_{M+1}\|\|u\|,$$

and substituting the bounds:

$$M\gamma \leq \sqrt{MR^2}.$$

Solving for M gives:

$$M \leq \left(\frac{R}{\gamma}\right)^2.$$

□

Lower Bound on $\|w_t\|$

Lemma 9.1.3 (Lower Bound on $\|w_t\|$)**.** *Let M_t denote the number of mistakes made up to iteration t, and assume there exists a vector v with $\|v\| = 1$ such that the data is linearly separable by a margin $\gamma > 0$. Then:*

$$M_t\gamma \leq \|w_t\|.$$

Proof. We observe that:

$$\|w_t\| \geq w_t \cdot v,$$

because $\|v\| = 1$ (Cauchy-Schwarz inequality). Using induction on t, we show that:

$$w_t \cdot v \geq M_t\gamma.$$

- **Base Case ($t = 1$)**: $w_1 = 0$, so $w_1 \cdot v = 0$, and the claim holds as $M_1 = 0$.
- **Inductive Step**: Assume the claim holds for t. For $t + 1$, if a mistake occurs ($M_{t+1} = M_t + 1$), the update rule gives:

$$w_{t+1} = w_t + y_t x_t.$$

Then:

$$w_{t+1} \cdot v = (w_t + y_t x_t) \cdot v = w_t \cdot v + y_t(x_t \cdot v).$$

By the separability assumption, $y_t(x_t \cdot v) \geq \gamma$. Therefore:

$$w_{t+1} \cdot v \geq w_t \cdot v + \gamma \geq M_t\gamma + \gamma = (M_t + 1)\gamma.$$

Thus, $w_{t+1} \cdot v \geq M_{t+1}\gamma$, completing the induction. □

Combining the Upper and Lower Bounds

Using the results from the upper and lower bounds, we have:

$$M\gamma \leq \|w_{M+1}\| \leq \sqrt{M}R,$$

where $R = \max_t \|x_t\|$. *Rearranging terms gives:*

$$M\gamma^2 \leq R^2 \quad \Longrightarrow \quad M \leq \left(\frac{R}{\gamma}\right)^2.$$

Theorem 9.1.4 (Perceptron Mistake Bound (Novikoff))**.** *For any sequence of examples* $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$ *where* $x_t \in \mathbb{R}^n$, $y_t \in \{-1, +1\}$, *if there exists a vector* v *with* $\|v\| = 1$ *and margin* $\gamma > 0$ *such that:*

$$y_t(x_t \cdot v) \geq \gamma, \quad \forall t,$$

then the total number of mistakes M *made by the Perceptron algorithm is bounded by:*

$$M \leq \left(\frac{R}{\gamma}\right)^2,$$

where $R = \max_t \|x_t\|$.

-

Comments on Perceptron

Theorem 9.1.5. *It is often convenient to express the mistake bound in the following form. Let* $u := \frac{v}{\gamma}$. *Then:*

$$M \leq R^2\|u\|^2,$$

where u *satisfies:*

$$y_t(u \cdot x_t) \geq 1, \quad \forall t.$$

For linearly separable data:

1. w_{m+1} *does not necessarily separate* S *even after convergence because the Perceptron does not optimize for the largest margin.*

2. *The Perceptron algorithm can be adapted to find* ***a separating hyperplane*** w*, e.g., by averaging the weight updates or introducing margin constraints.*

3. *Computing the separating hyperplane depends on the number of mistakes* M*, which scales as* $\mathcal{O}(M)$*.*

Remark. *Variants of the Perceptron algorithm exist, such as:*

- ***Perceptron with Margin****: Modifies the update rule to enforce a larger margin.*
- ***Kernel Perceptron****: Maps the input* x_t *into a higher-dimensional feature space using a kernel* $k(x, x') = \phi(x) \cdot \phi(x')$*.*
- ***SVM-like Perceptron****: Optimizes for the maximum-margin linear separator.*

9.2 Regret Bounds for Linear Separation

9.2.1 Supervised Learning and Regularization

In supervised learning, we aim to find a hypothesis h^* *that minimizes both the empirical loss and a regularization penalty:*

$$h^* = \arg\min_{h \in \mathcal{H}} \sum_{t=1}^{m} L(y_t, h(x_t)) + \lambda \textit{penalty}(h).$$

This framework is commonly used in models like:

- ***Ridge Regression****: Regularizing with* ℓ_2*-norm:*

$$\arg\min_{w \in \mathbb{R}^n} \sum_{t=1}^{m} L(y_t, w \cdot x_t) + \lambda \|w\|^2.$$

- ***Soft Margin SVM****: Using hinge loss for classification:*

$$\arg\min_{w \in \mathbb{R}^n, b \in \mathbb{R}} \sum_{t=1}^{m} L_{hi}(y_t, w \cdot x_t + b) + \lambda \|w\|^2,$$

where $L_{hi}(y, \hat{y}) = \max(0, 1 - y\hat{y})$*.*

9.2.2 From Batch to Online Learning

*While the above models operate in a **batch** setting (where all data is available at once), we often need to learn in an **online** fashion. The goal is to update the model after each new sample (x_{t+1}, y_{t+1}), ensuring:*

- ***Fit to the new data point**.*
- ***Continuity with the previous model** (not deviating too much).*

This motivates an online update rule:

$$h_{t+1} = \arg\min_{h \in \mathcal{H}} L(y_t, h(x_t)) + \lambda \textit{penalty}(h, h_t).$$

Standard Perceptron Update

For linearly separable data, the Perceptron algorithm iteratively updates the weight vector as:

$$w_{t+1} = \begin{cases} w_t + y_t x_t, & \textit{if } y_t(w_t \cdot x_t) \leq 0, \\ w_t, & \textit{otherwise.} \end{cases}$$

Perceptron with Margin

For non-separable data, a margin $\delta > 0$ is introduced to ensure confident classification:

$$w_{t+1} = \begin{cases} w_t + y_t x_t, & \textit{if } y_t(w_t \cdot x_t) \leq \delta, \\ w_t, & \textit{otherwise.} \end{cases}$$

Kernel Perceptron

To handle non-linear separability, we extend the Perceptron using a feature map $\phi(x_t)$ and a kernel function $k(x, x') = \phi(x) \cdot \phi(x')$. The update rule becomes:

$$w_{t+1} = w_t + y_t \phi(x_t).$$

9.2.3 Online Gradient Descent (OGD) for Linear Classifiers

Regret in Online Learning

The regret in an online learning setting measures how much worse the algorithm performs compared to the best fixed weight vector in hindsight:

$$R_T = \sum_{t=1}^{T} \ell_t(w_t) - \min_{w \in \mathcal{W}} \sum_{t=1}^{T} \ell_t(w).$$

where $\ell_t(w)$ is the loss function at time t, and $\mathcal{W}$ is the hypothesis space.

Hinge Loss and Linear Models

For a linear classifier, the hinge loss is used:

$$\ell_t(w) = \max(0, 1 - y_t(w \cdot x_t)).$$

The hinge loss ensures that predictions have a margin of at least 1.

Online Gradient Descent with Hinge Loss

Applying Online Gradient Descent (OGD) with hinge loss and an ℓ_2-regularization term:

$$w_{t+1} = \arg\min_{w \in \mathbb{R}^n} L_{hi}(y_t, w \cdot x_t) + \lambda \|w - w_t\|^2.$$

Solving for w_{t+1}, we obtain the update rule:

$$w_{t+1} = \begin{cases} w_t, & y_t(w_t \cdot x_t) > 1, \\ w_t + \frac{y_t x_t}{2\lambda}, & y_t(w_t \cdot x_t) < 1. \end{cases}$$

Interpretation:

- *If the prediction is already correct and has a sufficient margin, no update is made.*
- *If the prediction is incorrect or too close to the decision boundary, an adjustment is made in the direction of $y_t x_t$.*

Online Gradient Descent (OGD) Algorithm

9.2.4 Regret Bounds for Hinge Loss

Using the regret analysis approach from multiplicative weight updates, we obtain the bound:

$$\sum_{t=1}^{m} (v_t \cdot \ell_t - u \cdot \ell_t) \leq \frac{\ln n}{\eta} + \frac{\eta}{2} \sum_{t=1}^{m} \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2.$$

Algorithm 2 OGD Algorithm for Linear Classification

Initialize: $w_1 := 0$, $L_{\text{OGD}} := 0$.
Select: Learning rate $\eta \in (0, \infty)$, where $\eta = \frac{1}{2\lambda}$.

1: **for** $t = 1$ to m **do**
2: Receive instance $x_t \in \mathbb{R}^n$.
3: Predict: $\hat{y}_t := w_t \cdot x_t$.
4: Receive label $y_t \in \{-1, 1\}$.
5: Compute hinge loss:

$$L_{\text{OGD}} := L_{\text{OGD}} + L_{\text{hi}}(y_t, \hat{y}_t).$$

6: Update weights:

$$w_{t+1} := w_t + 1_{\{y_t \hat{y}_t < 1\}} \eta y_t x_t.$$

7: **end for**

Choosing $\eta = \sqrt{\frac{\ln n}{2m}}$ minimizes this bound to:

$$\mathbb{E}[L_{OGD}(S)] - \min_i L_i(S) \leq \sqrt{2m \ln n}.$$

Extension to Loss in Range $[0, B]$

If the loss is in the range $[0, B]$, we scale η accordingly:

$$\eta = \sqrt{\frac{\ln n}{2mB^2}}.$$

This results in the adjusted regret bound:

$$\mathbb{E}[L_{OGD}(S)] - \min_i L_i(S) \leq B\sqrt{2m \ln n}.$$

Theorem 9.2.1 (Regret Bound for OGD with Hinge Loss)**.** *For the hinge loss $\ell_t(w)$, assume:*

- $\|x_t\| \leq R$, $\forall t$,
- $\|w\| \leq U$, *for* $w \in \mathcal{W}$,
- *Learning rate:* $\eta = \frac{U}{R\sqrt{T}}$.

Then the regret for Online Gradient Descent satisfies:

$$R_T \leq UR\sqrt{T}.$$

Sketch of Proof. The proof relies on:

1. Bounding the regret as:
$$R_T = \sum_{t=1}^{T} \ell_t(w_t) - \sum_{t=1}^{T} \ell_t(w^*),$$
where $w^* = \arg\min_{w \in \mathcal{W}} \sum_{t=1}^{T} \ell_t(w)$.
2. Applying convexity of $\ell_t(w)$ and using the update rule for Online Gradient Descent:
$$w_{t+1} = \Pi_{\mathcal{W}}(w_t - \eta \nabla \ell_t(w_t)).$$
3. Bounding the step size $\|w_t - w^*\|$ using the geometry of the convex set $\mathcal{W}$.

□

—

9.2.5 Online Support Vector Machine (SVM)

Definition 9.2.2 (Regularized Hinge Loss)**.** To handle non-separable data, we introduce a regularization term into the hinge loss:

$$\ell_t^{\text{reg}}(w) = \frac{\lambda}{2}\|w\|^2 + \max(0, 1 - y_t(w \cdot x_t)),$$

where $\lambda > 0$ is the regularization parameter.

Theorem 9.2.3 (Regret Bound for Regularized Hinge Loss)**.** *For the regularized hinge loss, the regret satisfies:*

$$R_T^{reg} \leq \frac{1}{2\lambda} + \sqrt{T}.$$

Remark. *The regularization term $\frac{\lambda}{2}\|w\|^2$ prevents the weight vector w from growing indefinitely, stabilizing the learning process.*

—

9.2.6 Kernelized Online Learning

For non-linear data, we can extend the Perceptron and Online Gradient Descent algorithms using kernels.

Definition 9.2.4 (Kernel Trick)**.** The kernel trick implicitly maps inputs $x \in \mathbb{R}^n$ to a higher-dimensional feature space $\phi(x)$ using a kernel function $k(x, x') = \phi(x) \cdot \phi(x')$. For example:

- Polynomial kernel: $k(x, x') = (x \cdot x' + c)^d$,
- RBF kernel: $k(x, x') = \exp\left(-\frac{\|x-x'\|^2}{2\sigma^2}\right)$.

Remark. *Kernelized algorithms maintain an explicit weight vector only in terms of support vectors (examples where $\ell_t(w_t) > 0$).*

—

9.2.7 Wrapping Up

Remark (Comparing Algorithms)**.** *- **Perceptron**: Guarantees convergence for linearly separable data with mistake bounds. Fails to handle non-separable data. - **Online Gradient Descent**: Minimizes regret for convex loss functions, including hinge loss. - **Kernel Methods**: Extend Perceptron and OGD to handle non-linear data effectively.*

Remark (Summary of Regret Bounds)**.** *For linearly separable data:*

$$M \leq \left(\frac{R}{\gamma}\right)^2.$$

For non-separable data:

$$R_T \leq UR\sqrt{T}, \quad R_T^{reg} \leq \frac{1}{2\lambda} + \sqrt{T}.$$

Chapter 10

Structured Prediction

10.1 Surrogate Methods

10.1.1 Introduction to Structured Prediction

Definition 10.1.1. Structured prediction involves predicting outputs with internal dependencies, unlike traditional classification or regression tasks where outputs are independent.

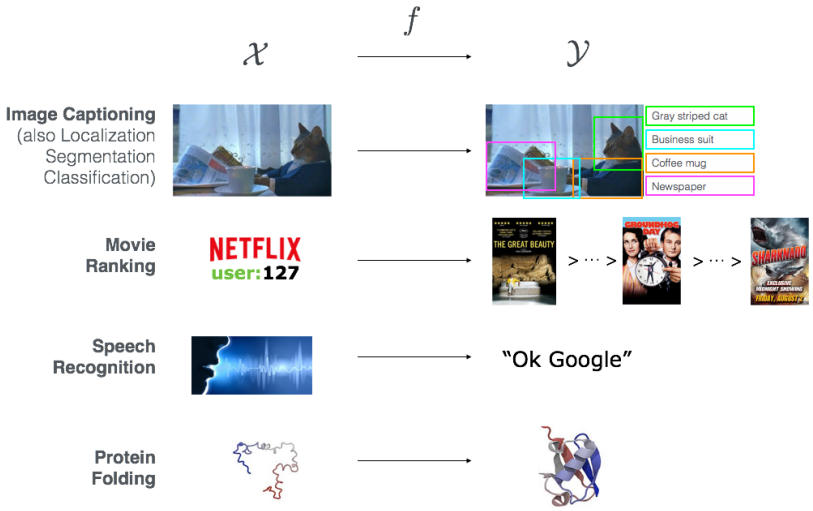

Figure 10.1: Examples of structured prediction tasks.

Some common structured prediction problems include:

1. *Image Captioning (localization, segmentation, classification)*

2. *Movie Ranking*
3. *Speech Recognition*
4. *Protein Folding*

10.1.2 Structured Prediction and Local Information

Remark. *Structured prediction can incorporate local information to improve accuracy.*

Examples of such improvements include:

- *Object detection in images with bounding boxes.*
- *Grid-based spatial dependencies in structured models.*
- *Dependency parsing in natural language processing.*
- *Feature-based hierarchical structures in scene understanding.*

10.1.3 Structured Prediction vs. Supervised Learning

Theorem 10.1.2. *Structured prediction is a form of supervised learning, but standard learning methods do not apply due to* ***dependencies in the output space****.*

Remark. *In standard supervised learning, we aim to learn a function $f : \mathcal{X} \to \mathcal{Y}$ given training examples $(x_i, y_i)_{i=1}^n$. However, in structured prediction, $\mathcal{Y}$ has an internal structure that must be respected, which prevents direct application of traditional learning methods.*

Supervised Learning 101

The goal in supervised learning is to find a function f^* that minimizes expected risk:

$$f^* = \arg\min_{f:\mathcal{X}\to\mathcal{Y}} \mathcal{E}(f), \quad \mathcal{E}(f) = \mathbb{E}[\ell(f(x), y)] \tag{10.1}$$

given ***only*** *the dataset $(x_i, y_i)_{i=1}^n$ sampled independently from ρ, where $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ is a loss function and ρ is an* ***unknown probability distribution****.*

Supervised Learning 101 - A Wishlist

- ***Consistency:*** $\lim_{n\to\infty} \mathcal{E}(f_n) - \mathcal{E}(f^*) = 0$
- ***Learning Rates:*** $\mathcal{E}(f_n) - \mathcal{E}(f^*) \leq O(n^{-\gamma})$*, for some $\gamma > 0$ (the larger the better).*

10.1.4 Prototypical Approach: Empirical Risk Minimization

Definition 10.1.3. Empirical Risk Minimization (ERM) solves:

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i) \tag{10.2}$$

where $\mathcal{F}$ is a function space (usually a convex function space).

Theorem 10.1.4. *If $\mathcal{Y}$ is a **vector space**, $\mathcal{F}$ is "easy" to choose/optimize over: linear models, Kernel methods, Neural Networks, etc. If $\mathcal{Y}$ is a **structured space**, challenges arise in choosing $\mathcal{F}$, optimizing over it, and studying its statistics.*

Prototypical Results

Several results allow to study ERM's consistency and rates when:

- $\mathcal{Y} = \mathbb{R}^d$ *and,*
- $\mathcal{F}$ *is a "standard" space of functions (e.g., a reproducing kernel Hilbert space).*

Examples of techniques/notions involved to obtain these results:

- *VC dimension,*
- *Rademacher & Gaussian complexity,*
- *Covering numbers,*
- *Stability,*
- *Empirical processes,*
- ...

10.1.5 Classification as Structured Prediction

Definition 10.1.5. Classification is a structured prediction task since **linear models do not always satisfy the function requirement** $f : \mathcal{X} \to \mathcal{Y}$. Consider two linear models:

$$f_1(x) = w_1^T \phi(x), \quad f_2(x) = w_2^T \phi(x) \tag{10.3}$$

A linear (or even convex) combination does not necessarily belong to $\mathcal{Y}$:

$$f_n(x) = \alpha_1 f_1(x) + \alpha_2 f_2(x) = (\alpha_1 w_1 + \alpha_2 w_2)^T \phi(x) \tag{10.4}$$

where $\mathcal{Y} = 1, -1$

Example 10.1.6. A common classification approach is:

1. Learn a linear model $f : \mathcal{X} \to \mathbb{R}$ such that $f(x) = w^T \phi(x)$.
2. Map $f(x)$ back to $\mathcal{Y} = \{-1, 1\}$ using sign$(f(x))$.

However, this raises two main questions about this:

- Does this approach actually work?
- How to extend **sign-based mapping** when $\mathcal{Y}$ contains complex structures (e.g., graphs)?

10.1.6 Structured Prediction Methods

$\mathcal{Y}$ arbitrary: How do we parametrize $\mathcal{F}$ and learn f_n?

Surrogate Approaches

- ***Advantages:*** *Well-defined theory, including convergence and learning rates.*
- ***Limitations:*** *Applicable mainly to specific cases (classification, ranking, multi-label learning).*

Likelihood Estimation Methods

- ***Advantages:*** *General algorithmic framework (e.g., Structured SVMs).*
- ***Limitations:*** *Lacks theoretical consistency.*

10.1.7 Surrogate Approaches

Surrogate approaches generalize classification by defining:

Definition 10.1.7.

- An **encoding** $c : \mathcal{Y} \to \mathcal{H}$ maps the output space into a **suitable *linear* feature space** $\mathcal{H}$ (e.g., $\mathcal{H} = \mathbb{R}$).
- A **surrogate loss** $L : \mathcal{H} \times \mathcal{H} \to \mathbb{R}$ defines a learning problem to find $g_n : \mathcal{X} \to \mathcal{H}$ in a suitable (linear) *hypothesis space* $\mathcal{G}$:

$$g_n = \arg\min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^{n} L(g(x_i), c(y_i)). \tag{10.5}$$

- A **decoding** function $d : \mathcal{H} \to \mathcal{Y}$ ensures the final model is written as:

$$f_n = d \circ g_n : \mathcal{X} \to \mathcal{Y}, \quad \text{such that} \quad f_n(x) = d(g_n(x)). \tag{10.6}$$

10.1.8 Binary Classification

Let $\mathcal{Y} = \{-1, 1\}$:

- ***Encoding:*** *The function $c : \mathcal{Y} \to \mathbb{R}$ is the identity map.*
- ***Surrogate loss:*** *Common choices include squared, hinge, logistic, and exponential loss functions. The function $g_n : \mathcal{X} \to \mathbb{R}$ is learned by solving a regression problem.*
- ***Decoding:*** *The function $d : \mathbb{R} \to \mathcal{Y}$ is the sign function.*

10.1.9 Multi-class Classification

Let $\mathcal{Y} = \{1, 2, \ldots, T\}$ for T classes:

- ***Encoding:*** *The function $c : \mathcal{Y} \to \mathbb{R}^T$ is the one-hot encoding $c(t) = e_t \in \mathbb{R}^T$, where e_t is a vector of all zeros except in the t-th entry.*
- ***Surrogate loss:*** *The One-vs-All (OVA) loss is defined as:*

$$L(g(x), e_y) = \sum_{t=1}^{T} \ell(g(x)^{(t)}, e_y^{(t)}), \tag{10.7}$$

  where ℓ can be squared, hinge, logistic, exponential loss, etc.

- ***Decoding:*** *The function $d : \mathbb{R}^T \to \mathcal{Y}$ is defined as:*

$$d(g(x)) = \arg \max_{t=1,\ldots,T} e_t^\top g(x). \tag{10.8}$$

10.1.10 Likelihood Estimation Methods

Definition 10.1.8. Likelihood Estimation Methods estimate probabilities directly from data. Given a dataset of joint samples $(x_i, y_i)_{i=1}^n$, the method follows two steps:

- ***Approximation:*** *Estimate $\mathbb{P}(y|x)$ using some function $P_n : \mathcal{Y} \times \mathcal{X} \to \mathbb{R}$. The advantage is that P_n becomes **real-valued**, allowing linear models:*

$$P_n(y, x) = w_n^\top \psi(y, x), \tag{10.9}$$

  where $\psi : \mathcal{Y} \times \mathcal{X} \to \mathbb{R}$ is a feature representation.

- ***Prediction:*** *The most likely output is predicted as:*

$$f_n(x) = \arg \max_{y \in \mathcal{Y}} P_n(y, x). \tag{10.10}$$

10.2 Implicit Loss Embeddings

10.2.1 Wish List

We would like a method that:

- *Is **flexible**: can be applied to (m)any $\mathcal{Y}$ and ℓ.*
- *Leads to efficient **computations**.*
- *Has strong **theoretical** guarantees (i.e., consistency, rates).*

10.2.2 Angle of Attack

Remark. *We will try to define a Surrogate framework that can be applied in **general** and not on an ad-hoc basis.*

We will try to work backwards by:

- *Starting from the characterization of the **ideal solution** f^* (Since it does not require a choice of $\mathcal{F}$).*
- *Hopefully finding out that f^* exhibits some **interesting structures/patterns**.*
- *Seeing if we can exploit such patterns to derive some rules to define a **general surrogate framework**.*

10.2.3 Ideal Solution

Theorem 10.2.1 (Expected Risk)**.** *The expected risk of our problem is given by:*

$$\mathcal{E}(f) = \int \ell(f(x), y)\, d\rho(x, y) \tag{10.11}$$

which can be rewritten as:

$$\mathcal{E}(f) = \int \left(\int \ell(f(x), y)\, d\rho(y|x) \right) d\rho_{\mathcal{X}}(x). \tag{10.12}$$

To minimize the risk pointwise, the best $f^* : \mathcal{X} \to \mathcal{Y}$ is:

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} \int \ell(z, y)\, d\rho(y|x). \tag{10.13}$$

Remark. *f^* is the point-wise minimizer of the expectation $\mathbb{E}_{y|x}\ell(\cdot, y)$ conditioned w.r.t. x.*

10.2.4 Finite Dimensional Intuition

Definition 10.2.2. Consider the finite case, where $\mathcal{Y} = \{1, \dots, T\}$. This generalizes the multi-class classification setting (since we are not assuming a 0-1 loss). Then, any $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ is represented by a **matrix** $V \in \mathbb{R}^{T \times T}$:

$$\ell(y, z) = V_{yz} = e_y^\top V e_z, \quad \forall y, z \in \mathcal{Y} \tag{10.14}$$

where e_y is the **one-hot** encoding of y, or more formally, it is the y-th element of the canonical basis of $\mathbb{R}^T$.

Going back to the point-wise characterization of f^*:

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} \int \ell(z, y)\, d\rho(y|x) \tag{10.15}$$

$$= \arg\min_{z \in \mathcal{Y}} \int e_z^\top V e_y\, d\rho(y|x) \tag{10.16}$$

$$= \arg\min_{z \in \mathcal{Y}} e_z^\top V \underbrace{\int e_y\, d\rho(y|x)}_{g_*(x)}. \tag{10.17}$$

Now, define $g_* : \mathcal{X} \to \mathbb{R}^T$ as the conditional expectation of e_y given x:

$$g_*(x) = \mathbb{E}[e_y|x] = \int e_y\, d\rho(y|x). \tag{10.18}$$

Then:

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} e_z^\top V g_*(x). \tag{10.19}$$

Angle of Attack

*We will try to define a Surrogate framework that can be applied in general and not on an ad-hoc basis. We will try to work backwards by starting from the characterization of the **ideal solution** f^* (Since it does not require a choice of $\mathcal{F}$) and hopefully find out that f^* exhibits some interesting **structures/patterns**, which we can exploit to derive some rules to define a general surrogate framework.*

Remark. ***Idea:*** *Replace $g_* : \mathcal{X} \to \mathbb{R}^T$ in*

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} e_z^\top V g_*(x). \tag{10.20}$$

... with an estimator $g_n : \mathcal{X} \to \mathbb{R}^T$:

$$f_n(x) = \arg\min_{z \in \mathcal{Y}} e_z^\top V g_n(x). \tag{10.21}$$

Questions:

- *How to **generalize** this approach to any (not only finite) $\mathcal{Y}$?*
- *How to **learn** g_n?*

10.2.5 General Case: Implicit Embeddings

***Goal:** Generalize the intuition from the finite case to any $\mathcal{Y}$.*

Definition 10.2.3. A continuous $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ admits an **Implicit Embedding (IE)** if there exists a map $c : \mathcal{Y} \to \mathcal{H}$ into a separable Hilbert space $\mathcal{H}$ and a linear operator $V : \mathcal{H} \to \mathcal{H}$ such that:

$$\ell(z, y) = \langle c(z), Vc(y) \rangle_{\mathcal{H}}. \tag{10.22}$$

- For $V = I$, we recover the notion of **reproducing kernel**!
- Accounts for non-positive definite, non-symmetric functions.
- Holds also for **infinite dimensional** spaces $\mathcal{H}$!

IE Example: Squared Loss

Consider $\ell(y, y') = (y - y')^2$ with $\mathcal{Y} = \mathbb{R}$.
Define the polynomial map $c : \mathcal{Y} \to \mathbb{R}^3$:

$$c(y) = \begin{pmatrix} y^2 \\ \sqrt{2}y \\ 1 \end{pmatrix}, \quad V = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}. \tag{10.23}$$

Then we can verify that:

$$\ell(y, y') = c(y)^\top V c(y'). \tag{10.24}$$

IE Example: Kernel Dependency Estimation (KDE)

*KDE losses follow the intuition that reproducing kernels are a sort of **measure of similarity** between points (from Lecture 2).*

Definition 10.2.4. Given a kernel $k : \mathcal{Y} \times \mathcal{Y} \to [0, 1]$ on the output space $\mathcal{Y}$, the KDE loss of k is defined as:

$$\triangle(y, y') = 1 - k(y, y'), \tag{10.25}$$

which represents a **measure of "dissimilarity"** between output points.

***Exercise:** Find an explicit choice for c and V. Assume k is normalized, i.e., $k(y, y') \equiv 1$ for all $y \in \mathcal{Y}$.*

10.2.6 Structured Prediction with Implicit Embeddings

Theorem 10.2.5. *If ℓ has an **implicit embedding**, then (by repeating the exact same steps as for finite $\mathcal{Y}$:*

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} \langle c(z), V g_*(x) \rangle_{\mathcal{H}}, \tag{10.26}$$

where $g_* : \mathcal{X} \to \mathcal{H}$ is defined as:

$$g_*(x) = \int c(y)\, d\rho(y|x), \tag{10.27}$$

*which corresponds to the **conditional mean embedding** of $\rho(\cdot|x)$ with respect to the **output kernel**:*

$$k(z, y) = \langle c(z), c(y) \rangle_{\mathcal{H}}. \tag{10.28}$$

10.2.7 Learning f_n in the General Case

***Idea:** Replace $g_* : \mathcal{X} \to \mathcal{H}$ in:*

$$f^*(x) = \arg\min_{z \in \mathcal{Y}} \langle c(z), V g_*(x) \rangle_{\mathcal{H}} \tag{10.29}$$

with an estimator $g_n : \mathcal{X} \to \mathcal{H}$:

$$f_n(x) = \arg\min_{z \in \mathcal{Y}} \langle c(z), V g_n(x) \rangle_{\mathcal{H}}. \tag{10.30}$$

10.3 A General Surrogate Algorithm

10.3.1 Approximating g_*

Theorem 10.3.1 (Least-Squares Risk Minimization)**.** *The function $g_*(x)$ is a **conditional expectation**:*

$$g_*(x) = \int c(y) d\rho(y|x) = \mathbb{E}_{y|x}[c(y)]. \tag{10.31}$$

where g_* is the minimizer of the least-squares risk:

$$g_* = \arg\min_{g : \mathcal{X} \to \mathcal{H}} \mathcal{R}(g), \tag{10.32}$$

and

$$\mathcal{R}(g) = \int \|g(x) - c(y)\|^2 d\rho(x, y). \tag{10.33}$$

Therefore, we can take g_n to be the least-squares ERM estimator.

10.3.2 Defining a General Surrogate Approach

Definition 10.3.2. A **Surrogate Approach** consists of:

- An **encoding** $c : \mathcal{Y} \to \mathcal{H}$ into a suitable **linear** feature space $\mathcal{H}$ (e.g., $\mathcal{H} = \mathbb{R}$).
- A **Surrogate loss** $L : \mathcal{H} \times \mathcal{H} \to \mathbb{R}$ defining a problem to learn a function $g_n : \mathcal{X} \to \mathcal{H}$ over a suitable **hypothesis space** $\mathcal{G}$:

$$g_n = \arg\min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^{n} \|g(x_i) - c(y_i)\|_{\mathcal{H}}^2. \tag{10.34}$$

- A **decoding** function $d : \mathcal{H} \to \mathcal{Y}$, such that the final model is:

$$f_n = d \circ g_n : \mathcal{X} \to \mathcal{Y}, \quad \text{where} \quad f_n(x) = d(g_n(x)). \tag{10.35}$$

10.3.3 Structured Prediction with Implicit Embeddings

Let $\mathcal{X} = \mathbb{R}^d$. We approximate g_* with:

$$g_n(x) = W_n x. \tag{10.36}$$

W_n is obtained as:

$$W_n = \arg\min_{W \in \mathcal{H} \otimes \mathbb{R}^d} \frac{1}{n} \sum_{i=1}^{n} \|c(y_i) - W x_i\|^2 + \lambda \|W\|_F^2. \tag{10.37}$$

Interpretation:

- *If $\mathcal{H} = \mathbb{R}^T$, then $W \in \mathbb{R}^{T \times d}$ is a matrix.*
- *If $\mathcal{H}$ is infinite-dimensional, then $W \in \mathcal{H} \otimes \mathbb{R}^d$ is an operator.*

The closed-form solution is then given by:

$$W_n = Y^\top X (X^\top X + n\lambda I)^{-1}. \tag{10.38}$$

where:

- *$X \in \mathbb{R}^{n \times d}$ contains x_i as rows.*
- *$Y \in \mathbb{R}^n \otimes \mathcal{H}$ contains $c(y_i)$ as rows.*

10.3.4 Implicit Embeddings and Kernel Methods

Theorem 10.3.3 (Generalized Solution)**.** *W_n may contain infinitely many parameters. However, we can rewrite:*

$$g_n(x) = W_n x = Y^\top X(X^\top X + n\lambda I)^{-1}x = \sum_{i=1}^{n} \alpha_i(x)c(y_i), \tag{10.39}$$

where the weights $\alpha : \mathcal{X} \to \mathbb{R}^n$ satisfy:

$$\alpha(x) = (K + n\lambda I)^{-1}v(x). \tag{10.40}$$

*if we have a kernel $\| : \mathcal{X} * \mathcal{X}$. **Kernel Interpretation:***

- *$K \in \mathbb{R}^{n\times n}$ is the kernel matrix, where $K_{ij} = k(x_i, x_j)$.*
- *$v(x) \in \mathbb{R}^n$ is the evaluation vector, where $v(x)_i = k(x_i, x)$.*

Remark. *If $\mathcal{F}$ is the RKHS associated to $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ with feature map $\phi : \mathcal{X} \to \mathcal{F}$, then:*

$$g_n(x) = W_n\phi(x) = \sum_{i=1}^{n} \alpha_i(x)c(y_i), \tag{10.41}$$

which solves the generalized problem:

$$W_n = \arg\min_{W\in\mathcal{H}\otimes\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \|c(y_i) - Wx_i\|_{\mathcal{H}}^2 + \lambda\|W\|_{HS}^2. \tag{10.42}$$

This automatically generalizes to non-linear functions using implicit embeddings.

10.3.5 Final Surrogate Algorithm

*The only remaining question is how to design a **"practical"** decoding.*

- *An **encoding** $c : \mathcal{Y} \to \mathcal{H}$ into a suitable **linear** feature space.*
- *A **Surrogate (squared) loss**:*

$$g_n = \arg\min_{g\in\mathcal{G}} \frac{1}{n}\sum_{i=1}^{n} \|g(x_i) - c(y_i)\|_{\mathcal{H}}^2 + \lambda\|g\|_{\mathcal{G}}^2. \tag{10.43}$$

- *A **decoding** function $d : \mathcal{H} \to \mathcal{Y}$ such that:*

$$d \circ g = \arg\min_{y\in\mathcal{Y}} \langle c(y), Vg(x)\rangle. \tag{10.44}$$

Theorem 10.3.4 (Loss Trick)**.** *Using implicit embeddings, we obtain, analogously to the case of f^* and f^* . . .*

$$f_n(x) = \arg\min_{z\in\mathcal{Y}} \langle c(y), V g_n(x)\rangle \tag{10.45}$$

$$= \arg\min_{z\in\mathcal{Y}} \left\langle c(y), V\left(\sum_{i=1}^{n} \alpha_i(x) c(y_i)\right)\right\rangle \tag{10.46}$$

$$= \arg\min_{z\in\mathcal{Y}} \sum_{i=1}^{n} \alpha_i(x) \underbrace{\langle c(z), V c(y_i)\rangle}_{\textit{loss trick}}. \tag{10.47}$$

In other words,

$$f_n(x) = \arg\min_{z\in\mathcal{Y}} \sum_{i=1}^{n} \alpha_i(x)\ell(z, y_i) \tag{10.48}$$

which avoids explicit knowledge of $(\mathcal{H}, c, V)$!

This approach has two phases:

- ***Learning.*** *Where the score function $\alpha : \mathcal{X} \to \mathbb{R}^n$ is estimated.*
- ***Prediction.*** *Where we need to solve*

$$f_n(x) = \arg\min_{z\in\mathcal{Y}} \sum_{i=1}^{n} \alpha_i(x)\ell(z, y_i). \tag{10.49}$$

Note. *One needs to know how to minimize ℓ over $\mathcal{Y}$ (which can be hard!).*

Surrogate Approaches - Summary

We have finally found our general Surrogate framework!

- *An **encoding** $c : \mathcal{Y} \to \mathcal{H}$ into a suitable **linear** (output) feature space $\mathcal{H}$ (e.g., $\mathcal{H} = \mathbb{R}$).*
- *A **Surrogate loss** $L : \mathcal{H} \times \mathcal{H} \to \mathbb{R}$ defining a problem to learn a function $g_n : \mathcal{X} \to \mathcal{H}$ over a suitable (linear) **hypotheses space** $\mathcal{G}$.*
- *A **decoding** $d : \mathcal{H} \to \mathcal{Y}$, such that the final model is:*

$$d \circ g_n = \arg\min_{y\in\mathcal{Y}} \sum_{i=1}^{n} \alpha_i(x)\ell(y, y_i) \tag{10.50}$$

for suitable α_i.

Does this strategy have any theoretical guarantee?

10.4 Theoretical Properties

10.4.1 Comparison Inequality

Theorem 10.4.1. *Let ℓ admit an implicit embedding (IE). Define:*

$$Q = \sup_y \|c(y)\|_{\mathcal{H}}, \quad \textit{and} \quad \|V\| = \sup_{\|h\|_{\mathcal{H}} \leq 1} \|Vh\|_{\mathcal{H}}, \tag{10.51}$$

which represents the "operator" norm of V. Then, we obtain the following bound:

$$\mathcal{E}(d \circ g) - \mathcal{E}(f_*) \leq 2Q\|V\|\sqrt{\mathcal{R}(g) - \mathcal{R}(g_*)}. \tag{10.52}$$

where:

$$\mathcal{E}(f) = \int \ell(f(x), y) d\rho(x, y), \tag{10.53}$$

$$\mathcal{R}(g) = \int \|g(x) - c(y)\|^2_{\mathcal{H}} d\rho(x, y). \tag{10.54}$$

Direct Consequences:

- *If $\mathcal{R}(g_n) \to \mathcal{R}(g_*)$, then $\mathcal{E}(f_n) \to \mathcal{E}(f_*)$ for $f_n = d \circ g_n$.*
- ***Learning rates** for g_n **translate automatically** to f_n (admittedly slowed down by a squared root).*

10.4.2 Proof of the Comparison Inequality

Denote $f = d \circ g$. Since ℓ has an IE, the excess risk becomes:

$$\mathcal{E}(f) - \mathcal{E}(f^*) = \int_{\mathcal{X}\times\mathcal{Y}} \langle c(f(x)) - c(f^*(x)), Vc(y)\rangle d\rho(x, y) \tag{10.55}$$

$$= \int_{\mathcal{X}} \langle c(f(x)) - c(f^*(x)), V\left(\int_{\mathcal{Y}} c(y) d\rho(y|x)\right)\rangle d\rho_{\mathcal{X}}(x) \tag{10.56}$$

$$= \int_{\mathcal{X}} \langle c(f(x)) - c(f^*(x)), Vg_*(x)\rangle d\rho_{\mathcal{X}}(x). \tag{10.57}$$

By adding and removing the term $\langle c(f(x)), Vg(x)\rangle$:

$$\langle c(f(x)) - c(f^*(x)), Vg_*(x)\rangle = \langle c(f(x)), Vg(x)\rangle - \langle c(f^*(x)), Vg_*(x)\rangle \tag{10.58}$$

$$+ \langle c(f(x)), V(g_*(x) - g(x))\rangle. \tag{10.59}$$

Integrating with respect to $\rho_{\mathcal{X}}$, we conclude:

$$\mathcal{E}(f) - \mathcal{E}(f^*) = A + B, \tag{10.60}$$

where:

$$A = \int \langle c(f(x)), Vg(x)\rangle - \langle c(f^*(x)), Vg_*(x)\rangle d\rho_{\mathcal{X}}(x), \tag{10.61}$$

$$B = \int \langle c(f(x)), V(g_*(x) - g(x))\rangle d\rho_{\mathcal{X}}(x). \tag{10.62}$$

Bounding Term A

From the decoding definition:

$$d \circ g = \arg\min_{y\in\mathcal{Y}} \langle c(y), Vg(x)\rangle. \tag{10.63}$$

Thus:

$$\langle c(f(x)), Vg(x)\rangle = \min_{y\in\mathcal{Y}} \langle c(y), Vg(x)\rangle. \tag{10.64}$$

$$\langle c(f^*(x)), Vg_*(x)\rangle = \min_{y\in\mathcal{Y}} \langle c(y), Vg_*(x)\rangle. \tag{10.65}$$

Which implies that A is:

$$A = \int \min_{y\in\mathcal{Y}} \langle c(y), Vg(x)\rangle - \min_{y\in\mathcal{Y}} \langle c(y), Vg_*(x)\rangle d\rho_{\mathcal{X}}(x). \tag{10.66}$$

Using the inequality:

$$\min_y u(y) - \min_y v(y) \leq \sup_y |u(y) - v(y)|, \tag{10.67}$$

we obtain:

$$A \leq \int \sup_{y\in\mathcal{Y}} |\langle c(y), V(g_*(x) - g(x))\rangle| d\rho_{\mathcal{X}}(x). \tag{10.68}$$

Bounding Term B

By convexity of the absolute value:

$$B = \int \langle c(y), V(g_*(x) - g(x))\rangle d\rho_{\mathcal{X}}(x), \tag{10.69}$$

$$\leq \int \sup_{y\in\mathcal{Y}} |\langle c(y), V(g_*(x) - g(x))\rangle| d\rho_{\mathcal{X}}(x). \tag{10.70}$$

Thus, combining with A, we conclude:

$$\mathcal{E}(f) - \mathcal{E}(f^*) \leq 2 \int \sup_{y\in\mathcal{Y}} |\langle c(y), V(g_*(x) - g(x))\rangle| d\rho_{\mathcal{X}}(x). \tag{10.71}$$

Applying Cauchy-Schwarz and Jensen's Inequality

Applying Cauchy-Schwarz:

$$\sup_{y\in\mathcal{Y}} |\langle c(y), V(g_*(x) - g(x))\rangle| \leq \sup_{y\in\mathcal{Y}} \|V^* c(y)\|_{\mathcal{H}} \|g_*(x) - g(x)\|_{\mathcal{H}}. \tag{10.72}$$

Since:

$$\sup_{y\in\mathcal{Y}} \|V^* c(y)\|_{\mathcal{H}} = \sup_{y\in\mathcal{Y}} \|c(y)\|_{\mathcal{H}} = Q, \tag{10.73}$$

we obtain:

$$\mathcal{E}(f) - \mathcal{E}(f^*) \leq 2Q\|V\| \int \|g_*(x) - g(x)\| d\rho_{\mathcal{X}}(x). \tag{10.74}$$

*Applying **Jensen's inequality**:*

$$\int \|g_*(x) - g(x)\| d\rho_{\mathcal{X}}(x) \leq \sqrt{\mathcal{R}(g) - \mathcal{R}(g_*)}. \tag{10.75}$$

Thus, we conclude that:

$$\mathcal{E}(f) - \mathcal{E}(f^*) \leq 2Q\|V\| \sqrt{\mathcal{R}(g) - \mathcal{R}(g_*)}. \tag{10.76}$$

10.4.3 Universal Consistency

Theorem 10.4.2 (Universal Consistency)**.** *Let $\mathcal{X}, \mathcal{Y}$ be compact, and let ℓ admit an implicit embedding. Suppose $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a universal kernel*. Choose $\lambda = n^{-1/2}$ to train f_n. Then:*

$$\lim_{n\to\infty} \mathcal{E}(f_n) - \mathcal{E}(f^*) = 0, \tag{10.77}$$

with probability 1.

10.4.4 Learning Rates

Theorem 10.4.3 (Learning Rates)**.** *Let $\mathcal{X}, \mathcal{Y}$ be compact, and let ℓ admit an implicit embedding. Use $\lambda = n^{-1/2}$ to train f_n. Then, for all $\delta \in (0, 1)$,*

$$\mathcal{E}(f_n) - \mathcal{E}(f^*) \leq Q\|V\| \log(1/\delta) \frac{1}{n^{1/4}}, \tag{10.78}$$

holds with probability at least $1 - \delta$.

Comments:

- *Same rates as worst-case binary classification.*
- *Better rates are achievable with Tsibakov-like noise assumptions.*

*Technical requirement: Use, e.g., the Gaussian kernel $k(x, x') = e^{-\|x-x'\|^2/\sigma}$.

10.4.5 Final Evaluation - Wish List

Going back to our wishlist, this strategy:

- *Is **flexible**: can be applied to (m)any $\mathcal{Y}$ and ℓ.* ✓
- *Leads to efficient **computations**:*
  - *Yes in training,*✓
  - *It depends on $\mathcal{Y}$ and ℓ at test time.*
- *Has strong **theoretical** guarantees (i.e., consistency, rates)* ✓.